Programmation Objet Avancée

David Navarre

david.navarre@ut-capitole.fr

面向对象编程高级

大卫·纳瓦雷
david.navarre@ut-capitole.fr

Objectifs

- Connaître les design patterns (patrons de conception) classiques
- Utiliser des design patterns pour améliorer la maintenabilité du code

目标

- 掌握经典设计模式
- 运用设计模式以提升代码的可维护性

Introduction aux Design Patterns

- Un concept pour augmenter la réutilisabilité du logiciel orienté-objet
- Donner des archétypes de solution à des problèmes récurrents dans la conception de logiciel
- Solutions issues de l'expérience dans des projets réels, validées par l'usage
  - « On n'invente pas un Pattern, on le découvre »

设计模式入门

- 一种提升面向对象软件可复用性的概念
- 为软件设计中常见的问题提供解决方案范式
- 将实际项目中的实践经验转化为解决方案，并经实际应用验证。
  - “模式并非被发明出来的，而是被发现的。”

Inspiration Architecturale

- « Un pattern décrit un problème qui se présente fréquemment dans notre environnement, et décrit l'essence d'une solution à ce problème, de telle sorte que l'on puisse réutiliser cette solution un million de fois sans jamais la refaire deux fois de la même manière »
  - Christopher Alexander.

A pattern language, town, buildings, constructions, avec Sarah Ishikawa et Murray Silverstein, 1977, Oxford University Press

建筑灵感

- “一种模式描述了在我们的环境中频繁出现的问题，并阐述了解决该问题的核心方案，使得该方案可被重复使用一百万次，而无需以相同方式重复构建两次。”
  - 克里斯托弗·亚历山大。

*一种模式语言、城镇、建筑与构筑物，*与 Sarah 共同完成
Ishikawa 与 Murray Silverstein，1977，牛津大学出版社

Bibliographie sur les Patterns

- Design Patterns, Elements of Reusable Object-Oriented Software
  - E.Gamma, R. Helm, R. Johnson, J. Vlissides, Addison-Wesley, le «Gang of Four», 1995
- A System of Patterns : Pattern-oriented software architecture
  - Buschmann, F., R. Meunier, H. Rohnert, P. Sommerlad, and M. A Stal, Wiley 1996
- Smalltalk-80: the language and its implementations
  - Goldberg, A. and Robson, D.. Addison Wesley; 1983.
- Java

模式相关参考文献

- 设计模式——可复用面向对象软件的基础要素
  - E. Gamma，R. Helm，R. Johnson，J. Vlissides，Addison-Wesley，《Gang of Four》，1995
- 模式体系：面向模式的软件架构
  - Buschmann，F.、R. Meunier、H. Rohnert、P. Sommerlad 及 M. A. Stal，Wiley，1996
- Smalltalk-80：该语言及其实现
  - Goldberg，A. 与 Robson，D.，《Addison Wesley》；1983年。
- Java

Éléments d’un Pattern

- Nom du pattern
  - augmenter le vocabulaire du design, augmenter le niveau d'abstraction
- Description du problème
  - contexte d'application
- Solution
  - en termes d'un modèle OO générique
- Conséquences
  - avantages, inconvénients, compromis

模式构成要素

- 模式名称
  - 拓展设计词汇量，提升抽象思维能力
- 问题描述
  - 应用场景
- 解决方案
  - 就通用面向对象模型而言
- 后果
  - 优点、缺点及折中方案

Les patterns ne sont pas :

- Des bibliothèques de classes réutilisables
  - listes chaînées, arbres, …
- Des applications toutes faites
  - traitent uniquement un sous-problème bien défini
- Des classes génériques (templates)

这些模式并非：

- 可复用类库
  - 链表、树等
- 现成的应用程序
  - 仅处理一个定义明确的子问题
- 通用类（模板）

Catégories de Patterns

- Créationnels
  - Solutions pour l'instanciation des objets
  - Ex : Singleton, prototype, factory, …
- Structurels
  - Solutions pour la composition des classes ou objets
  - Ex : Proxy, façade, Adapter, …
- Comportementaux
  - Solutions pour l'interaction et la répartition des responsabilités entre les classes ou objets
  - Ex : Itérateur, visiteur, state, …

模式类别

- 创意类
  - 对象实例化解决方案
  - 示例：单例、原型、工厂模式……
- 结构
  - 类或对象构造解决方案
  - 示例：代理、门面、适配器等
- 行为学
  - 用于类或对象之间交互及责任分配的解决方案
  - 示例：Itérateur、visiteur、state 等

Les design patterns de références

Creational	Structural	Behavioral
Abstract Factory	Adapter	Chain of Responsability
Builder	Bridge	Command
Factory Method	Composite	Interpreter
Prototype	Decorator	Iterator
Singleton	Facade	Mediator
	Flyweight	Memento
	Proxy	Observer
		State
		Strategy
		Template Method
		Visitor

© Clever Age

参考设计模式

Creational	Structural	Behavioral
Abstract Factory	Adapter	Chain of Responsability
Builder	Bridge	Command
Factory Method	Composite	Interpreter
Prototype	Decorator	Iterator
Singleton	Facade	Mediator
	Flyweight	Memento
	Proxy	Observer
		State
		Strategy
		Template Method
		Visitor

© Clever Age

D’autres patterns et classification

10

Buschmann F., Henney K., Schmidt D.C., Pattern-Oriented Software Architecture, A Pattern Language for Distributed Computing, Vol 4, Wiley Series, 2007

其他模式与分类10

Buschmann F.、Henney K.、Schmidt D.C.，《面向模式的软件架构——分布式计算模式语言》，第 4 卷，Wiley 系列，2007

Architectural Patterns

- From Mud To Structure
  - Domain Model, Layers, Model-View-Controller, Presentation-Abstraction-Control, Microkernel, Reflection, Pipes and Filters, Shared Repository, Blackboard, Domain Object
- Distribution Infrastructure
  - Messaging, Message Channel, Message Endpoint, Message Translator, Message Router, Publisher-Subscriber, Broker, Client Proxy, Requestor, Invoker, Client Request Handler, Server Request Handler
- Event Demultiplexing and Dispatching
  - Reactor, Proactor, Acceptor-Connector, Asynchronous Completion Token
- Interface Partitioning
  - Explicit Interface, Extension Interface, Introspective Interface, Dynamic Invocation Interface, Proxy, Business Delegate, Facade, Combined Method, Iterator, Enumeration Method, Batch Method
- Component Partitioning
  - Encapsulated Implementation, Whole-Part, Composite, Master-Slave, Half-Object plus Protocol, Replicated Component Group
- Application Control
  - Page Controller, Front Controller, Application Controller, Command Processor, Template View, Transform View, Firewall Proxy, Authorization
- Concurrency
  - Half-Sync/Half-Async, Leader/ Followers, Active Object, Monitor Object

架构模式

- 从泥泞到结构
  - 领域模型、分层架构、模型-视图-控制器、呈现-抽象-控制、微内核、反射、管道与过滤器、共享存储库、Blackboard、领域对象
- 分布基础设施
  - 消息传递、消息通道、消息端点、消息转换器、消息路由器、发布方-订阅方、代理服务器、客户端代理、请求发起方、调用方、客户端请求处理器、服务器请求处理器
- 事件解复用与分发
  - 反应器、发起器、接收器-连接器、异步完成令牌
- 接口分区
  - 显式接口、扩展接口、自省接口、动态调用接口、代理、业务委托、门面、组合方法、迭代器、枚举方法、批量方法
- 组件划分
  - 封装式实现、整体-部分架构、复合架构、主从架构、半对象加协议架构、复制组件组
- 应用控制
  - 页面控制器、前端控制器、应用控制器、命令处理器、模板视图、转换视图、防火墙代理、授权
- 并发
  - 半同步/半异步、领导者/跟随者、活跃对象、监控对象

Architectural Patterns

- Synchronization
  - Guarded Suspension, Future, Thread-Safe Interface, Double-Checked Locking, Strategized Locking , Scoped Locking, Thread-Specific Storage, Copied Value, Immutable Value
- Object Interaction
  - Observer, Double Dispatch, Mediator, Command, Memento, Context Object, Data Transfer Object, Message
- Adaptation and Extension
  - Bridge, Object Adapter, Chain of Responsibility, Interpreter, Interceptor, Visitor, Decorator, Execute-Around Object, Template Method, Strategy, Null Object, Wrapper Facade, Declarative Component Configuration
- Modal Behavior
  - Objects for States, Methods for States, Collections for States
- Resource Management
  - Container, Component Configurator, Object Manager, Lookup, Virtual Proxy, Lifecycle Callback, Task Coordinator, Resource Pool, Resource Cache, Lazy Acquisition, Eager Acquisition, Partial Acquisition, Activator, Evictor, Leasing, Automated Garbage Collection, Counting Handle, Abstract Factory, Builder, Factory Method, Disposal Method
- Database Access
  - Database Access Layer, Data Mapper, Row Data Gateway, Table Data Gateway, Active Record

架构模式

- 同步
  - 受控挂起、未来版本、线程安全接口、双重检查锁、策略性锁、作用域锁、线程特定存储、值拷贝、不可变值
- 对象交互
  - 观察者、双重委托、调解者、命令、备忘录、上下文对象、数据传输对象、消息
- 适配与扩展
  - 桥式、对象适配器、责任链、解释器、拦截器、访问者、装饰器、环绕式执行对象、模板方法、策略、空对象、封装式门面、声明式组件配置
- 模态行为
  - 状态对象、状态方法、状态集合
- 资源管理
  - 容器、组件配置器、对象管理器、查找机制、虚拟代理、生命周期回调、任务协调器、资源池、资源缓存、延迟获取、立即获取、部分获取、激活器、驱逐器、租赁机制、自动垃圾回收、计数句柄、抽象工厂、构建器、工厂方法、处置方法
- 数据库访问
  - 数据库访问层、数据映射器、行数据网关、表数据网关、活跃记录

Ce qu’il faut retenir

13

Au moins au début

核心要点13

至少在开始时

Il y a beaucoup de patterns et ce n’est pas fini

- On ne peut pas tous les retenir
- Mais on doit les avoir parcourus au moins une fois

- Certains patterns sont déjà implémentés dans certains langages ou bibliothèques, ou sont construits autour
  - .net : Observer, Reactor, Iterator …
  - js : Proactor, Prototype, Iterator …
  - Java : Reflection, Introspective interface, Iterator …

- Face à un problème, le souvenir d’un pattern peut faciliter la conception et l’implémentation

存在许多模式，但这并非全部。

完成

- 我们无法全部留住。
- 但我们至少得把它们浏览一遍。

- 某些模式已在某些编程语言或库中实现，或基于这些基础构建而成。
  - .net：观察者、反应器、迭代器……
  - js：Proactor、Prototype、Iterator……
  - Java：反射、Introspective 接口、迭代器……

- 面对问题时，对某一模式的记忆可有助于设计及实现。

Pour la suite

- On va revenir à la classification d'origine
  - Créationnel, Structurel et Comportemental
- On va regarder en détail quelques patterns de ces catégories
- On va en assembler plusieurs pour résoudre un problème

继续阅读

- 我们将回归至原始分类体系。
  - 生成性、结构性与行为性
- 我们将详细分析这些类别中的若干模式。
- 我们将整合多个组件来解决一个问题。

Design Patterns Créationnels

Singleton

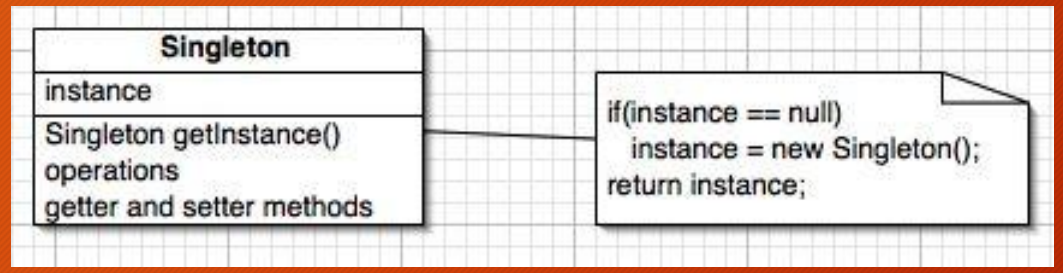
创造性设计模式

16

单例

Design Pattern “Singleton”

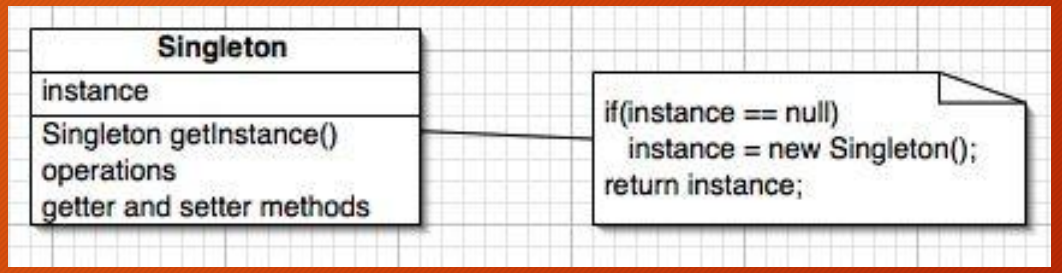
- Objectif : s'assurer qu'une classe donnée aura au plus une instance
- Grand Classique
- Utile pour créer des utilitaires partagés par plusieurs objets de classes différentes



- Par contre, des défauts
  - Non Thread safe a priori
  - Gestion de la suppression
  - Non configurable

设计模式“单例模式”

- 目标：确保给定类最多仅存在一个实例。
- 大古典
- 适用于创建由多个不同类实例共享的工具类。



- 然而，仍存在一些缺陷。
  - 默认不具备线程安全性
  - 删除管理
  - 不可配置

Design Pattern “Singleton”

- Implémentation classique
  - public class ClassicSingleton {
    private static ClassicSingleton instance = null;

    private ClassicSingleton() {
    // interdit l'instanciation.
    }
    public static ClassicSingleton getInstance() {
    if(null == instance) {
    instance = new ClassicSingleton();
    }
    return instance;
    }
    …
  - }

设计模式“单例模式”

- 经典实现
  - public class ClassicSingleton {
    private static ClassicSingleton instance = null;

    private ClassicSingleton() {
      // interdit l'instanciation.
    }
    public static ClassicSingleton getInstance() {
      if(null == instance) {
        instance = new ClassicSingleton();
      }
      return instance;
    }
    …
  - }

Design Patterns Créationnels

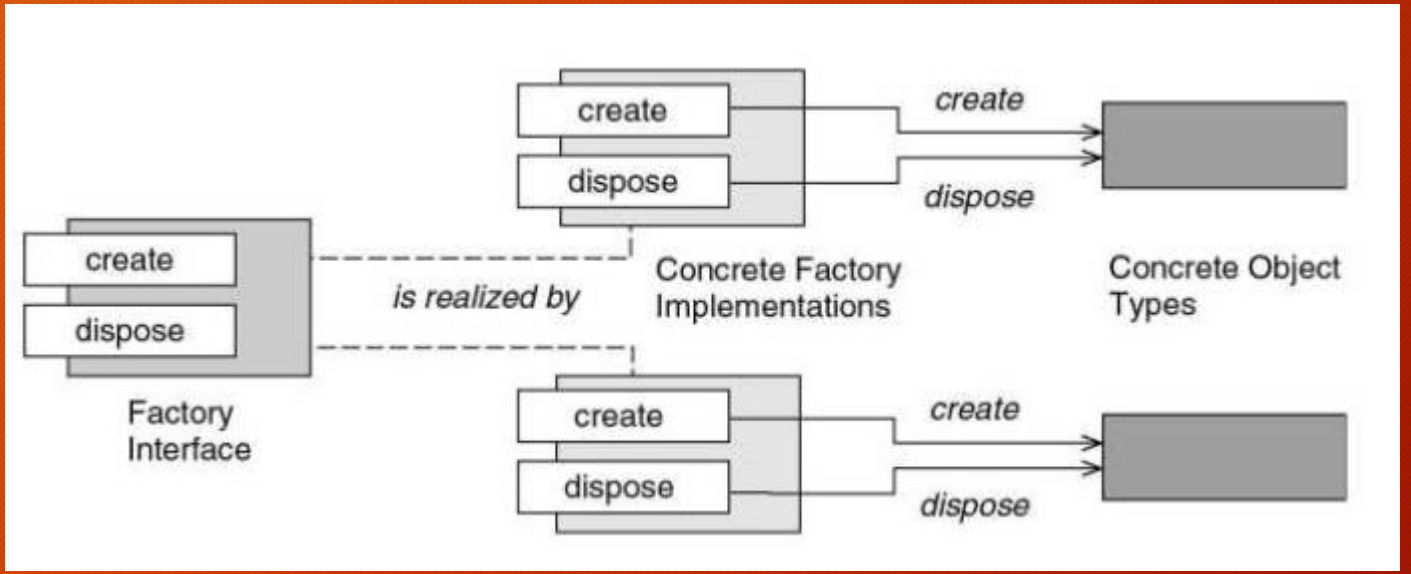
Abstract Factory

创造性设计模式

抽象工厂

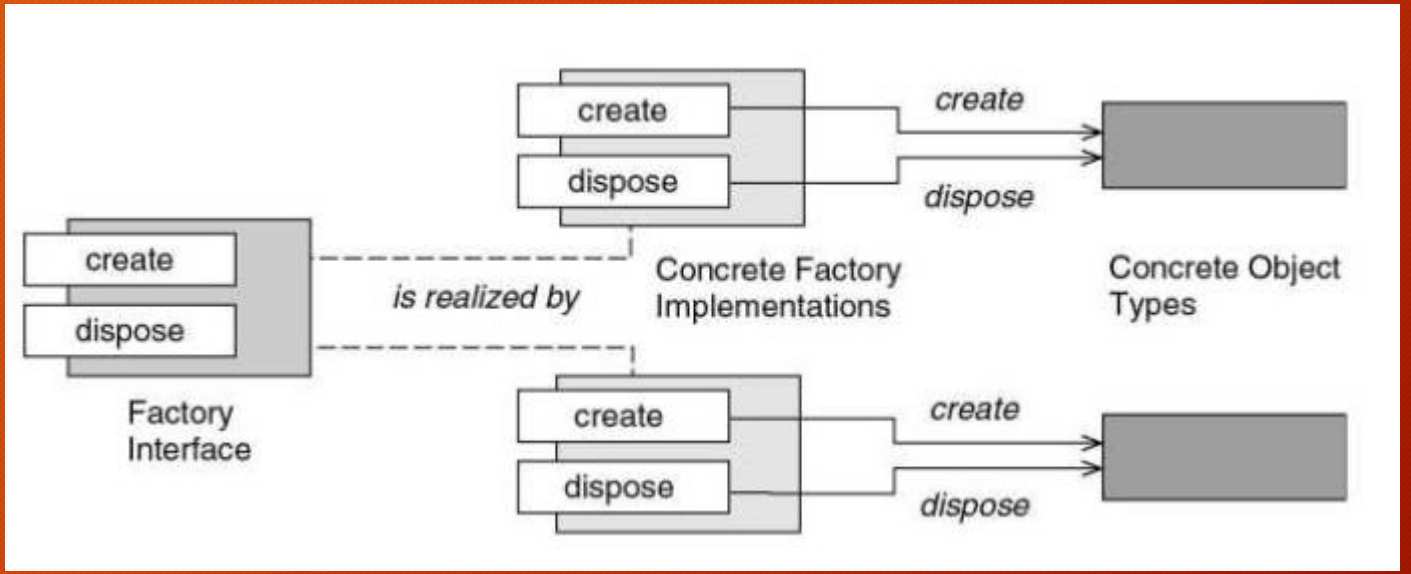
Design Pattern "Abstract Factory"

- Objectif
  - Fournit une interface pour créer des objets qui ont une relation sans spécifier leur classes concrètes



设计模式“抽象工厂”

- 目标
  - 提供一个接口，用于创建具有关联关系的对象，而无需指定其具体类。



Exemple d’un outil de dessin

- 3 classes sont responsables de représenter les dessins (Line, Circle, Rectangle)
- Les constructeurs sont différents
- Mais, on veut uniformiser les méthodes de création

- Solution : Abstract Factory (mixé avec Factory method)
- AbstractFactory contient une méthode de création assez large
  - Ex : Drawing createDrawing(Color aColor, Point… points)
- On crée des Factory concrètes pour Line, Rectangle, Circle, … en se basant sur le pattern « Singleton »
  - Class LineFactory implements AbstractFactory {
    - private static LineFactory defaultFactory = new LineFactory();
    - public static LineFactory getDefaultFactory() {
    - return defaultFactory;
    - }
    - public Drawing createDrawing(Color aColor, Point… points) {
      - return new Line(points[0], points[1], aColor);
    - }
  - }

绘图工具示例

- 共有 3 个类负责表示图形（线条、圆形、矩形）。
- 制造商各不相同。
- 然而，我们仍希望统一创作方法。

- 解决方案：抽象工厂模式（与工厂方法模式结合使用）
- AbstractFactory 包含一个功能广泛的创建方法。
  - 示例：绘制 createDrawing（颜色 aColor、点 ... 点）
- 我们基于“单例”模式，为 Line、Rectangle、Circle 等类创建具体的工厂类。
  - Class LineFactory 实现了 AbstractFactory 接口。
    - private static LineFactory defaultFactory = new LineFactory();
    - public static LineFactory getDefaultFactory() {
    - 返回 defaultFactory;
    - }
    - 公共方法 Drawing.createDrawing(Color aColor，Point... points) {
      - return new Line(points[0], points[1], aColor);
    - }
  - }

Design Patterns Créationnels

22

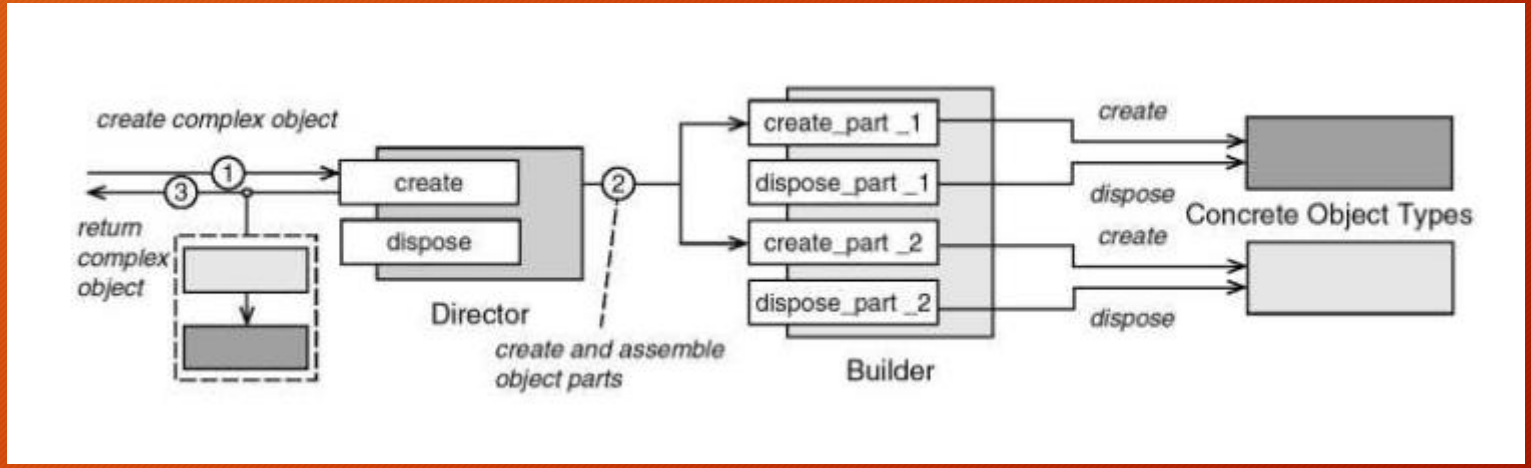
Builder

创造性设计模式

建造者

Design Pattern « Builder »

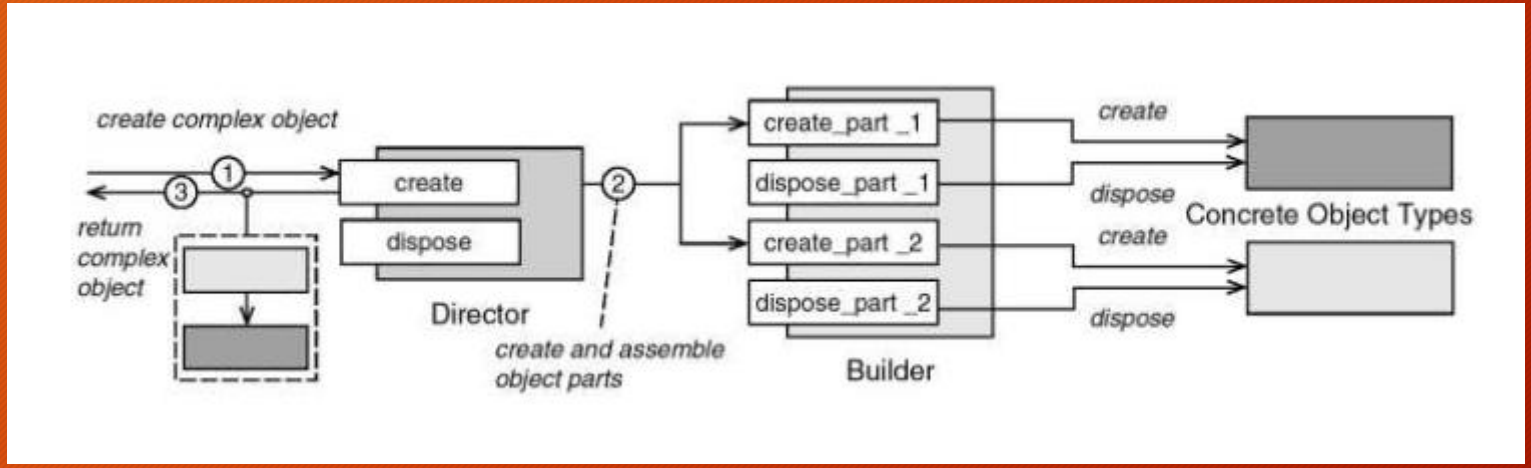
- Objectif
  - Support à la création d'objets complexes qui nécessitent plusieurs étapes
  - Evite le maintient de variables temporaires



- Exemple
  - StringBuilder en Java

设计模式「Builder」

- 目标
  - 支持创建需要多个步骤的复杂对象
  - 避免使用临时变量



- 示例
  - Java 中的 StringBuilder

Exemple d’un outil de dessin

- Un dessin se crée dans le temps par assemblage de formes
- Solution
  - Un builder combiné à des abstract factory

绘图工具示例

- 一幅画作是在时间中通过形状的组合而形成的。

- 解决方案
  - 一个构建器与抽象工厂组合使用

Design Patterns Structurels

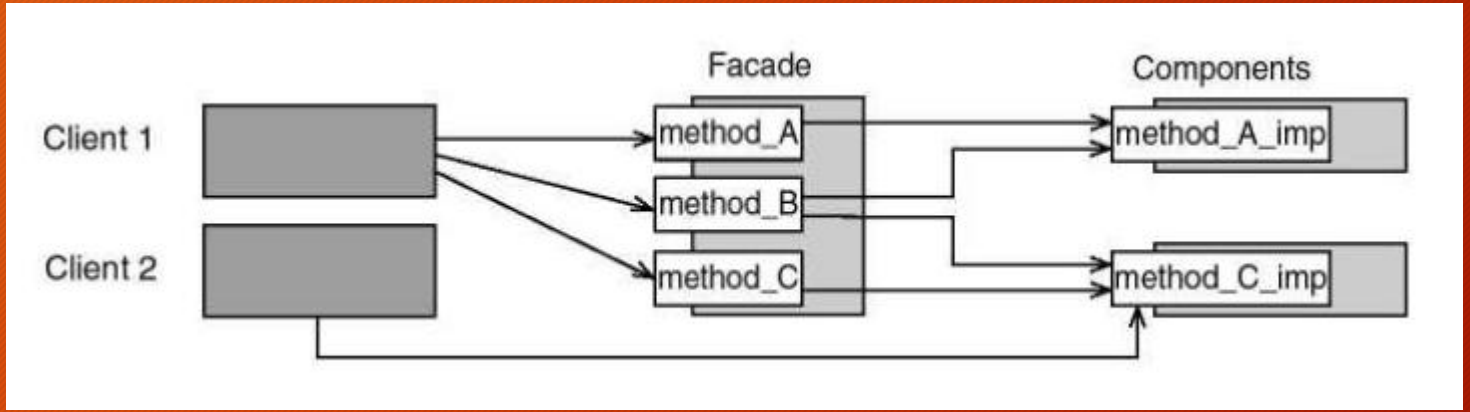
Façade

结构型设计模式

立面

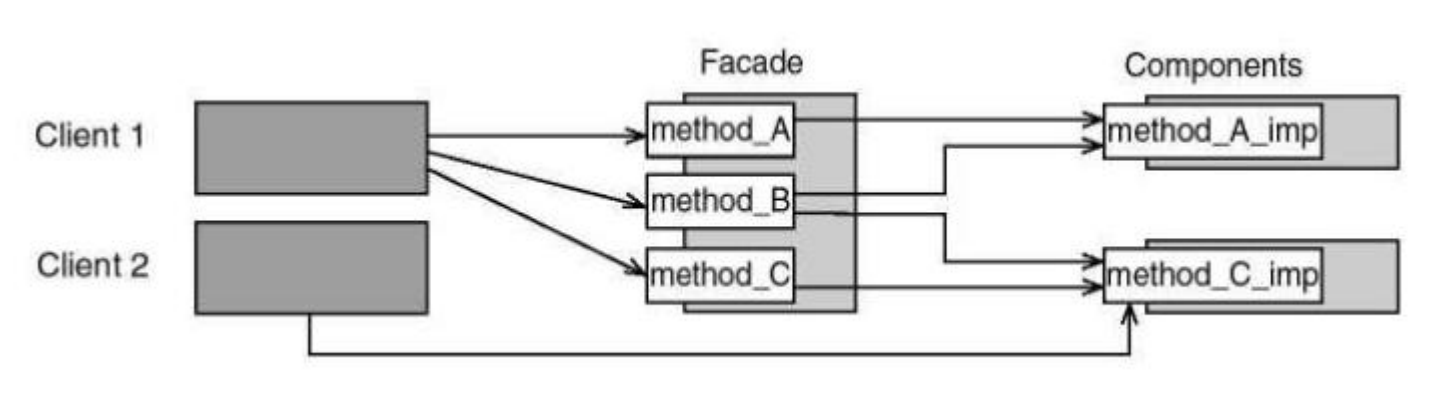
Design Pattern "Façade"

- Unifie et simplifie l'interface d'un sous-système
- Forme un point d'entrée simplifié dans une API
- Limite les points d'entrée d'une API
- Facilite l’utilisation d’un ensemble de classes



设计模式“façade”

- 统一并简化子系统的接口
- 在 API 中构建一个简化的入口点
- 限制 API 的入口点
- 简化类集合的使用



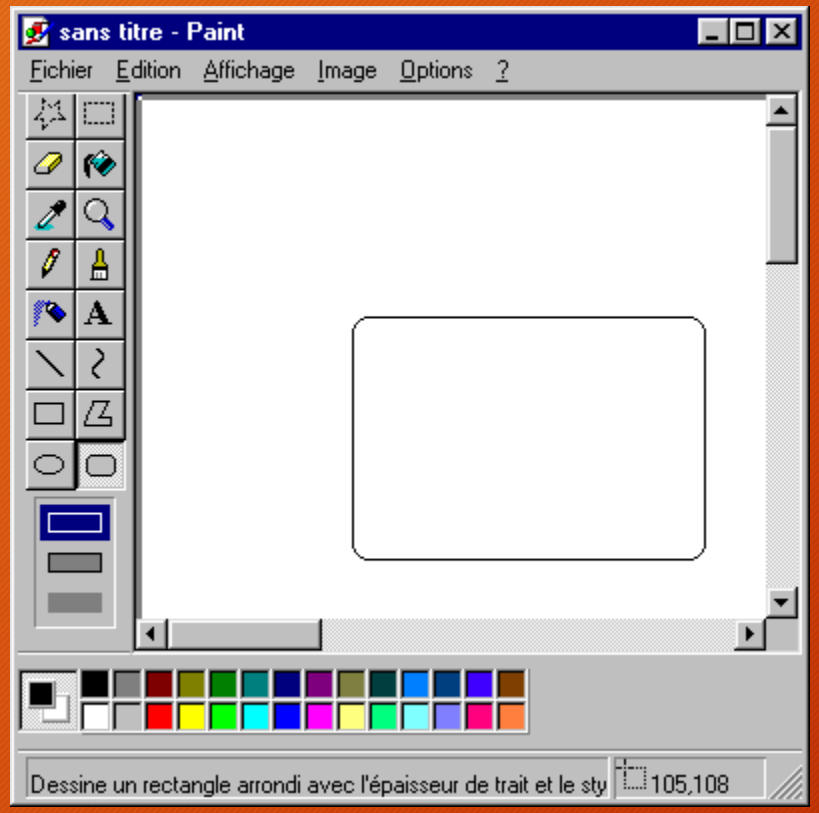
Design Patterns Comportementaux

State

行为设计模式

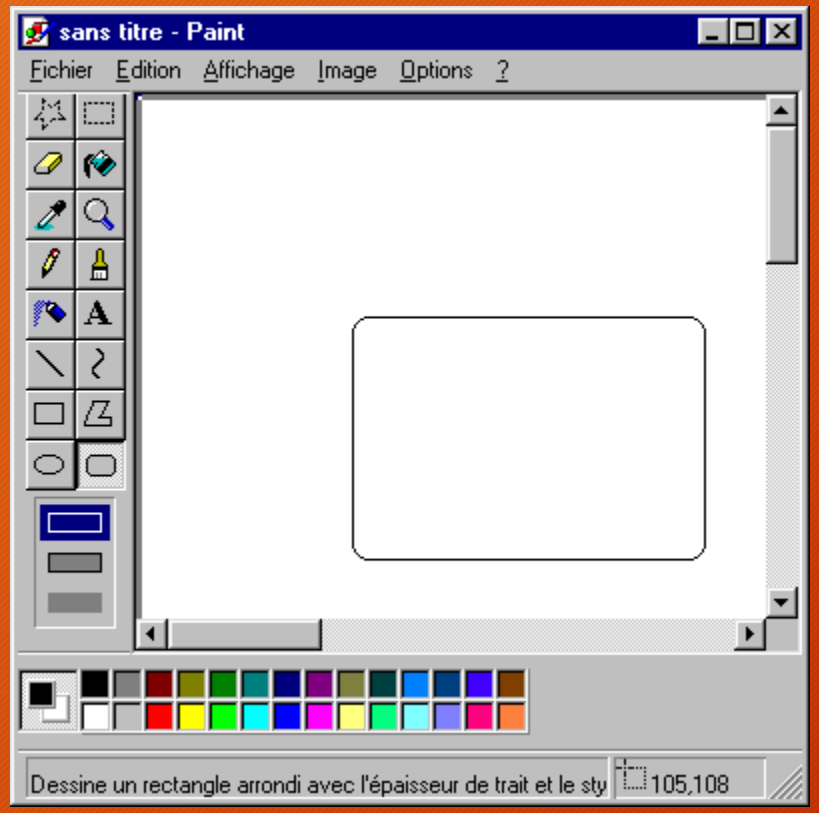
州

Le pattern État / Objet (State)



- Objectif : Permettre à un objet de changer de comportement quand son état interne change. L'objet se comporte comme s'il changeait de classe.
- Exemple typique : éditeur de dessin avec barre d'outils
  - « Modes d'opération » différents

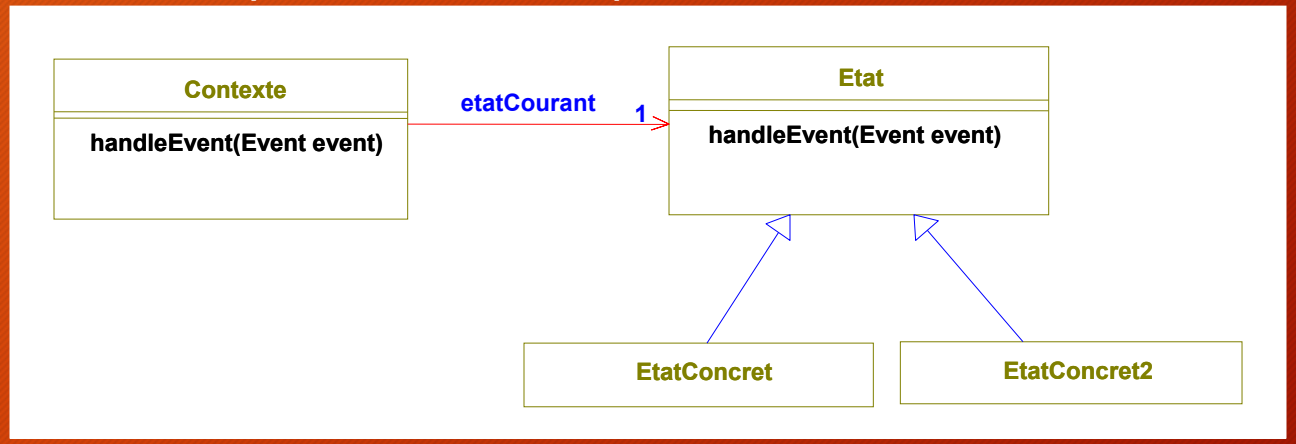
状态/对象模式 (State)



- 目标：使一个对象在内部状态发生变化时，能够改变其行为，使其表现得如同其类发生了变化一般。
- 典型示例：带有工具栏的绘图编辑器
  - “运行模式”各异

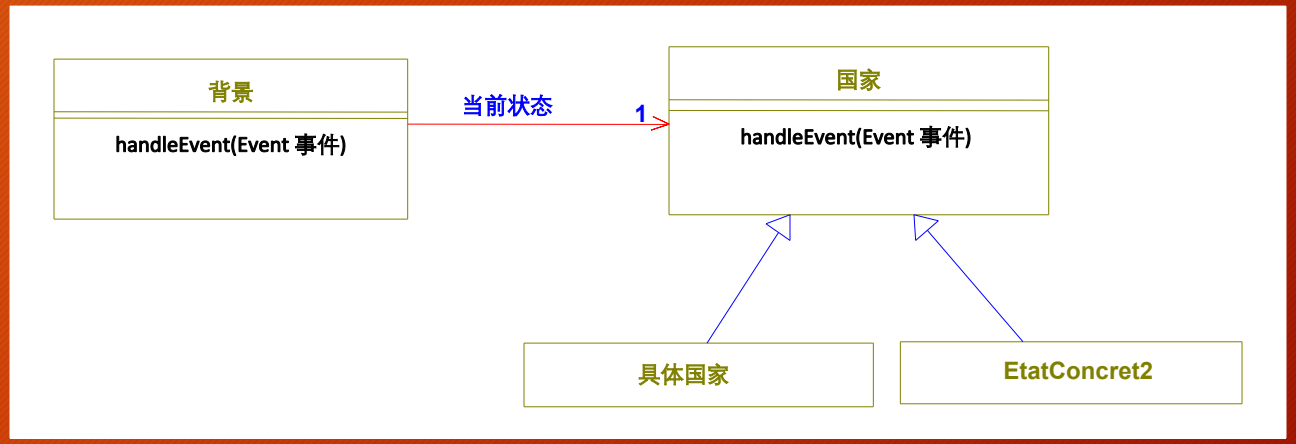
Structure générale

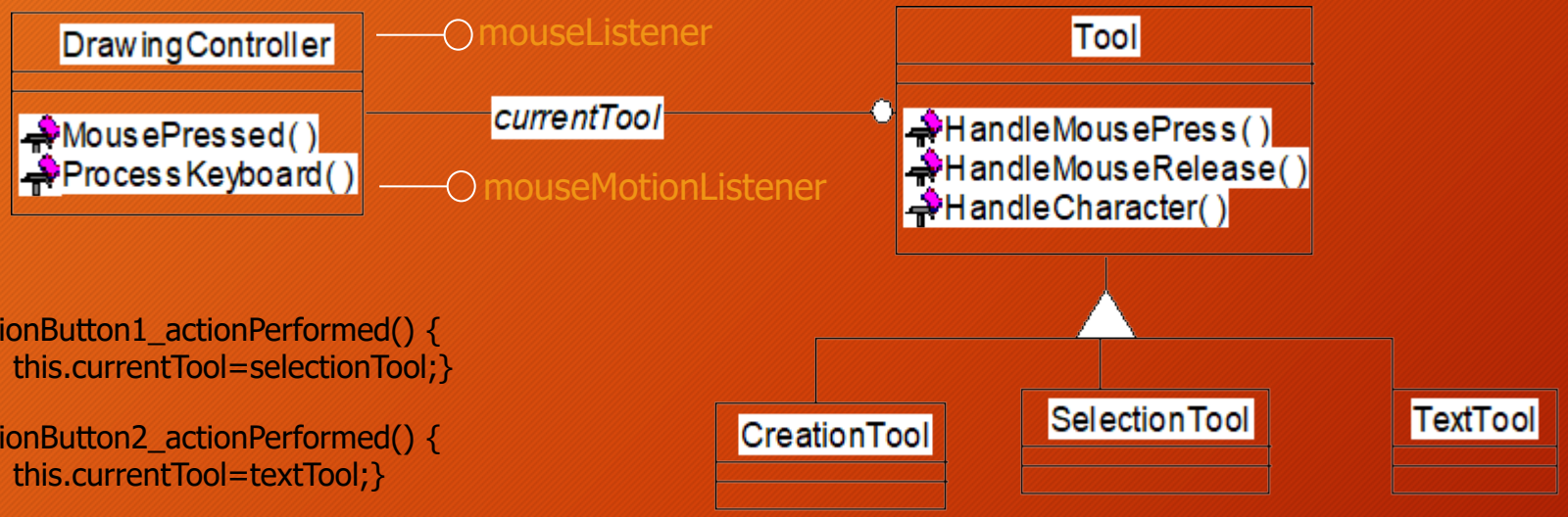
- Participants
  - Contexte :
    - définit l'interface utilisable par les clients
    - Maintient une instance de Etat qui définit l'état courant
  - Etat
    - Classe abstraite qui définit l'interface de chacun des états concrets
  - EtatConcret (classes dérivées)
    - Chaque sous-classe implémente le comportement effectif associé à un état du contexte



总体结构

- 参与者
  - 背景:
    - 定义了客户端可使用的接口
    - 维护一个定义当前状态的状态实例。
  - 国家
    - 一个抽象类，用于定义每个具体状态的接口。
  - EtatConcret（派生类）
    - 每个子类都实现了与特定上下文状态相关联的实际行为。






```
selectionButton1_actionPerformed() {
        this.currentTool=selectionTool;}

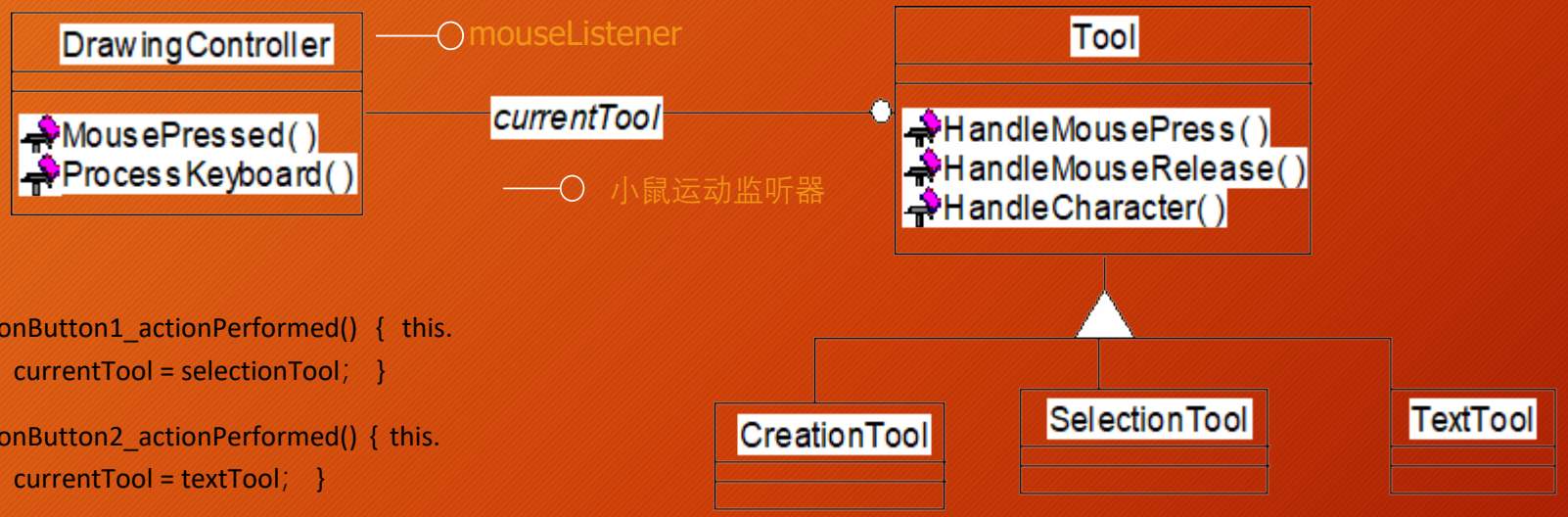
selectionButton2_actionPerformed() {
        this.currentTool=textTool;}
```

mouseListener

```
mousePressed() {
        currentTool.handleMousePress();}
mouseReleased() {
        currentTool.handleMouseReleased();}
mouseClicked() {
        currentTool.handleMouseClicked();}
mouseEntered() {
        currentTool.handleMouseEntered();}
mouseExited() {
        currentTool.handleMouseExited();}
```

mouseMotionListener

```
mouseMoved() {
        currentTool.handleMouseMove();
}
mouseDragged() {
        currentTool.handleMouseReleased();
}
```




```
selectionButton1_actionPerformed()  {  this.
        currentTool = selectionTool;   }

selectionButton2_actionPerformed()  {  this.
        currentTool = textTool;   }
```

mouseListener

```
mousePressed() {
        currentTool.handleMousePress();  }
mouseReleased() {
        currentTool.handleMouseReleased();  }
mouseClicked() {
        currentTool.handleMouseClicked();  }
mouseEntered() {
        currentTool.handleMouseEntered();  }
mouseExited() {
        currentTool.handleMouseExited();  }
```

鼠标运动监听器

```
mouseMoved() {
        currentTool.handleMouseMove();
}
mouseDragged() {
        currentTool.handleMouseReleased();
}
```

Conséquences

- Localise le comportement dépendant de l'état dans des sous-classes spécifiques
  - Évite la prolifération de clauses « if » ou « switch » dans le contrôleur
- Facilite l'introduction de nouveaux états
  - Définition d'une nouvelle sous-classe
  - Instanciation de l'objet/état
- Rend les changements d'états plus explicites

后果

- 在子类中定位与状态相关的行为
  特定的
  - 避免在控制器中大量使用“if”或“switch”语句。
- 促进新州的设立
  - 定义新子类
  - 对象/状态的实例化
- 使状态转换更加明确

Détails spécifiques

- Comment les changements d'états sont-ils contrôlés ?
  - Gérés par le contexte (le contexte implémente un automate à états)
  - Gérés par chaque sous-état
    - Chaque état connaît son (ses) état(s) suivants
    - implémentation « distribuée » d'une machine à états
- Comment sont instanciés les objets/états ?
  - Pré-instanciation
  - Instanciation dynamique selon les besoins

具体细节

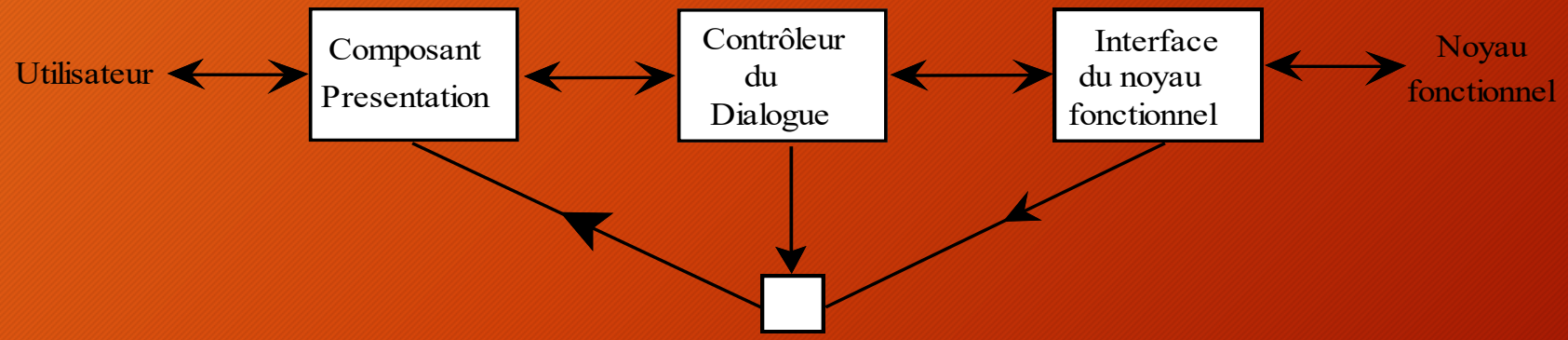
- 状态转换是如何控制的?
  - 由上下文驱动（该上下文实现了一个有限状态自动机）
  - 由各子状态管理
    - 每个状态都有其对应的以下状态。
    - 状态机的“分布式”实现
- 对象/状态是如何实例化的?
  - 预实例化
  - 按需动态实例化

Rappel : Architectures IHM classiques

提醒：经典人机交互界面架构

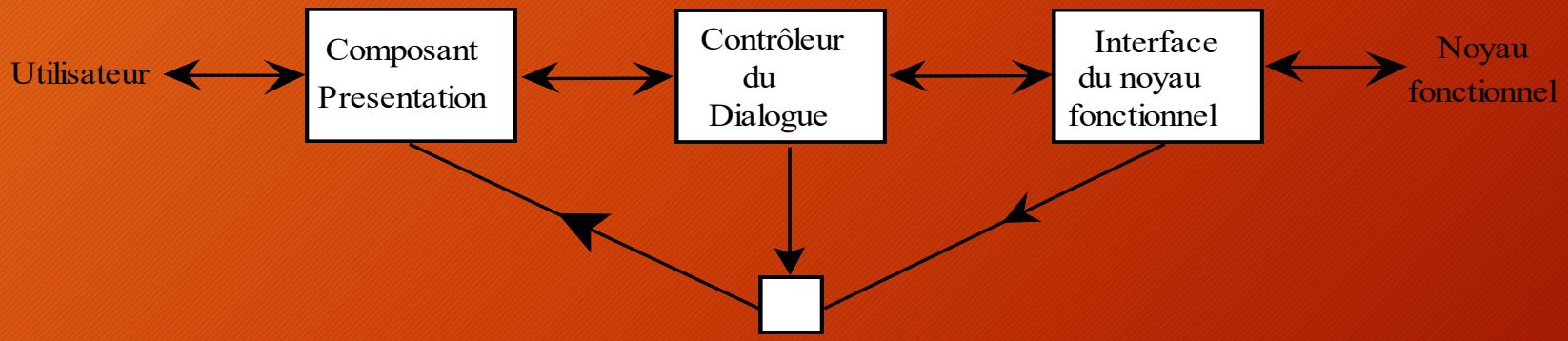
Seeheim 1983

- Pfaff, G. (ed). Seeheim Workshop on User Interface Management Systems. Eurographics Seminars, Springer-Verlag 1985.



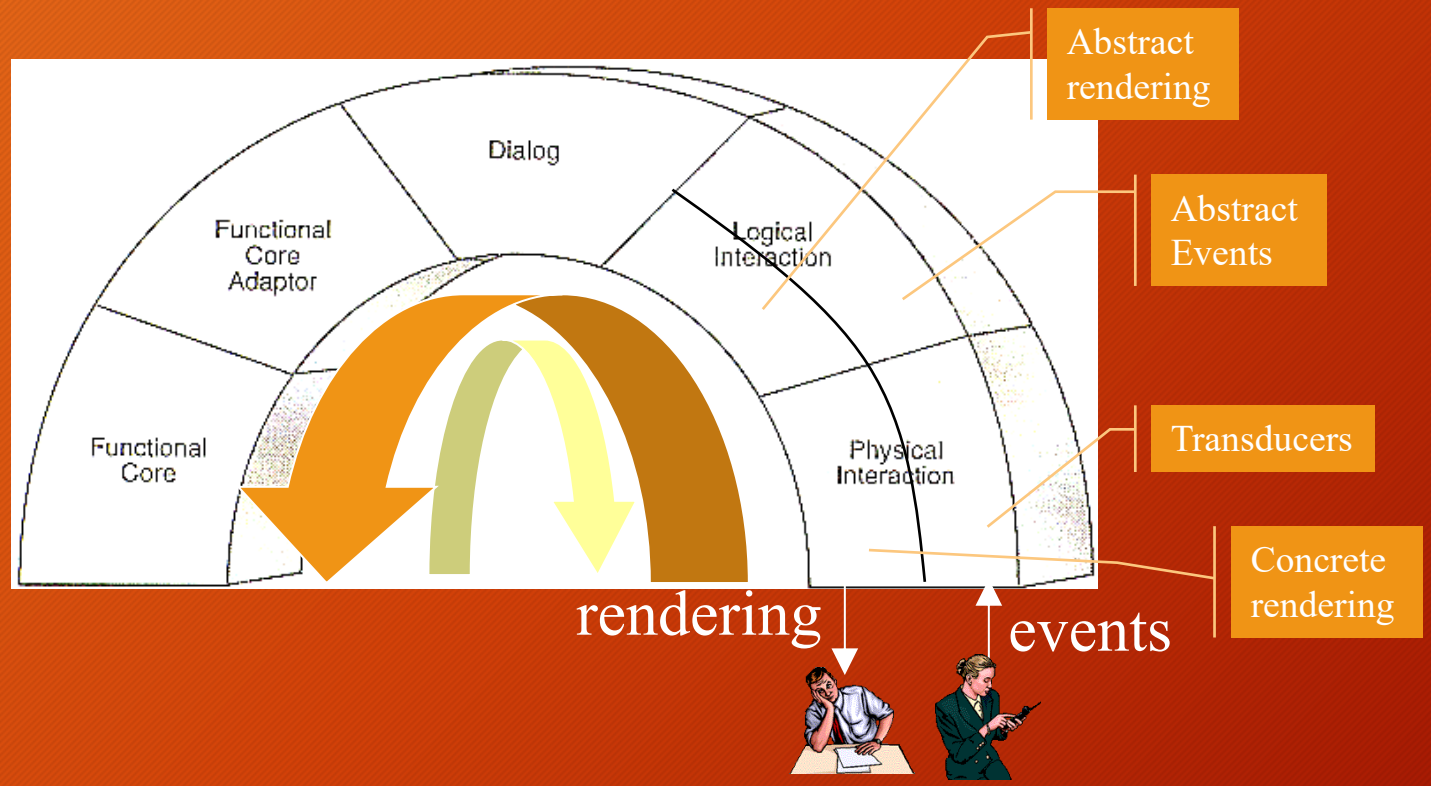
Seeheim，1983

- Pfaff，G.（编）：《Seeheim 用户界面管理系统研讨会》，Eurographics 系列研讨会，Springer-Verlag，1985。



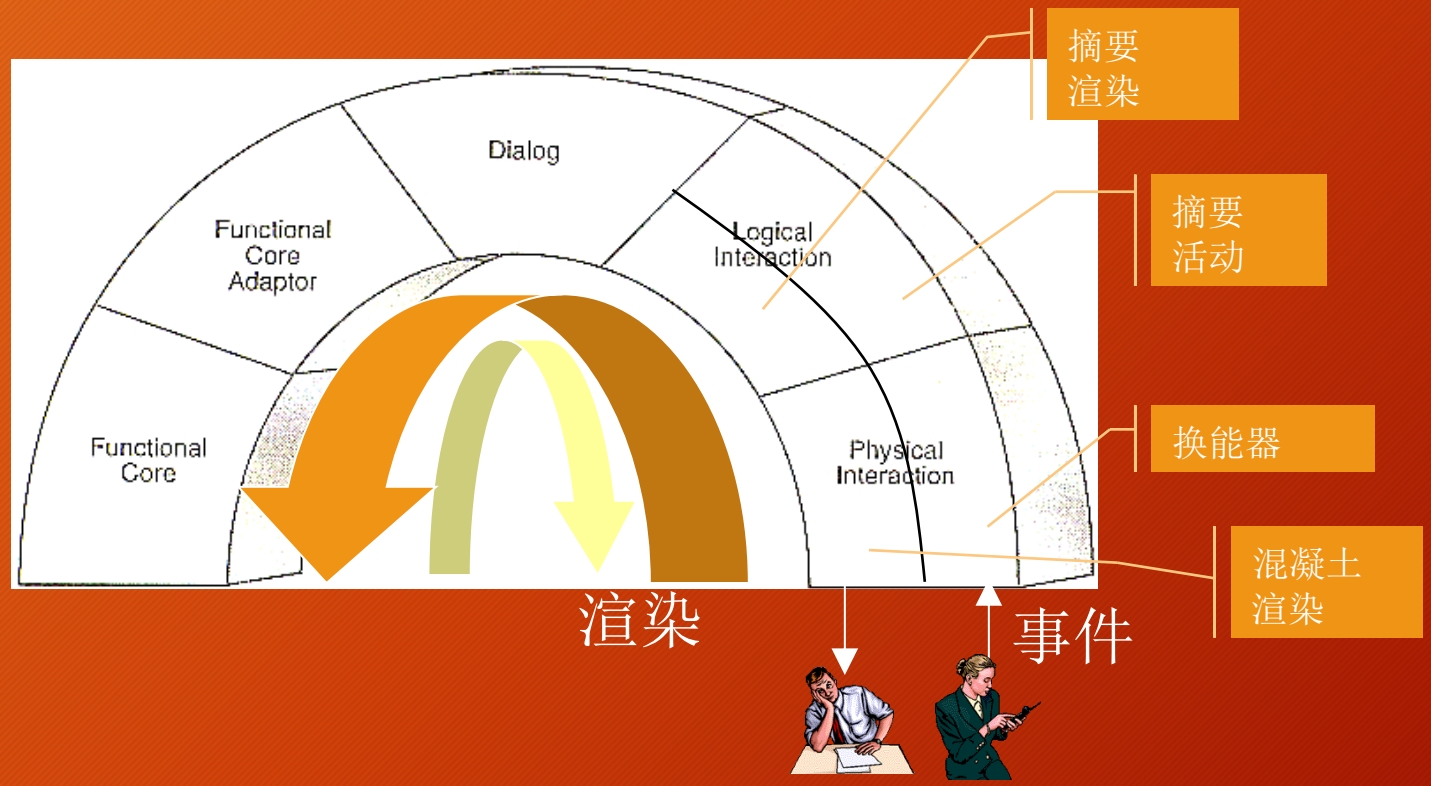
Arch/Slinky 1992

- UIMS Tool Developers Workshop. A Metmodel for the Runtime Architecture of an Interactive System. SIGCHI Bulletin 24(1), 1992.



Arch/Slinky 1992

- UIMS 工具开发者研讨会：面向交互式系统运行时架构的 Metmodel 模型。《SIGCHI 通报》第 24(1) 期，1992 年。



Avantages et Limites

- Approche modulaire
- Mise en avant du contrôleur de dialogue
- Possibilité de développer le noyau fonctionnel en parallèle de l'interface (métiers différents)

- Couplage fort entre les briques
- Duplication de données

优势与局限性

- 模块化方法
- 突出对话控制器
- 可在开发界面（不同业务领域）的同时，同步开发功能核心模块。

- 砖块间强结合
- 数据复制

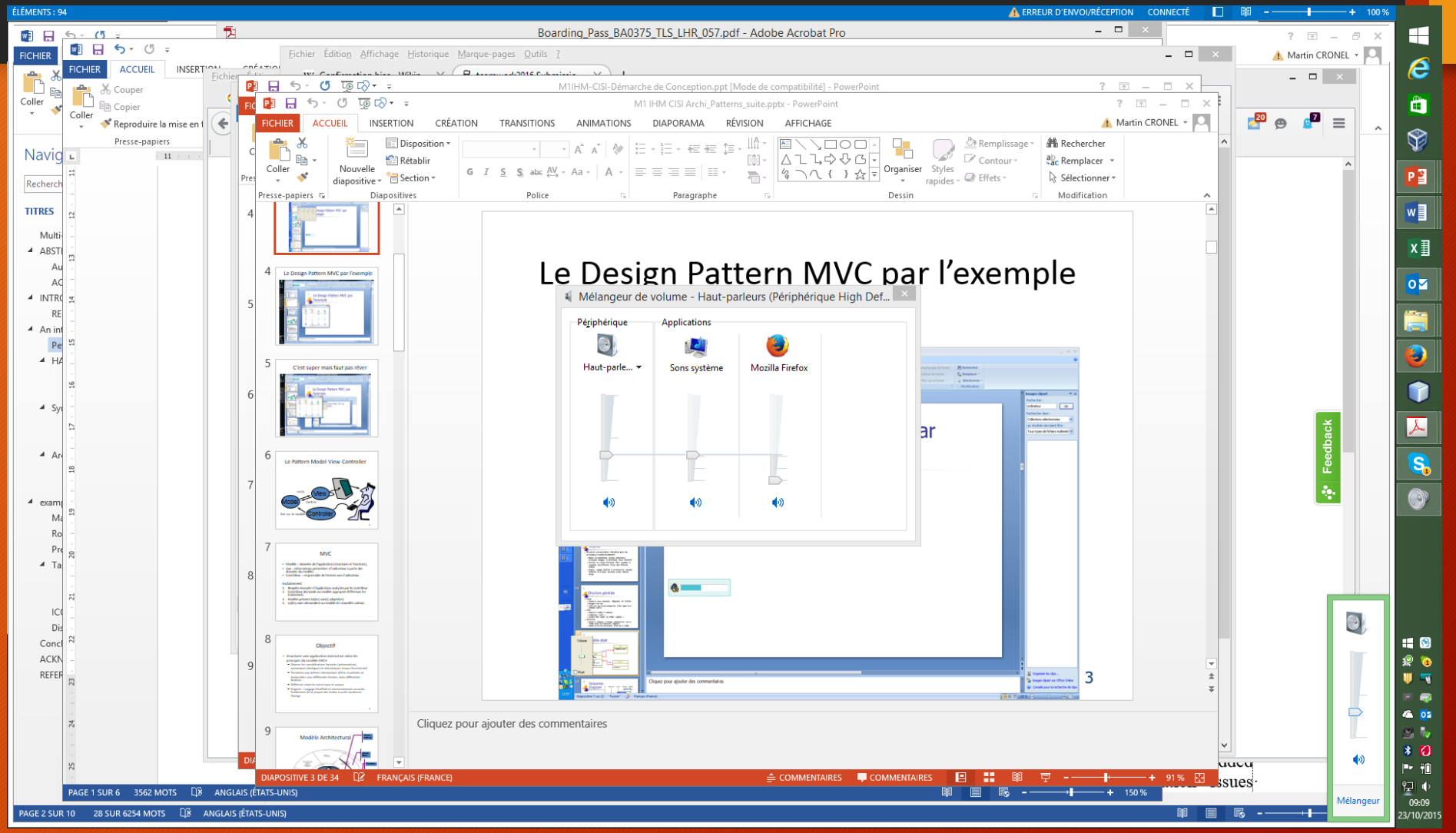
Design Patterns Comportementaux

MVC

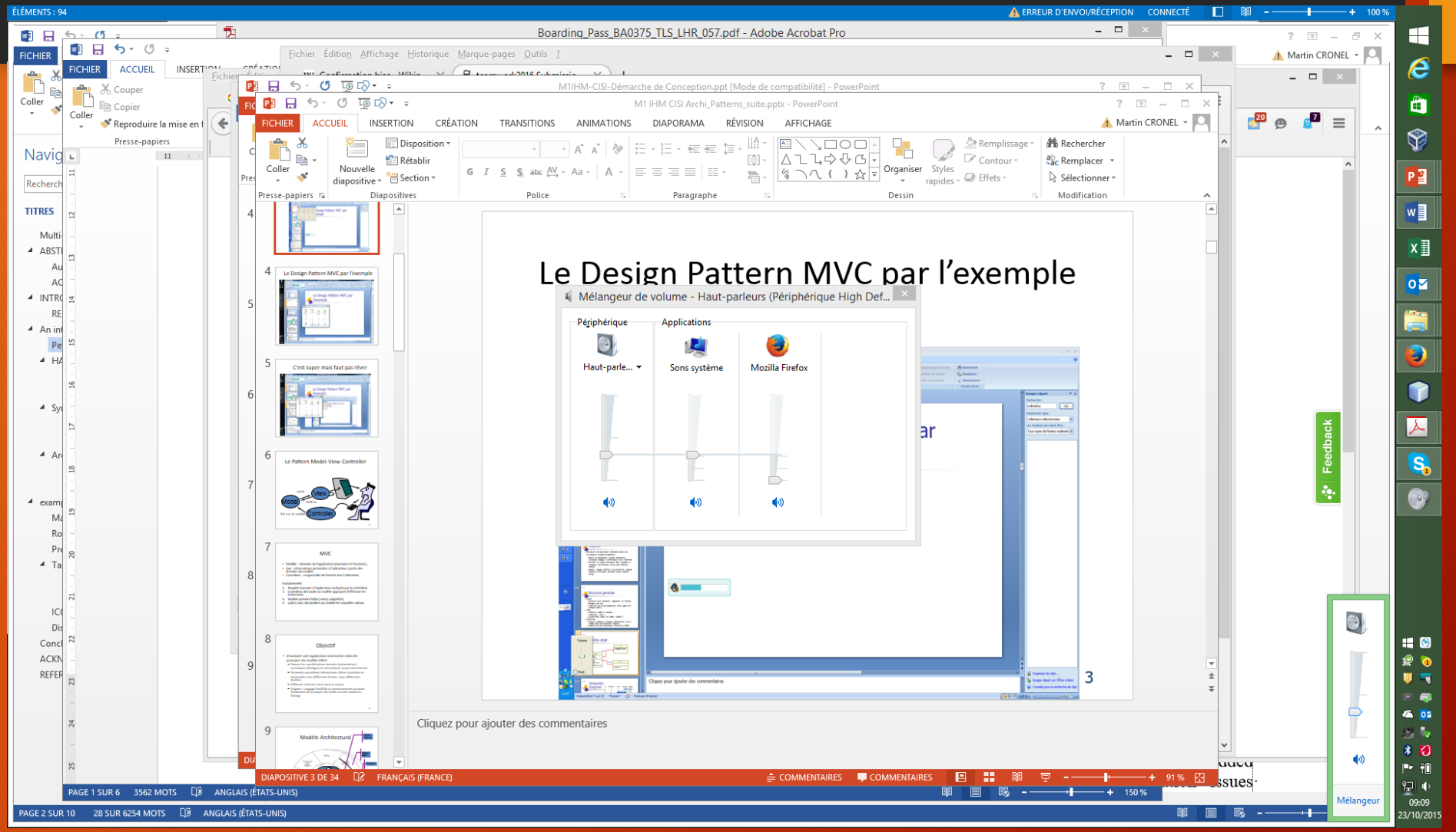
行为设计模式

MVC

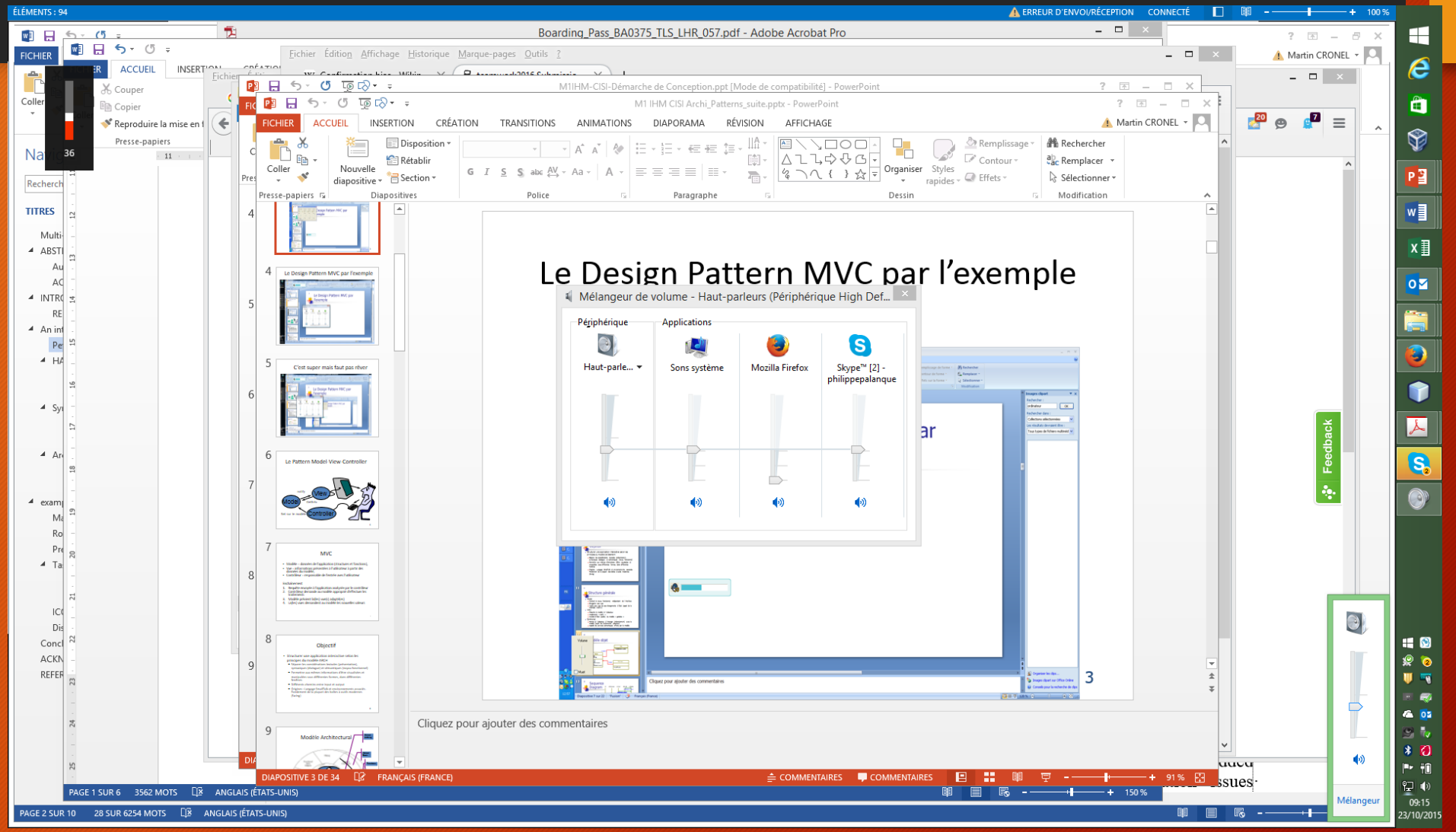
Le Design Pattern MVC par l'exemple



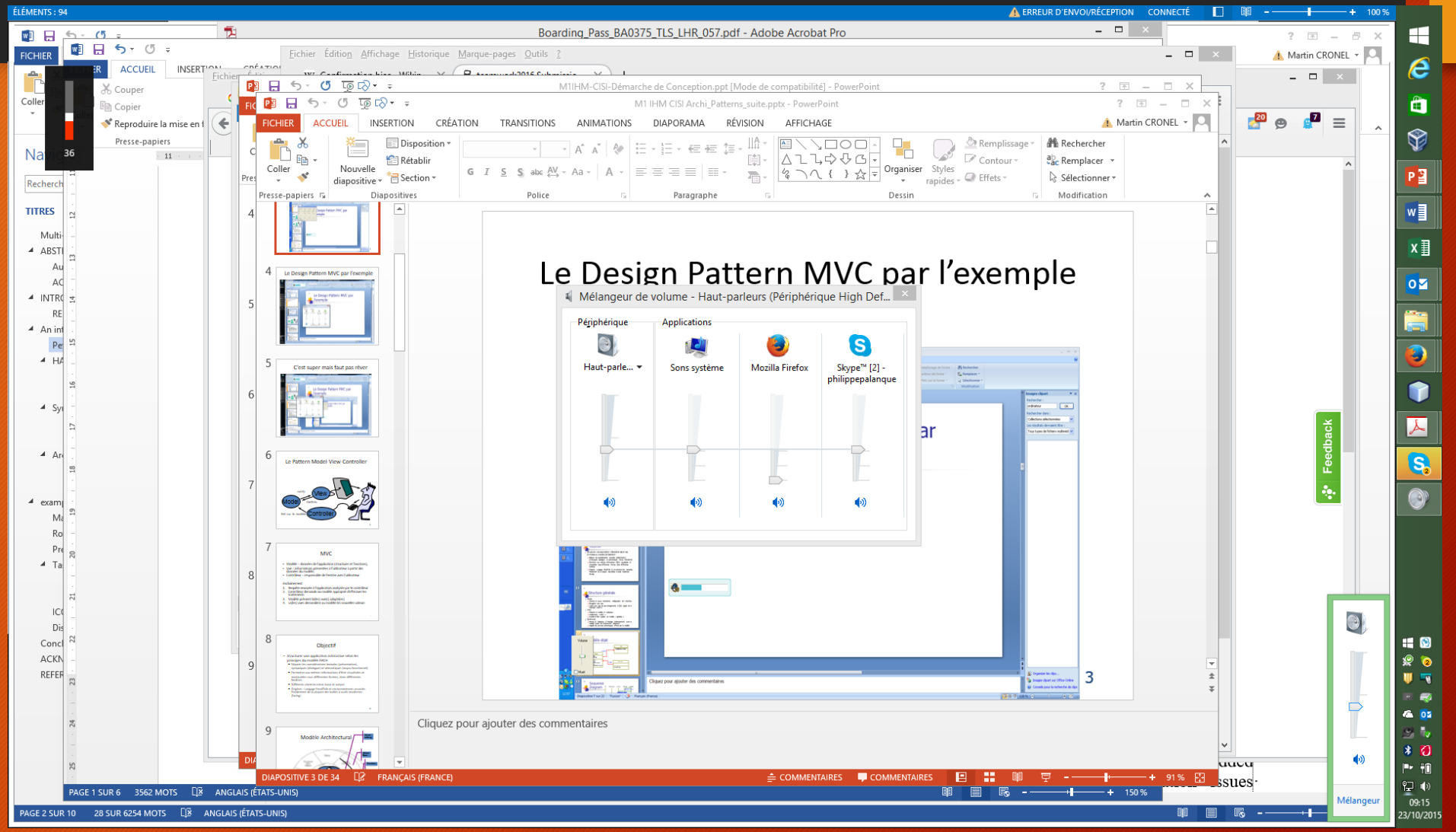
以 MVC 设计模式为例


Le Design Pattern MVC par l'exemple
Mélangeur de volume - Haut-parleurs (Périphérique High Def...
Périphérique
Applications
Haut-parle...
Sons système
Mozilla Firefox
Cliquez pour ajouter des commentaires
Feedback
Mélangeur

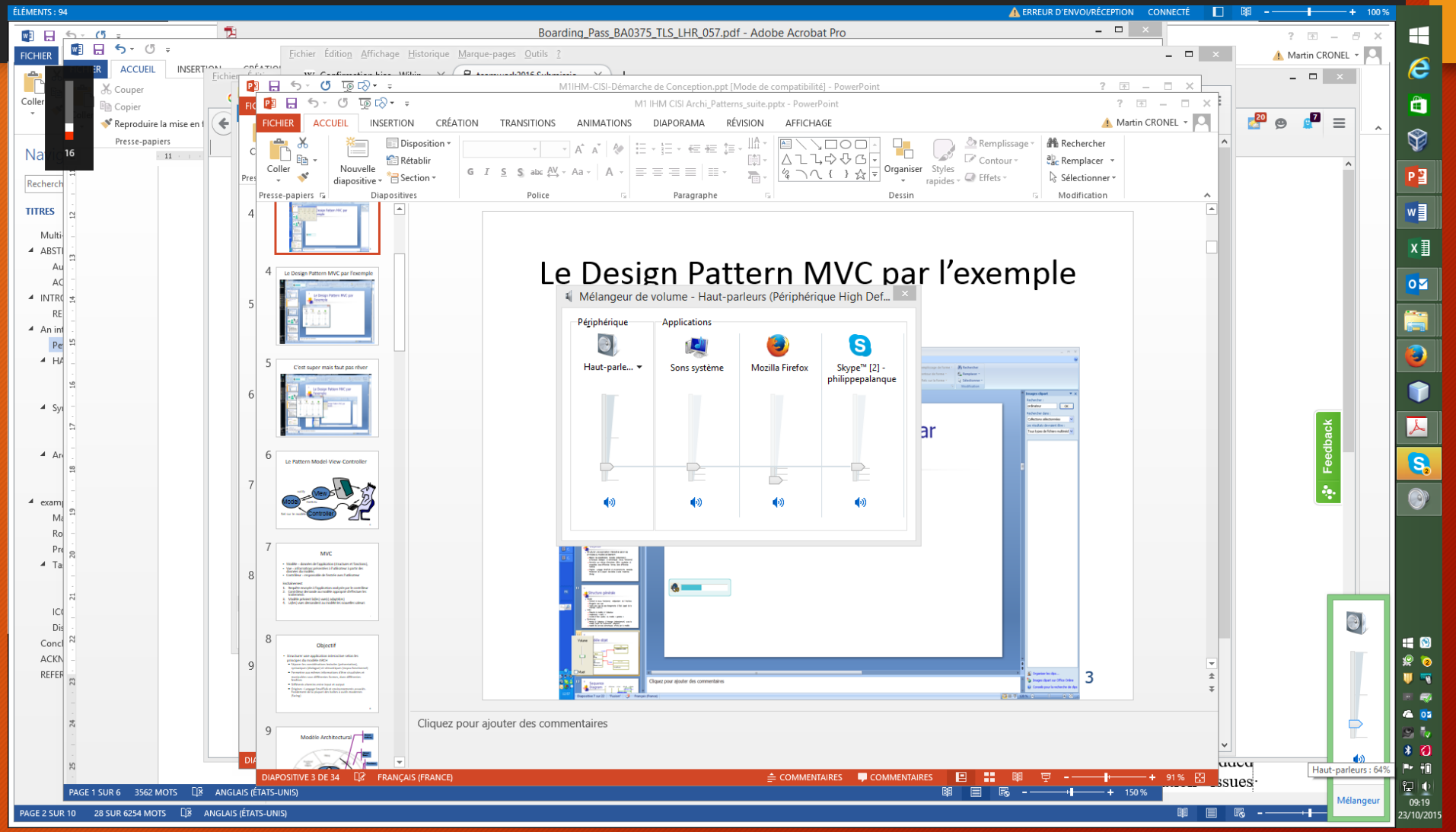
Le Design Pattern MVC par l'exemple



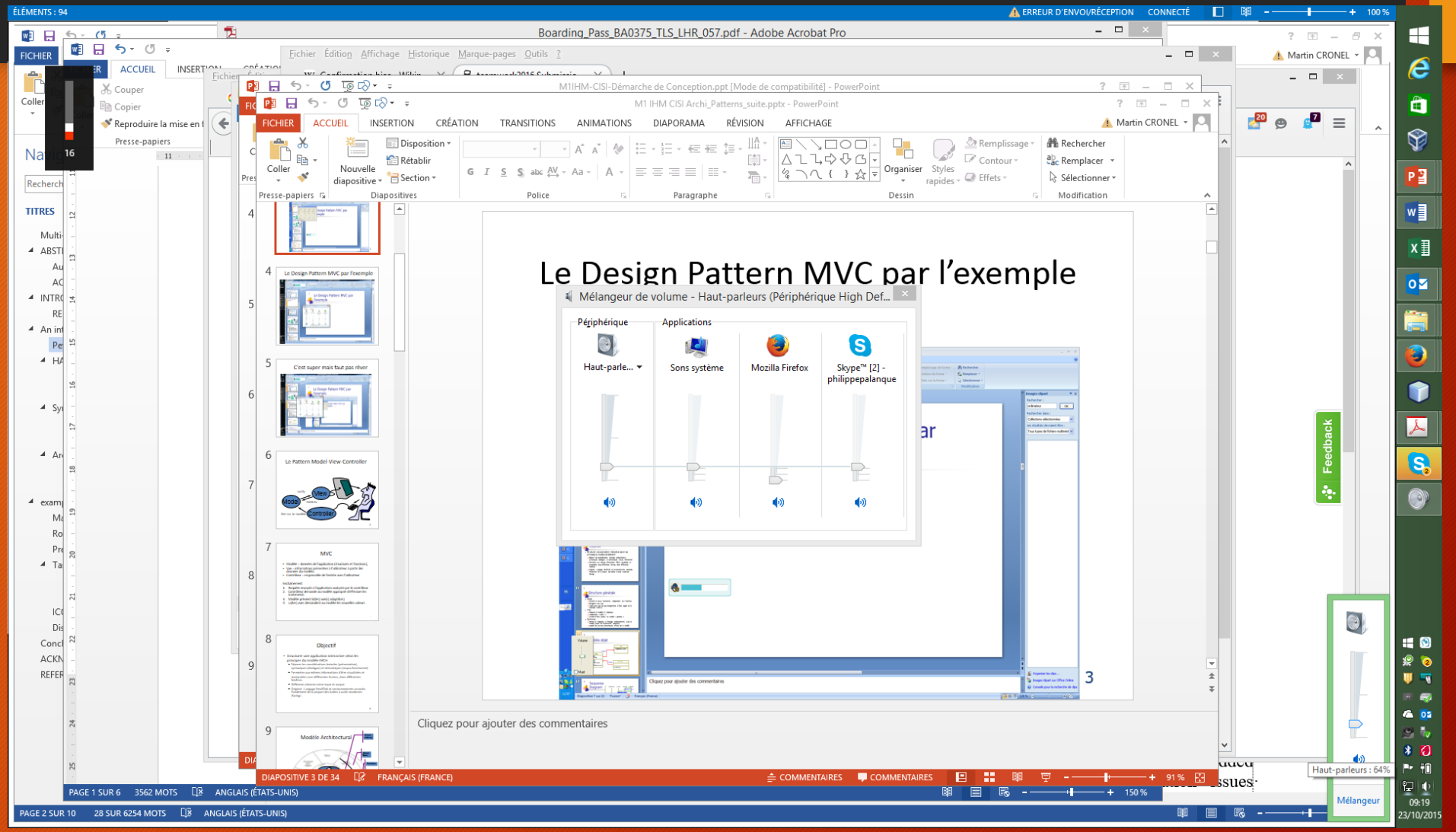
以 MVC 设计模式为例


Le Design Pattern MVC par l'exemple
Mélangeur de volume - Haut-parleurs (Périphérique High Def...
Périphérique
Applications
Haut-parle...
Sons système
Mozilla Firefox
Skype™ [2] - philippepalanque
Cliquez pour ajouter des commentaires
Feedback
Mélangeur

Le Design Pattern MVC par l'exemple

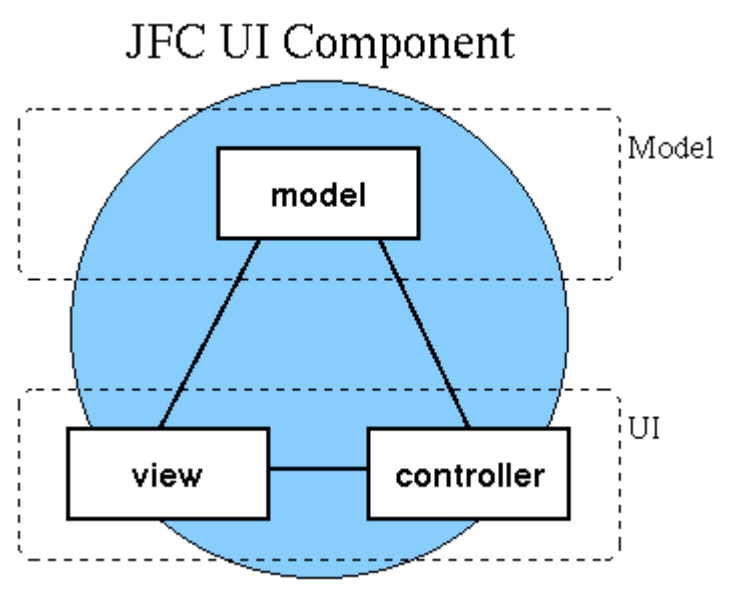


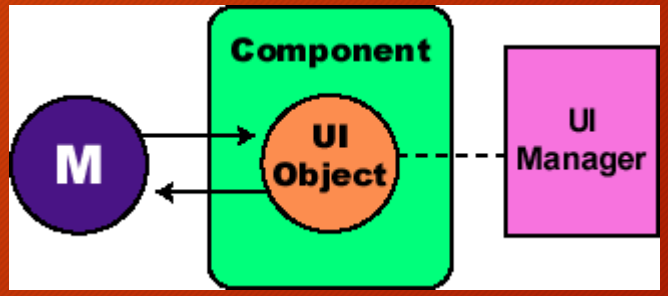
以 MVC 设计模式为例


Le Design Pattern MVC par l'exemple
Mélangeur de volume - Haut-parleurs (Périphérique High Def...
Périphérique
Applications
Haut-parle...
Sons système
Mozilla Firefox
Skype™ [2] - philippepalanque
Cliquez pour ajouter des commentaires
Feedback
Haut-parleurs : 64%
Mélangeur

Les composants Java Swing

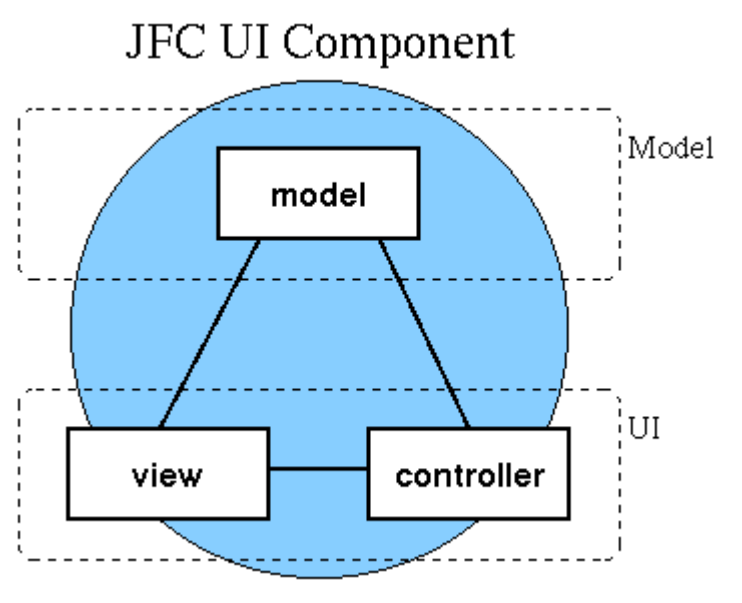
- Implémenté en respectant JavaBeans (propriétés, événements, facets)
- Architecture Swing = MVC ou M-UI

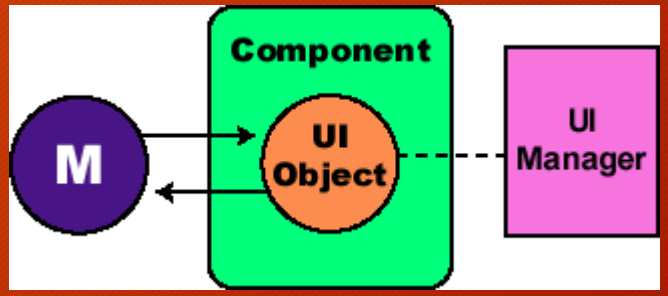




Java Swing 组件

- 基于 JavaBeans 规范实现（属性、事件、面）
- 架构模式：MVC 或 M-UI





Architecture des JComponents

- Modèle
  - Gère l'état interne (enabled, pressed, armed…)
  - Gère les listeners
  - Lève les événements (actionPerformed, …)
  - Peut être lié au noyau fonctionnel (exemple : JTable et Base de données)
- UI
  - Gestion du Look&Feel
    - Gère le dessin
    - Fournit les informations sur la géométrie
    - Gère les événements AWT
  - Peut-être changé dynamiquement
- Le tout est caché dans un object qui sert de façade
  - setModel
  - setUI

JComponent 架构

- 型号
  - 管理内部状态（启用、已按下、已武装……）
  - 管理听众
  - 获取事件（actionPerformed，...）
  - 可能与功能核心相关（例如：JTable 与数据库）
- 用户界面
  - 视觉与体验管理
    - 管理绘图
    - 提供几何相关信息
    - 管理 AWT 相关事件
  - 或许正在动态变化
- 这一切都被隐藏在一个充当外皮的对象之中。
  - 设置模型
  - 设置用户界面

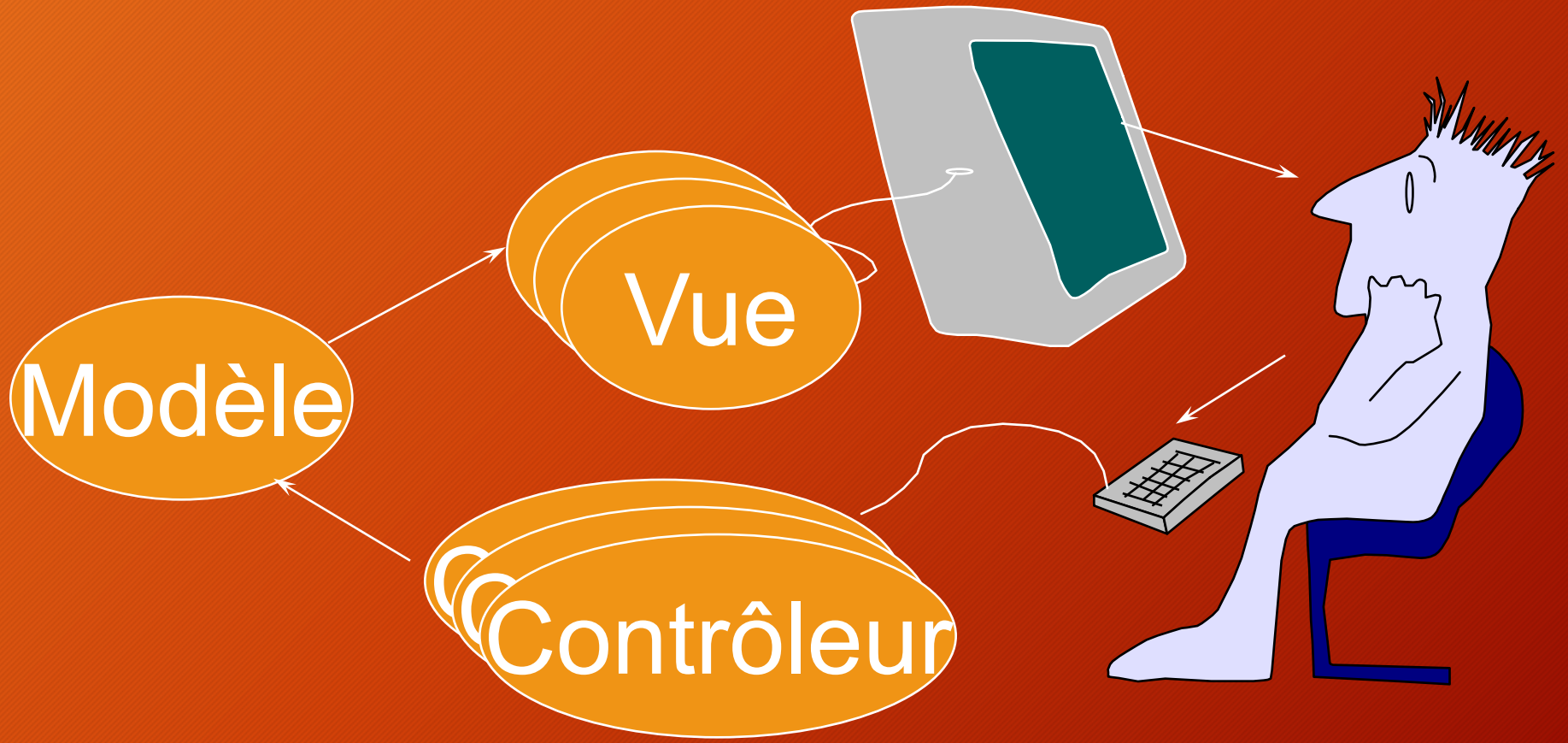
Exemple du JButton

- Démo partage de modèle entre boutons
- ButtonModel
  - Gestion des propriétés pressed, armed, selected
  - Gestion des listeners (et donc événéments) changed et action
- ButtonUI
  - Responsable du dessin
  - Instance partagée par plusieurs boutons
  - Sous-classée pour les différents Look&Feel
  - Définition de la géométrie (contains, preferredSize…)
  - Changement d'état du modèle en fonction des événement reçus via un ButtonUIListener

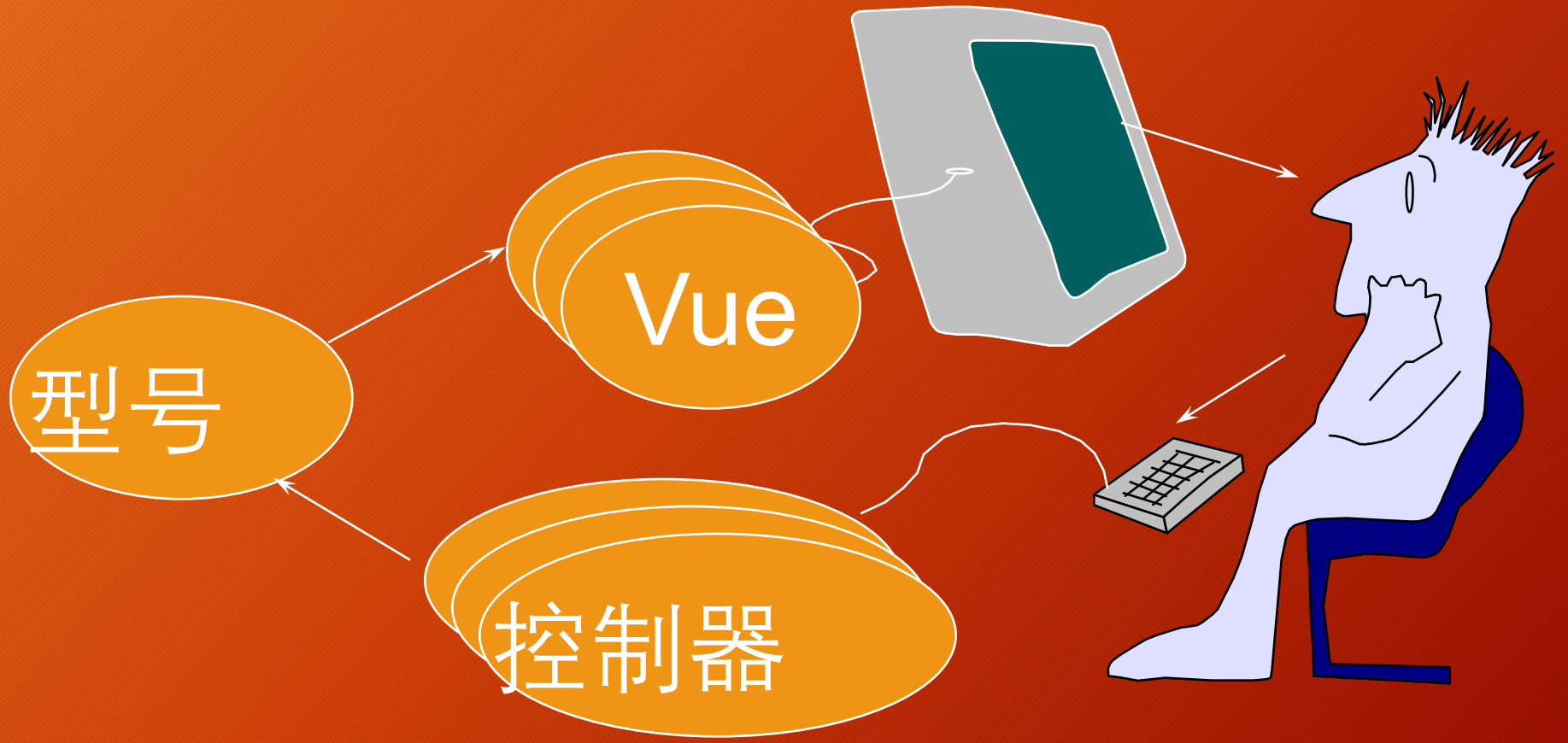
按钮示例

- 按钮间模型共享演示
- 按钮模型
  - 已压制、已武装、已选定财产的处置
  - 监听器管理（及相应事件）已更改及更新
- 按钮用户界面
  - 绘图负责人
  - 由多个按钮共享的实例
  - 根据不同的外观与触感进行细分
  - 几何结构定义（包含：contains、preferredSize ...）
  - 根据通过 ButtonUIListener 接收到的事件改变模型状态

Le Pattern Modèle - Vue - Contrôleur



模式模型 - 视图 - 控制器



Model-View-Controler

- Trygve Reenskaug: THING-MODEL-VIEW-EDITOR - an Example from a planningsystem. Technical note, Xerox PARC, May 1979.
- Trygve Reenskaug: MODELS - VIEWS - CONTROLLERS. Technical note, Xerox PARC, December 1979.
- Adèle Goldberg (Smalltalk), 1980

模型-视图-控制器

- Trygve Reenskaug：《THING-MODEL-VIEW-EDITOR》——来自规划系统的一个示例。技术笔记，Xerox PARC，1979年5月。
- Trygve Reenskaug：模型 - 视图 - 控制器。技术说明，Xerox PARC，1979 年 12 月。
- 阿黛尔·戈德堡（Smalltalk），1980年

MVC

- But : Séparation de la partie interface de la partie donnée dans le cadre de données volumineuses et complexes

- Modèle
  - gère les aspects applicatifs
  - met à jour les vues enregistrées
- Vue
  - se déclare auprès d'un modèle
  - se charge de la visualisation en sortie
  - *redirige les entrées vers un contrôleur associé*
- Contrôleur
  - gère les événements d'entrée
  - prévient le modèle associé

MVC

- 但：在海量且复杂的数据场景下，将接口部分与数据部分分离

- 型号
  - 负责应用层面的管理
  - 更新已保存的视图
- Vue
  - 向模特提出申请
  - 负责输出可视化显示
  - *将输入信号重定向至关联控制器*
- 控制器
  - 管理准入事件
  - 防止相关模型

Objectif

- Structurer une application interactive selon les principes du modèle de Seeheim
  - Séparer les considérations lexicales, syntaxiques et sémantiques
  - Permettre aux mêmes informations d'être visualisées et manipulées sous différentes formes, dans différentes fenêtres

目标

- 基于 Seeheim 模型原则构建交互式应用程序
  - 区分词汇、句法及语义层面的考量
  - 允许可用相同的信息以不同形式、在多个窗口中进行可视化展示与操作。

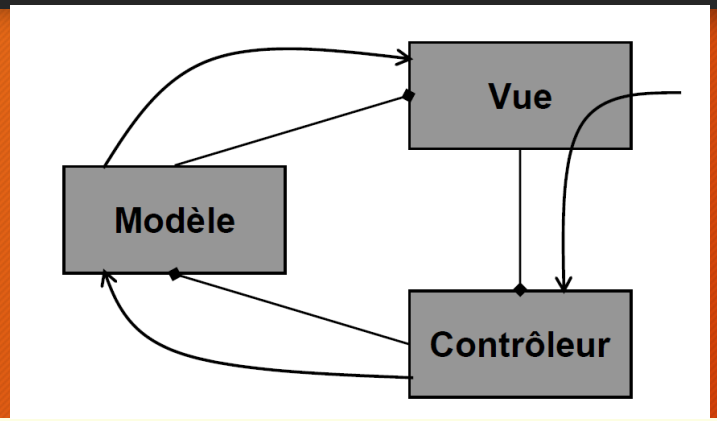
Structure générale

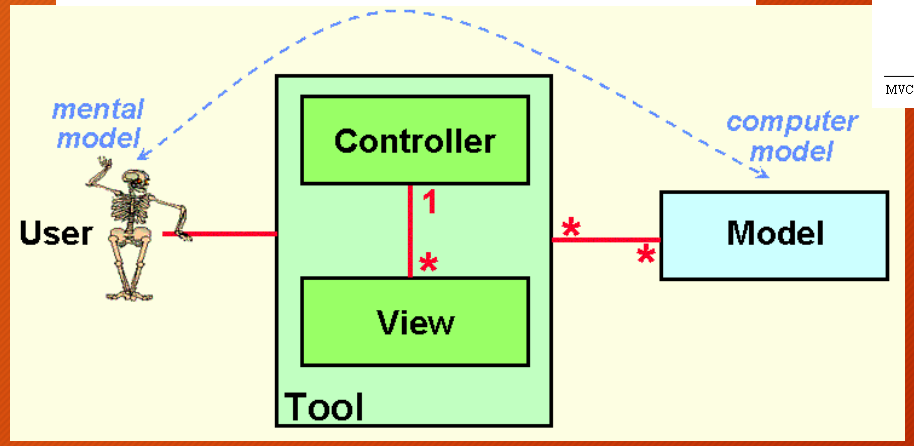
- Modèle
  - Contient le noyau fonctionnel, indépendant de l'interface
  - Enregistre ses vues
  - Notifie ses vues de ses changements d’état (appel de « notify »)
- Vue
  - Présente l’état du modèle à l’utilisateur
  - Implémente « notify »
  - Accède à l’état (public) du modèle (« getdata »)
- Contrôleur
  - Permet à l'utilisateur d’interagir (indirectement) avec le modèle (reçoit les événements utilisateur)
  - Appelle les services sémantiques offerts par le modèle

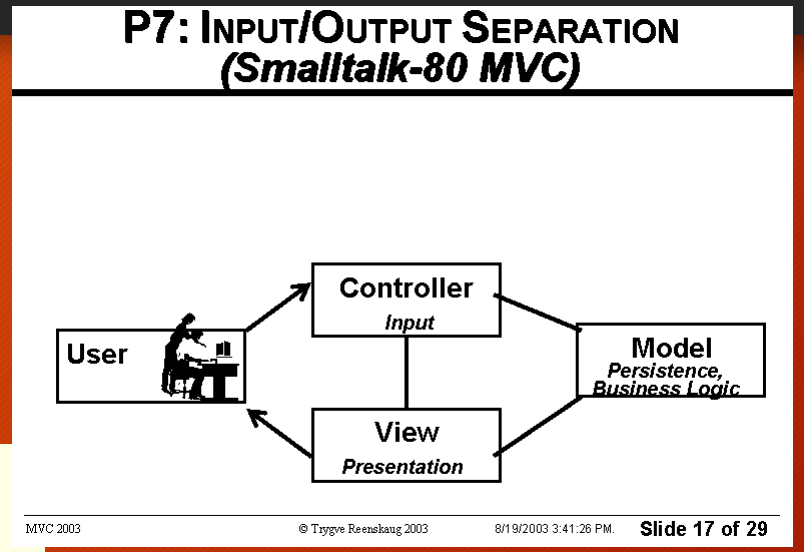
总体结构

- 型号
  - 包含与接口无关的功能核心
  - 记录浏览量
  - 通知其状态变更（称为“notify”）
- Vue
  - 向用户展示模型状态
  - 实现“notify”函数
  - 获取模型的状态（公开）（“getdata”）
- 控制器
  - 允许用户与模型进行（间接）交互（接收用户事件）
  - 调用该模型提供的语义服务

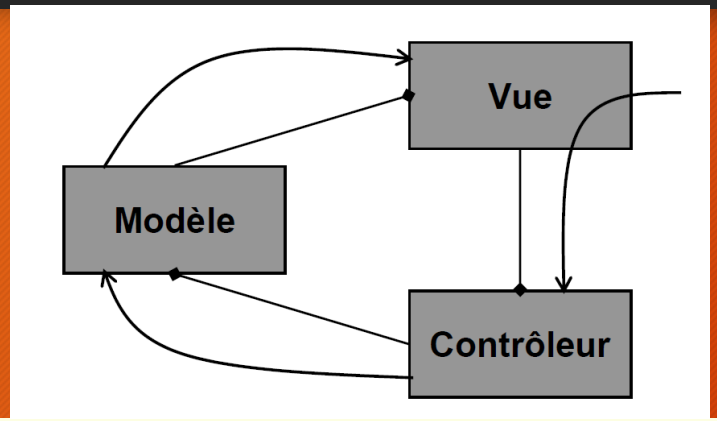
MVC : Versions Originales

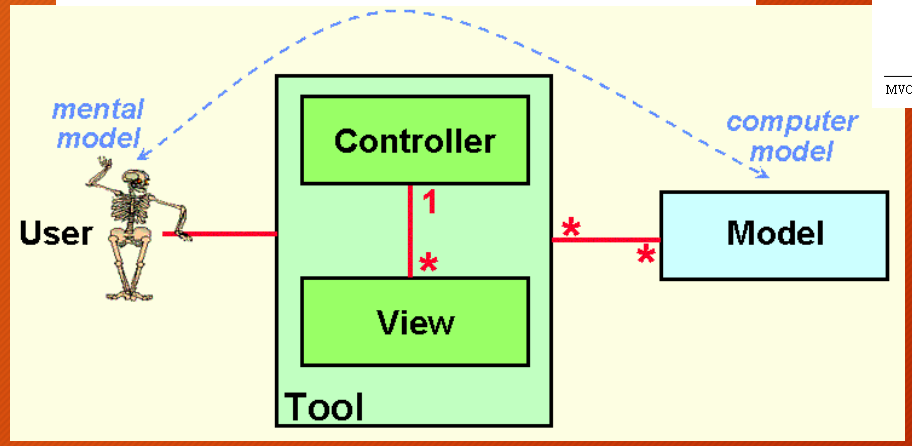


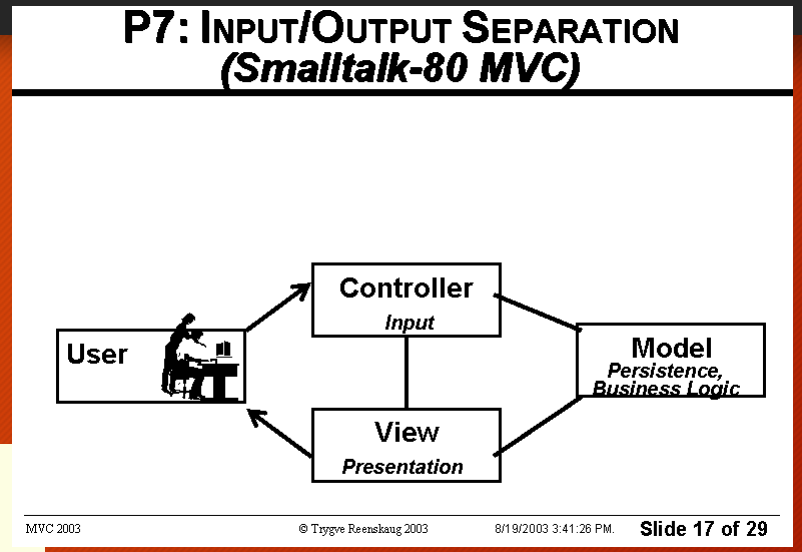




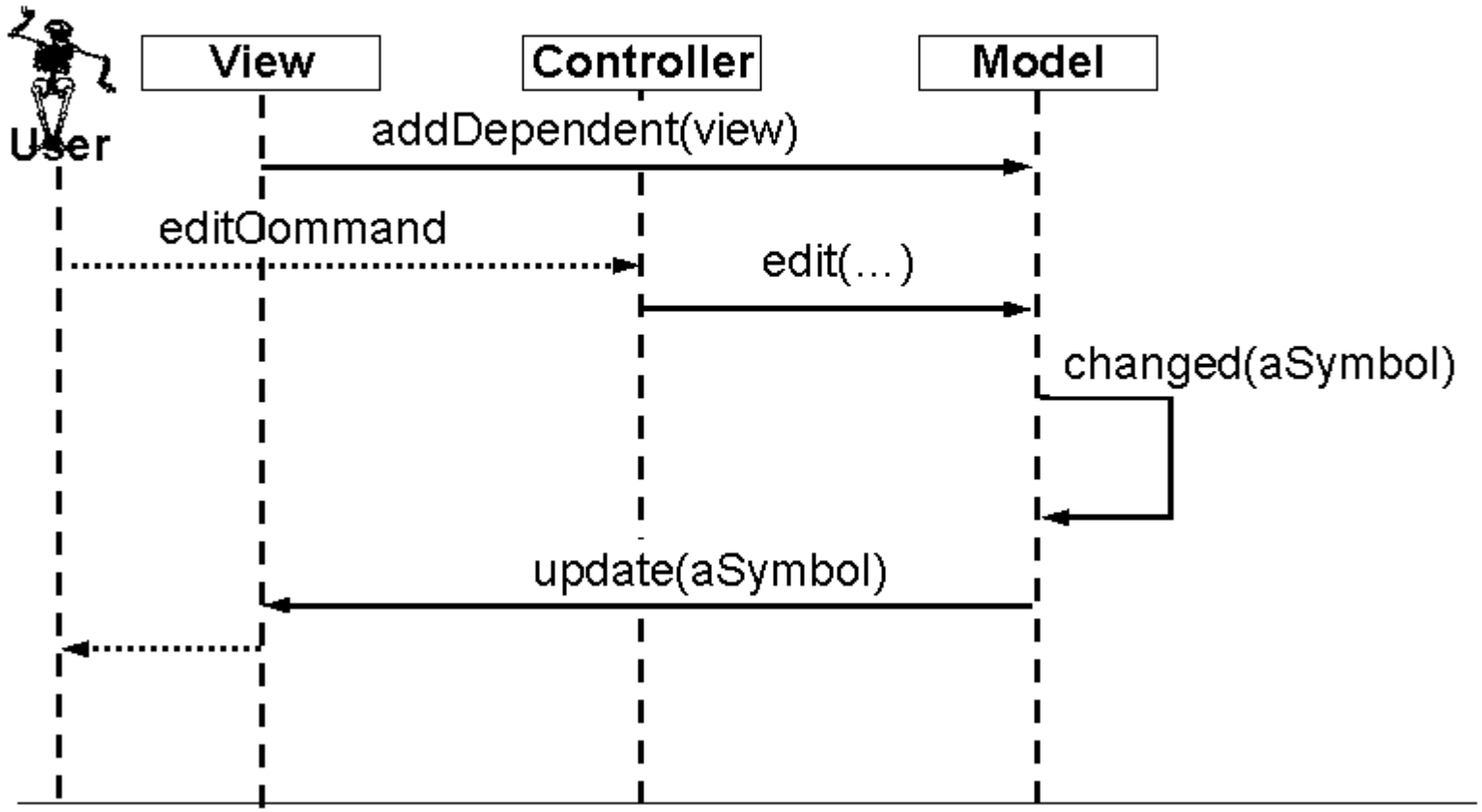
MVC：原始版本


Vue
Modèle
Contrôleur


mental model
computer model
User
Controller
1
*
View
Tool
*
*
Model

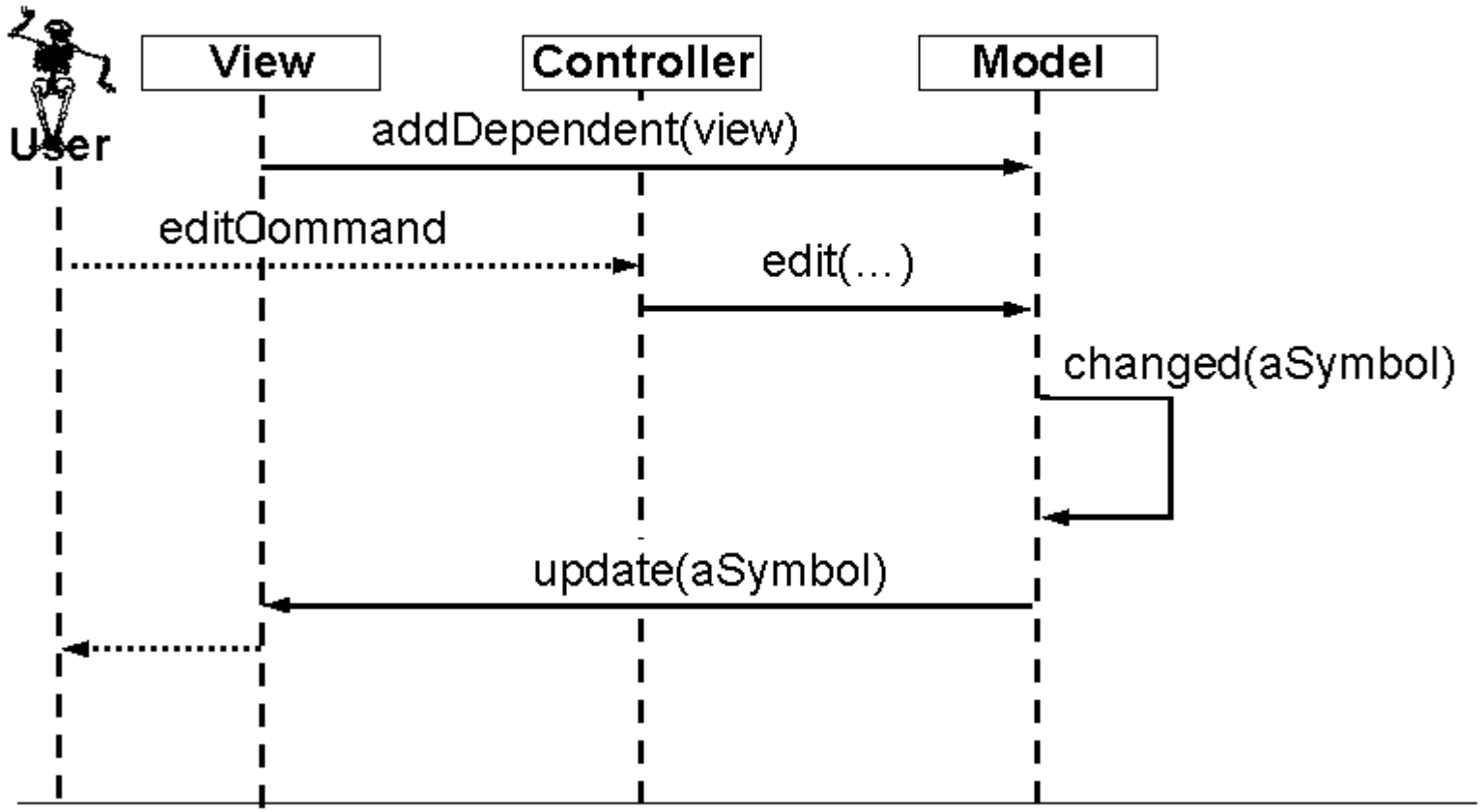

P7: INPUT/OUTPUT SEPARATION
(Smalltalk-80 MVC)
User
Controller
Input
View
Presentation
Model
Persistence,
Business Logic
MVC 2003
© Trygve Reenskaug 2003
8/19/2003 3:41:26 PM.
Slide 17 of 29

P11: Synchronize Model and View
ST-80 class library



© Trygve Reenskaug 2003

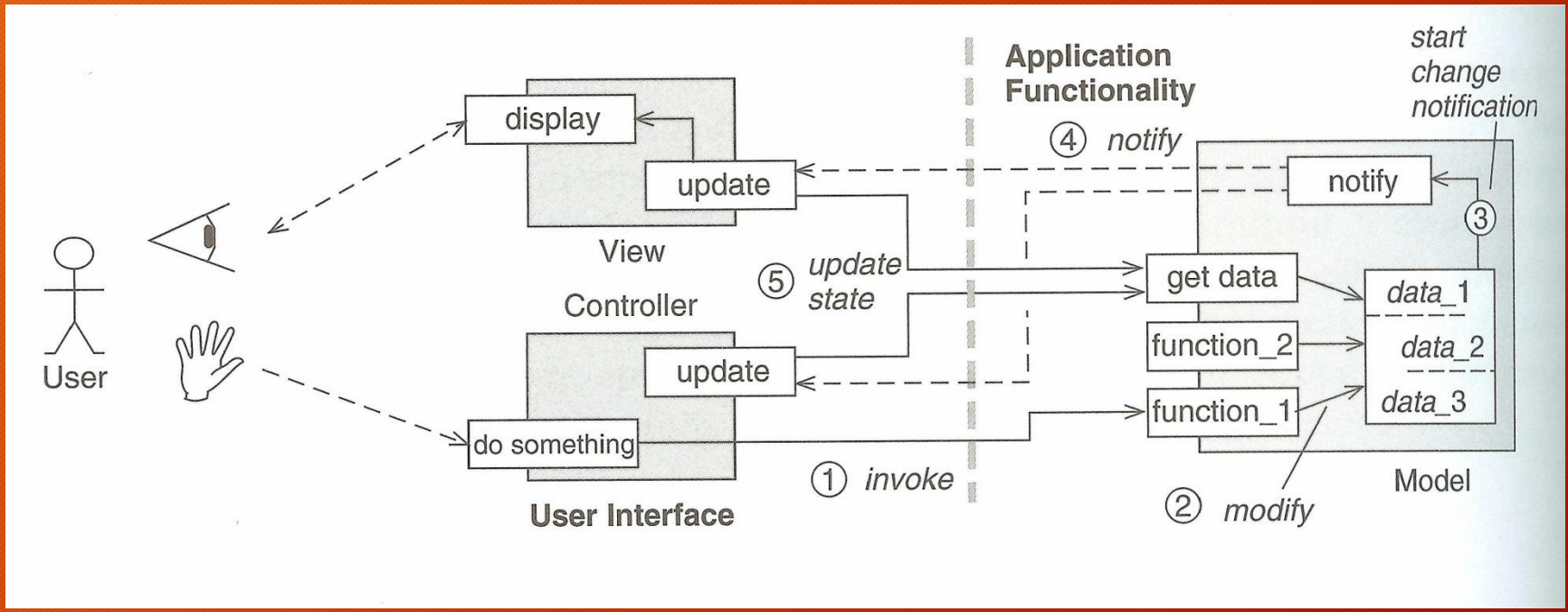
P11: Synchronize Model and View
ST-80 class library



© Trygve Reenskaug 2003

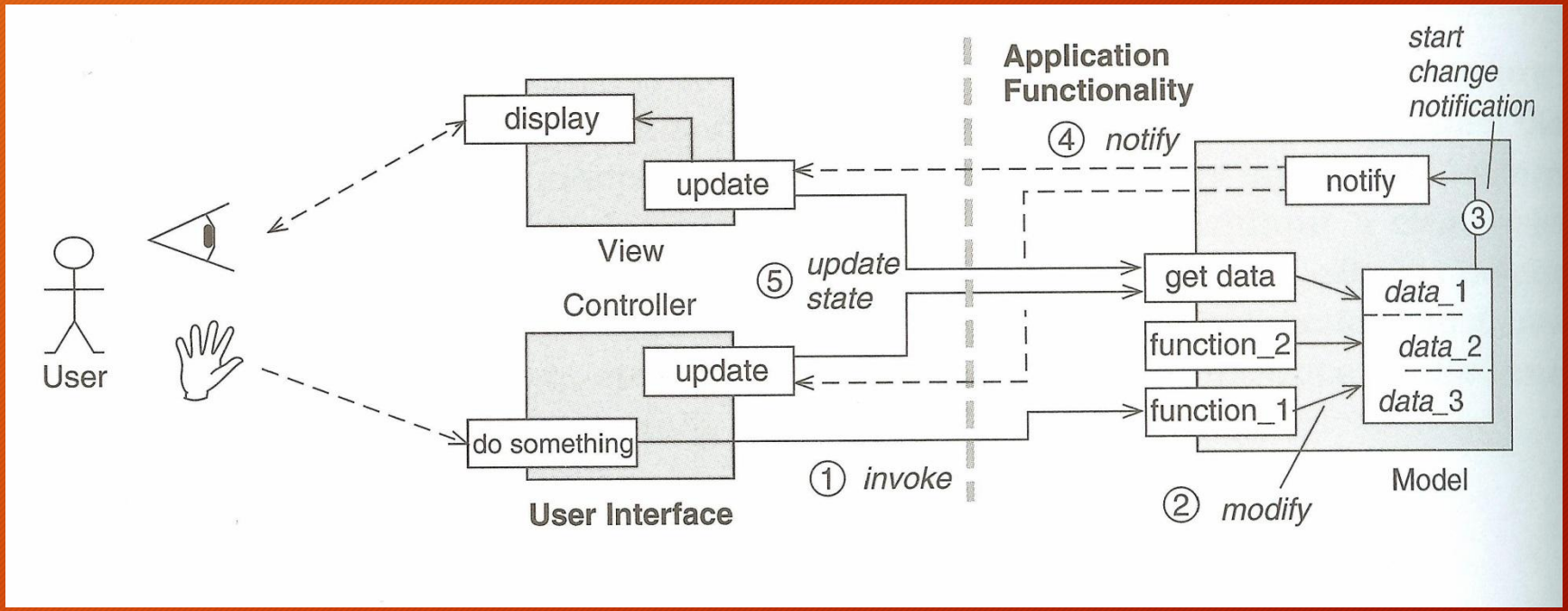
MVC : version « moderne »

- Buschmann F., Henney K., Schmidt D.C., Pattern-Oriented Software Architecture, A Pattern Language for Distributed Computing, Vol 4, Wiley Series, 2007



MVC：「现代」版本

- Buschmann F.、Henney K.、Schmidt D.C.，《面向模式的软件架构——分布式计算模式语言》，第 4 卷，Wiley系列丛书，2007



MVC

- Modèle = données de l'application (structures et fonctions),
- Vue = informations présentées à l'utilisateur à partir des données du modèle,
- Contrôleur = responsable de la gestion des entrées de l'utilisateur

Enchaînement

1. Requête envoyée à l'application analysée par le contrôleur
2. Contrôleur demande au modèle approprié d'effectuer les traitements
3. Modèle prévient la(les) vue(s) adapté(es)
4. La(les) vues demandent au modèle les nouvelles valeurs

MVC

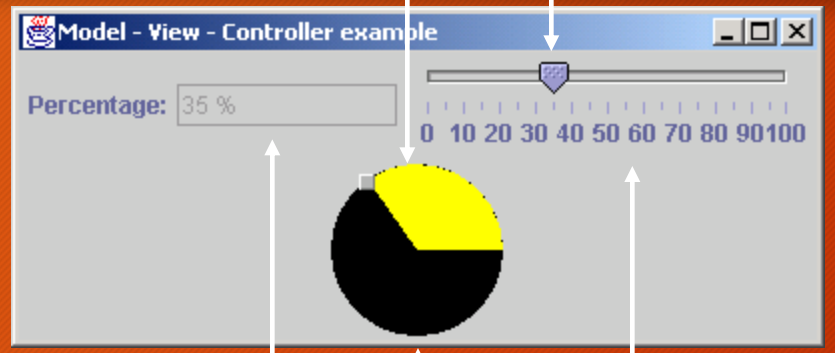
- 模型 = 应用程序数据（结构与函数），
- Vue = 根据模型数据向用户呈现的信息；•Controller = 负责管理用户输入的内容

训练

1. 已发送至由控制员审核的应用程序中的请求
2. 控制器要求相应型号执行相应处理。
3. 该型号配备适配的视觉功能
4. 相关观点要求模型提供新的价值观。

Exemple

Contrôleurs



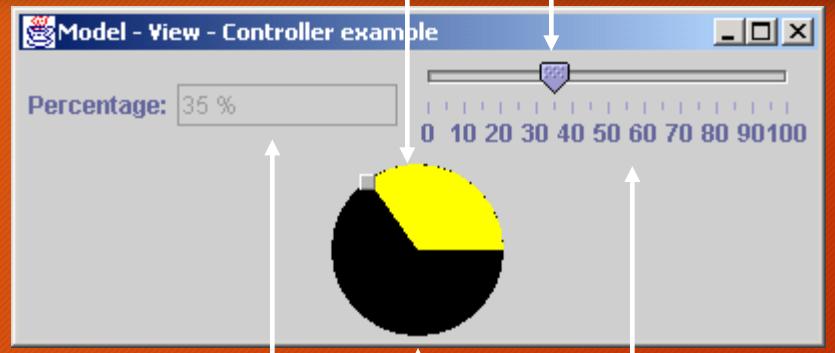
Vues

Modèle:

Pourcentage (valeur flottante 0 <= f < 1)

示例

控制器

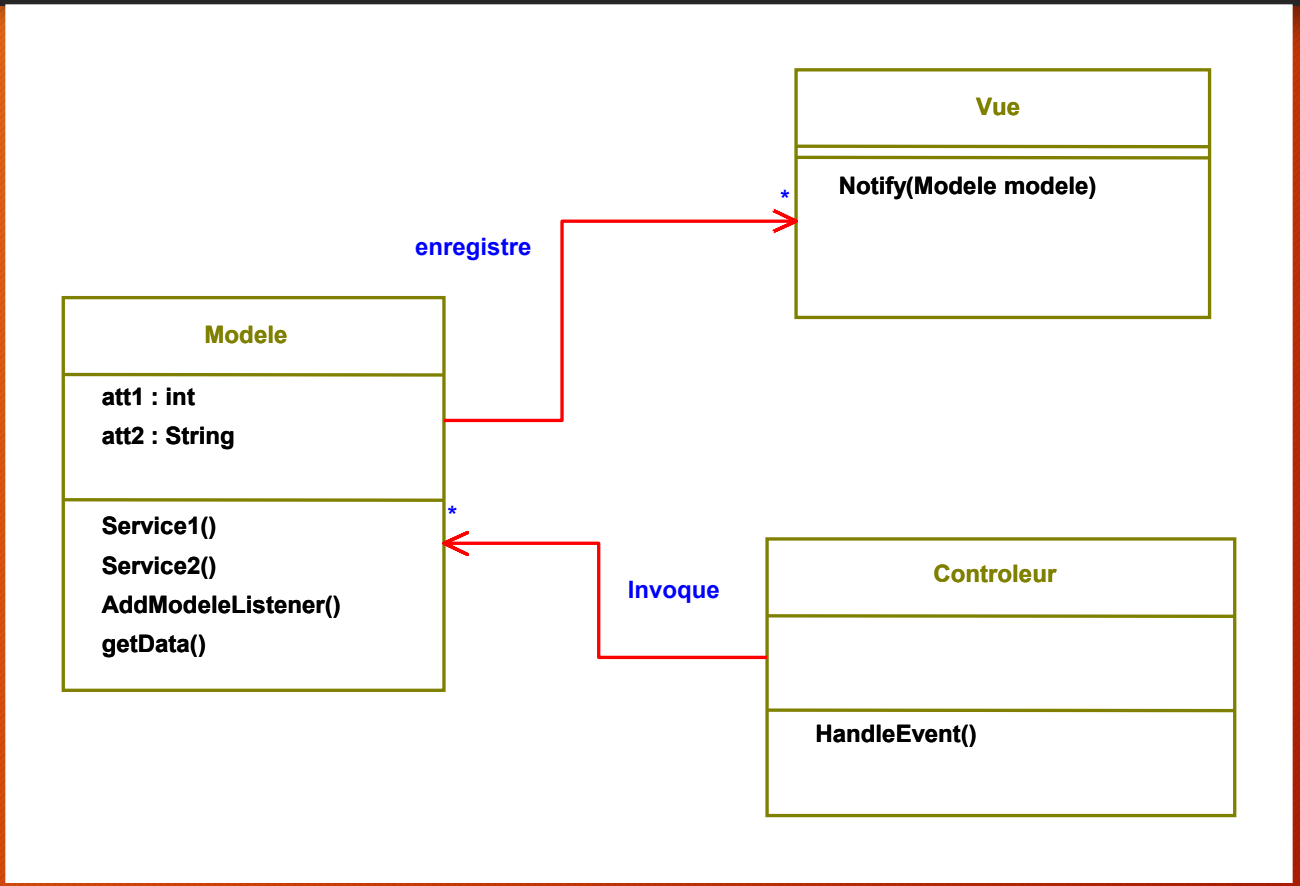


浏览量

模板:

百分比（浮动值 0 ≤ f ≤ 1）

Modèle de Classe



班级模型

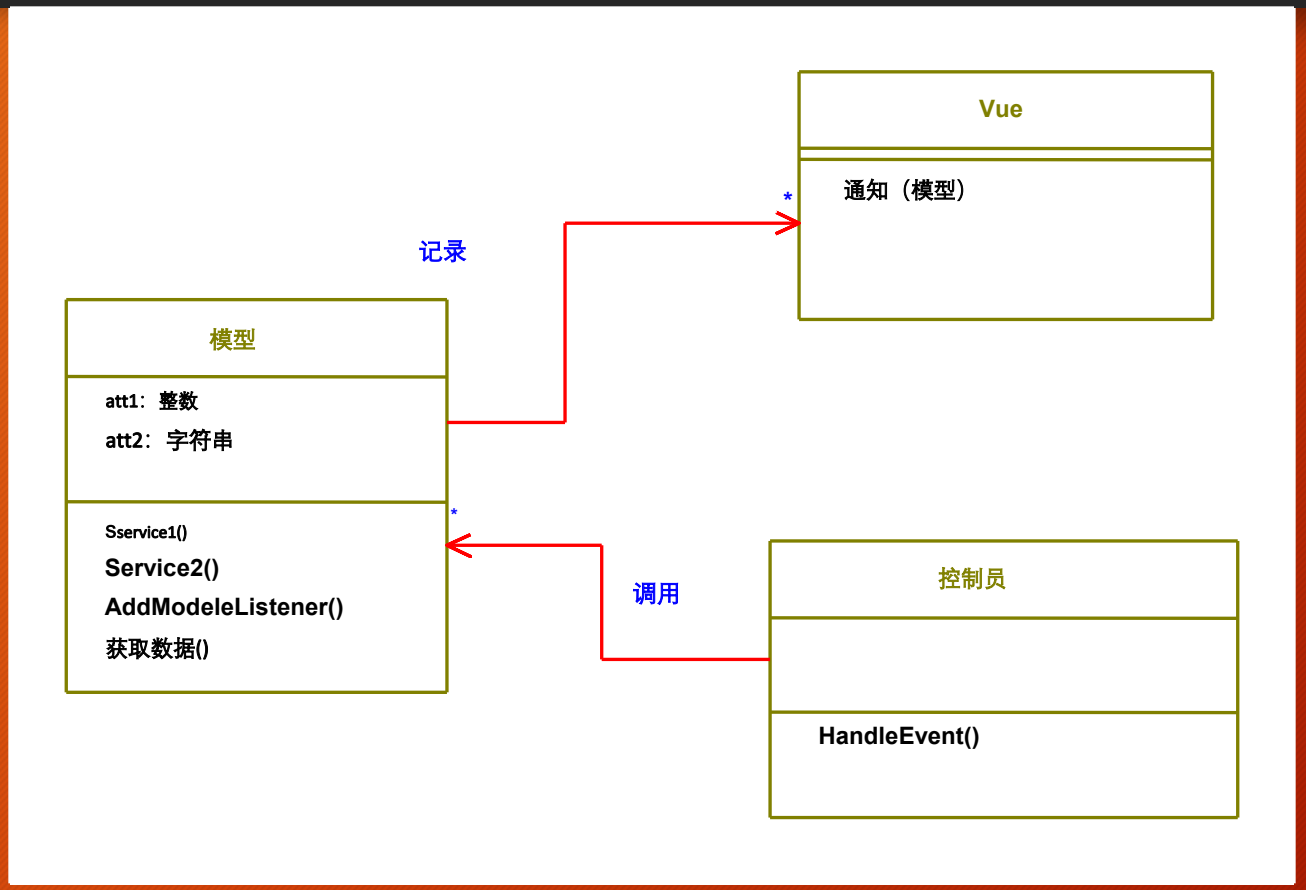
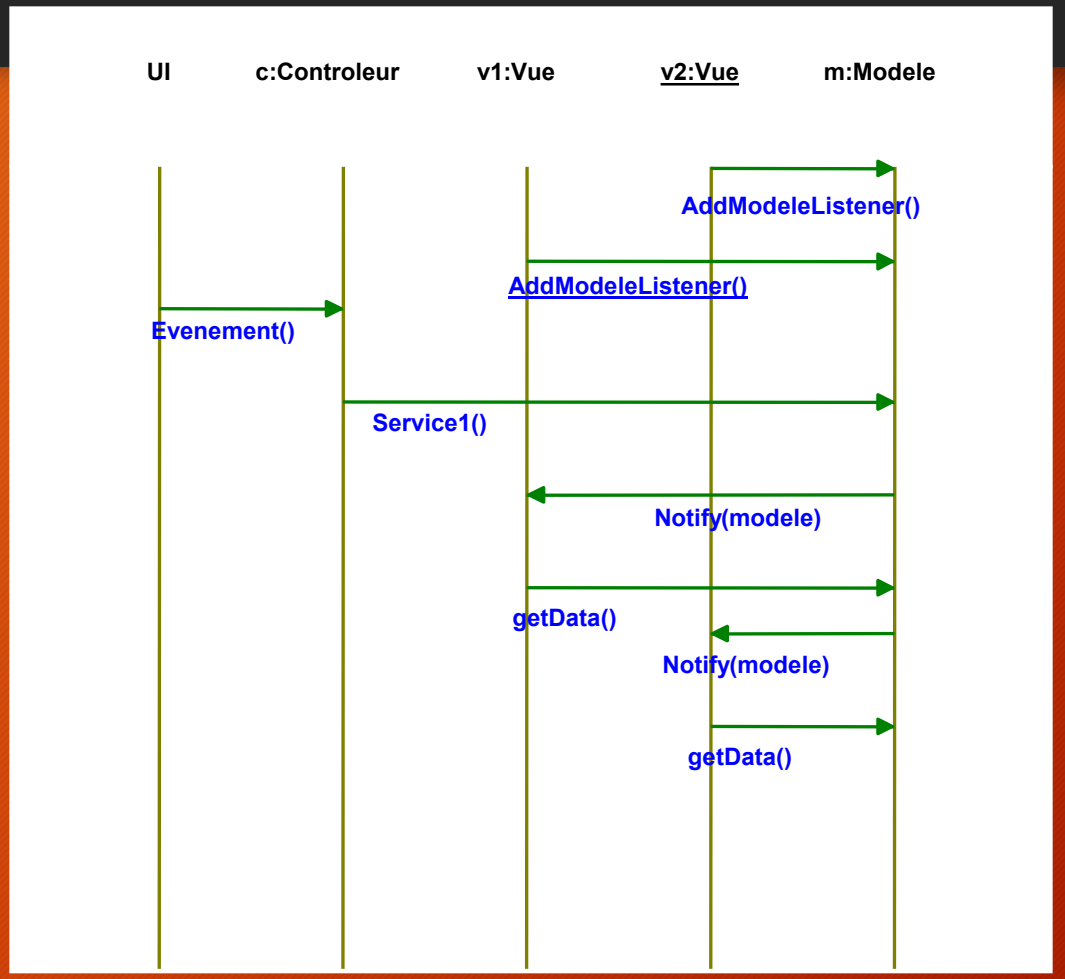
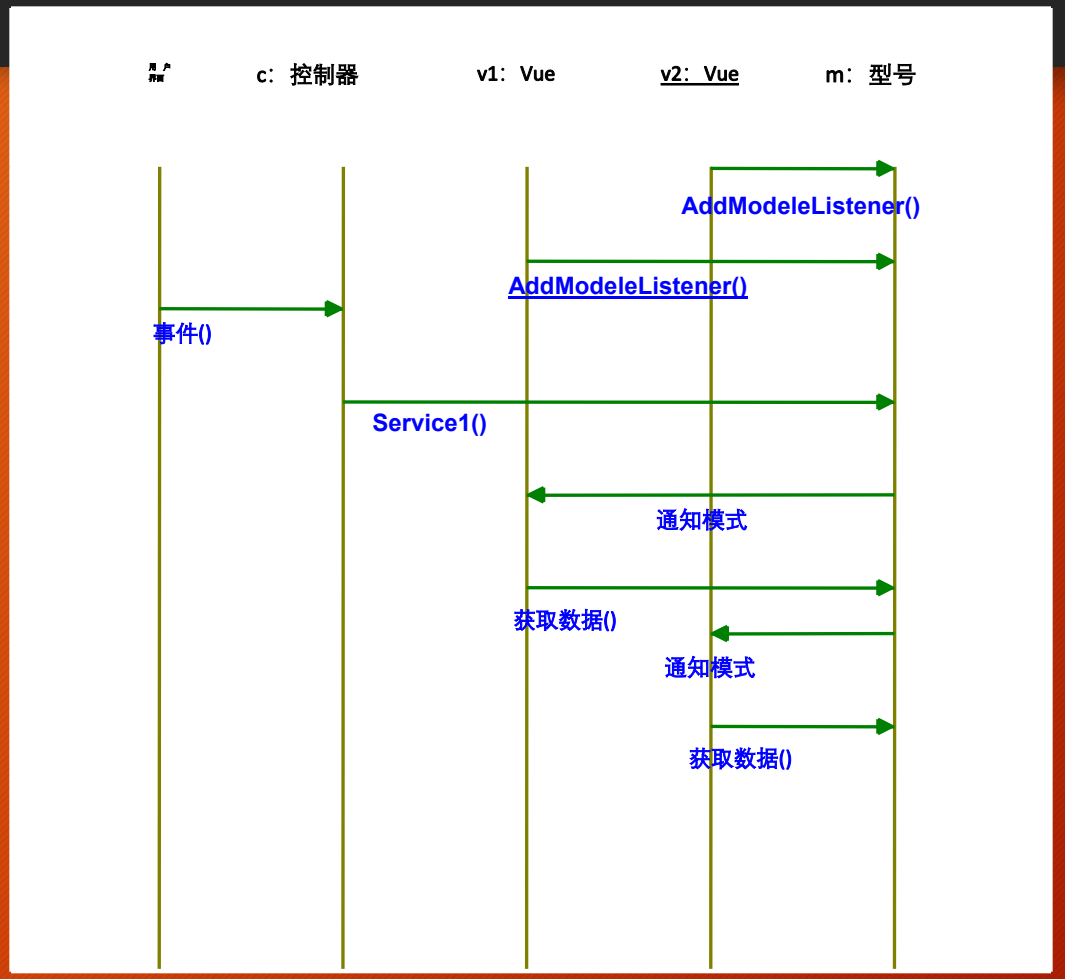

Vue
通知（模型）
*
记录
模型
att1：整数
att2：字符串
Sservice1()
Service2()
AddModeleListener()
获取数据()
*
调用
控制员
HandleEvent()

Diagramme de séquence



时序图


c：控制器
v1：Vue
v2：Vue
m：型号
AddModeleListener()
AddModeleListener()
事件()
Service1()
通知模式
获取数据()
通知模式
获取数据()

Conséquences

- Avantages
  - Facilité à présenter différentes vues du même objet et à les synchroniser
  - Facilité d'ajout d'une nouvelle représentation
- Inconvénients
  - Multiplication des « notify »
  - Inefficacité de l'accès au modèle
  - Couplage fort entre vue et contrôleur
    - Impossible en général de réutiliser un contrôleur indépendamment de sa vue
- Variantes
  - Document / View, PAC

后果

- 优势
  - 便于展示同一对象的多种视图，并实现视图同步。
  - 轻松添加新的表示形式
- 缺点
  - 增加“通知”数量
  - 模型获取效率低下
  - 视图与控制器之间强耦合
    - 通常无法独立于其视觉功能来复用控制器。
- 变体
  - 文档/查看，PAC

Exemple particulier le zoom

- Où sont les éléments M, V et C dans un la fenêtre?
- Décrivez l'architecture ainsi que les séquences d'appel de méthodes



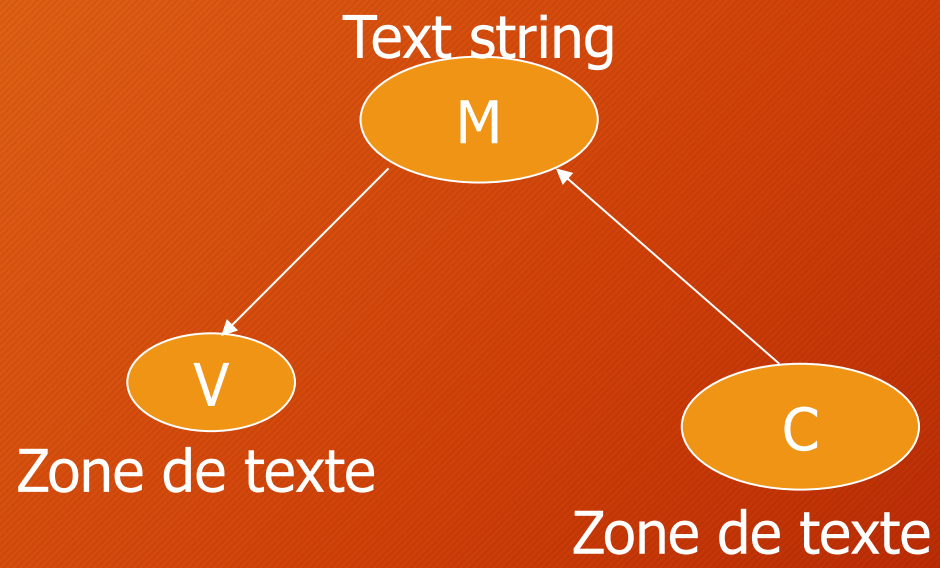
一个具体示例：缩放

- 窗口中的 M、V 和 C 元素分别位于何处?
- 请描述其架构以及方法调用的序列



Exemple particulier le zoom

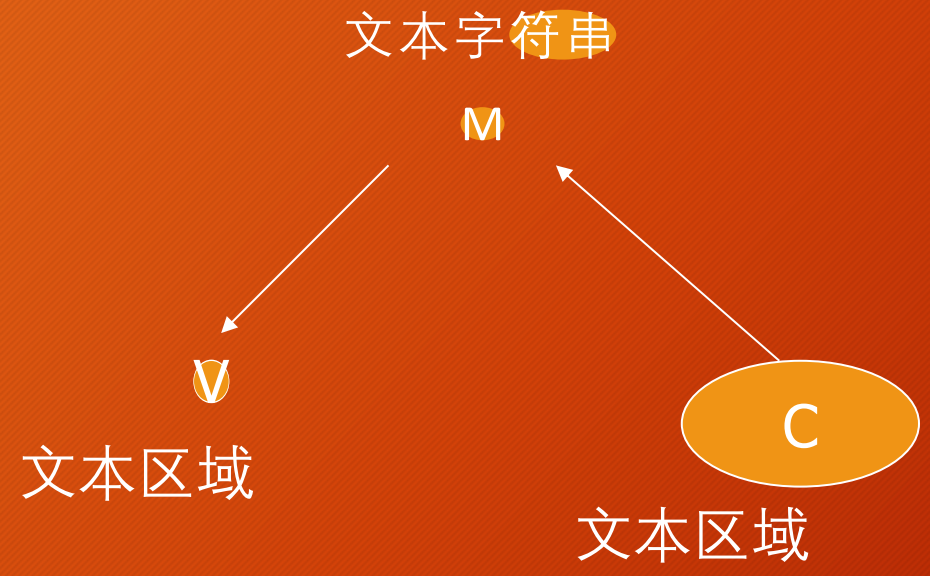
- Où sont les éléments M, V et C dans un la fenêtre?
- Décrivez l'architecture ainsi que les séquences d'appel de méthodes





一个具体示例：缩放

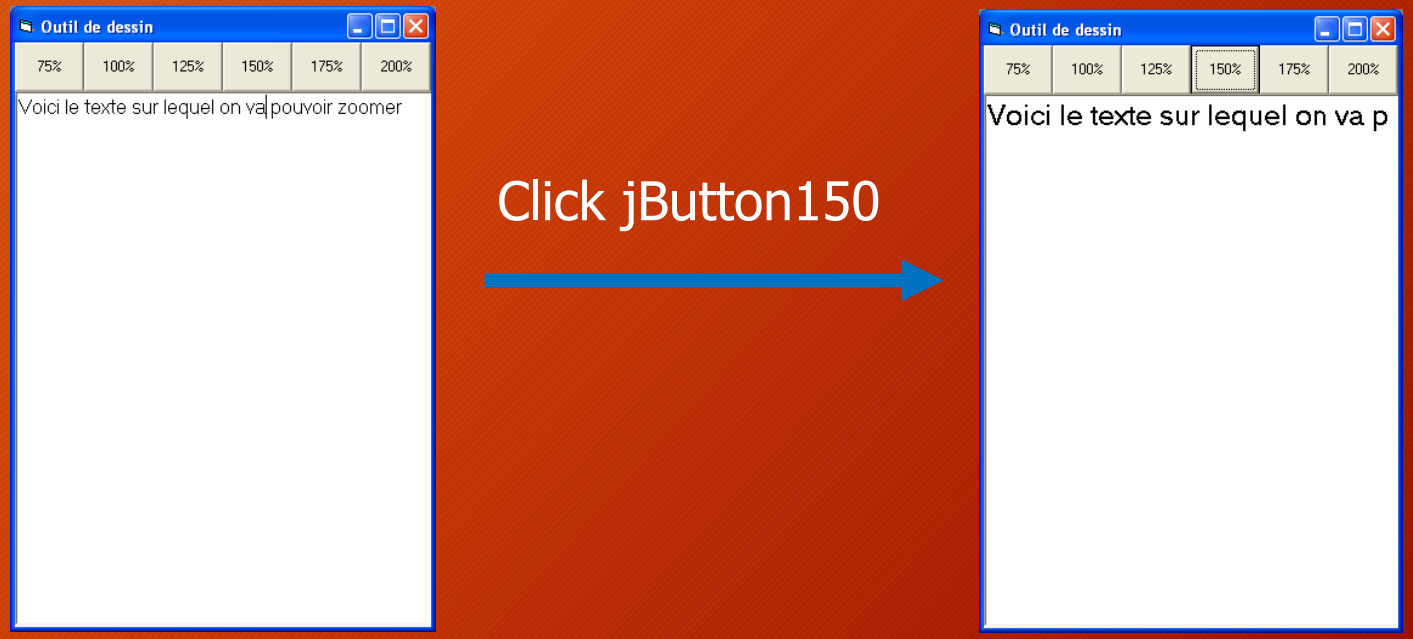
- 窗口中的 M、V 和 C 元素分别位于何处?
- 请描述其架构以及方法调用的序列





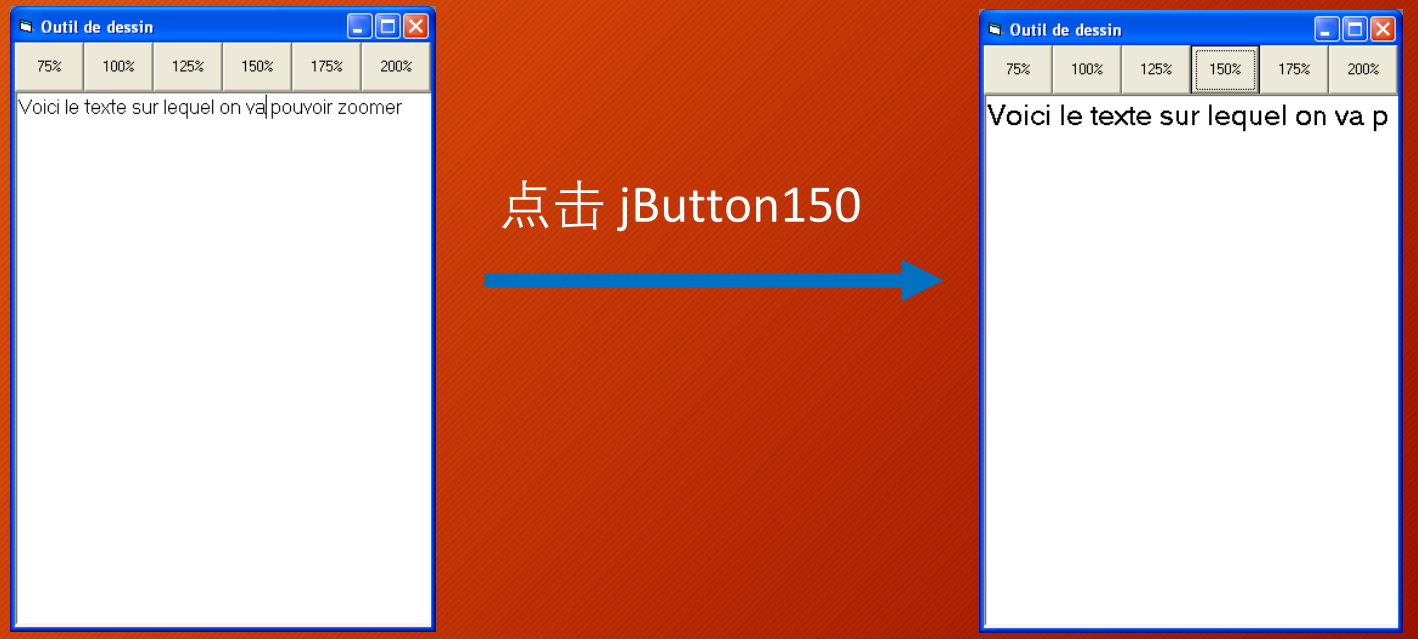
Exemple particulier le zoom

- Comment gérer plusieurs contrôleurs
- Décrivez l'architecture ainsi que les séquences d'appel de méthodes

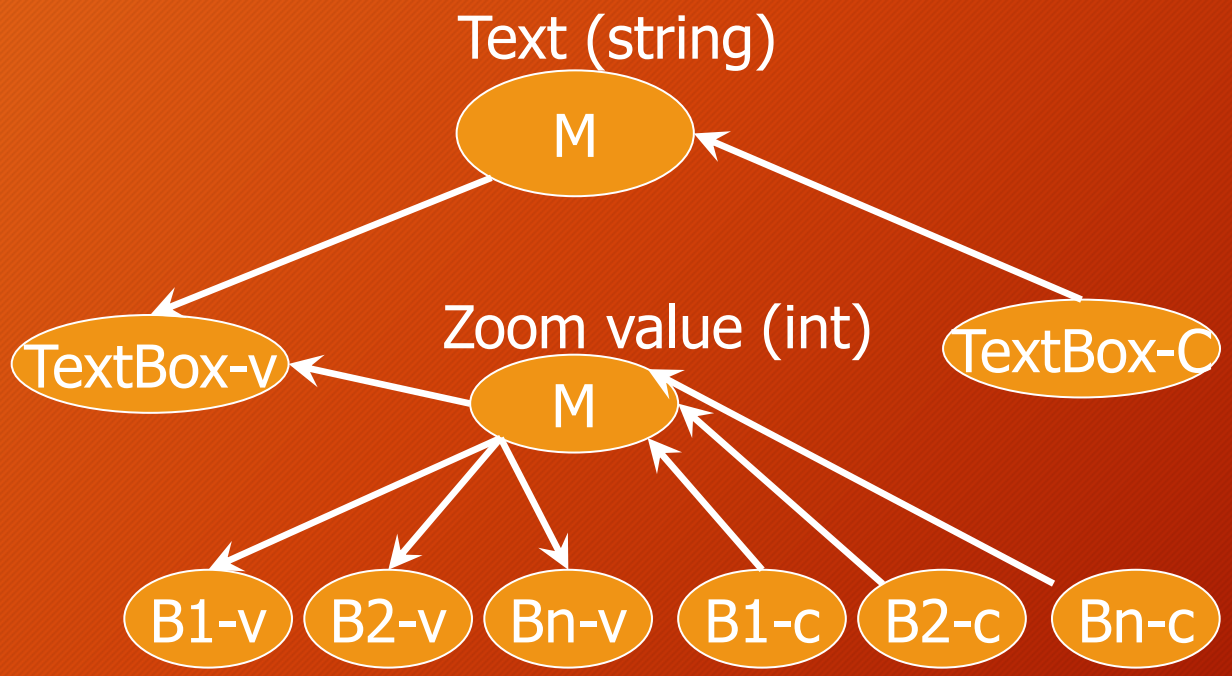


一个具体示例：缩放

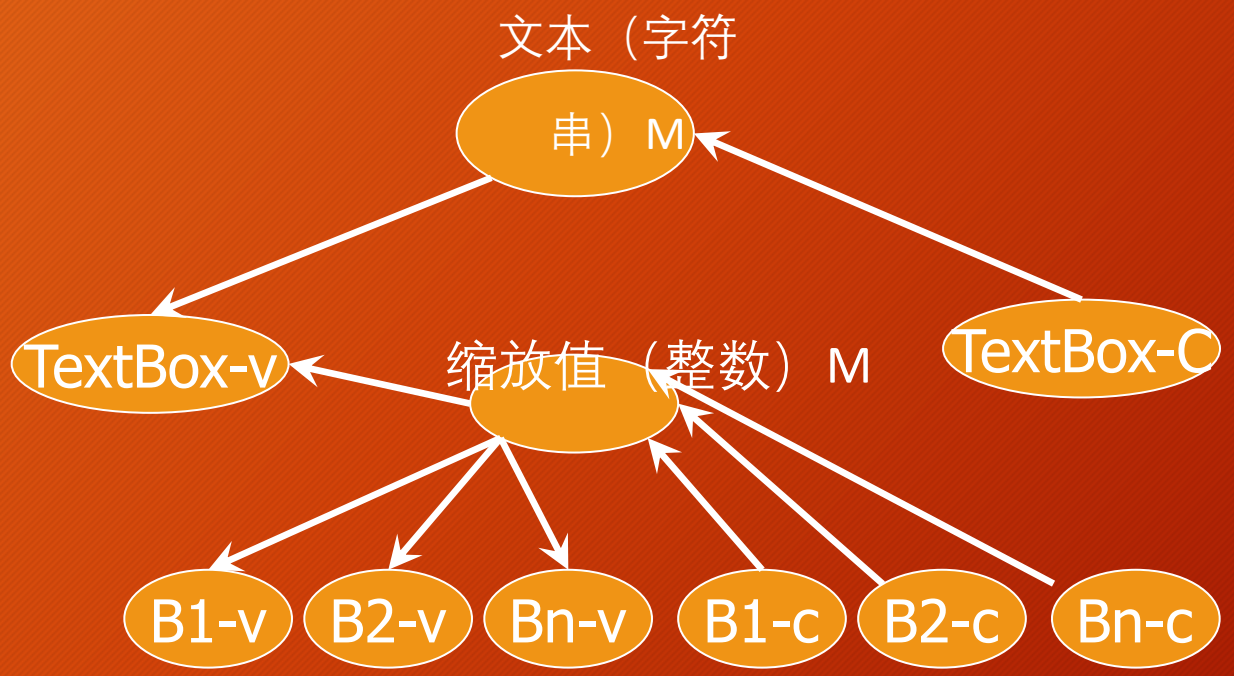
- 如何管理多个控制器
- 请描述其架构以及方法调用序列。



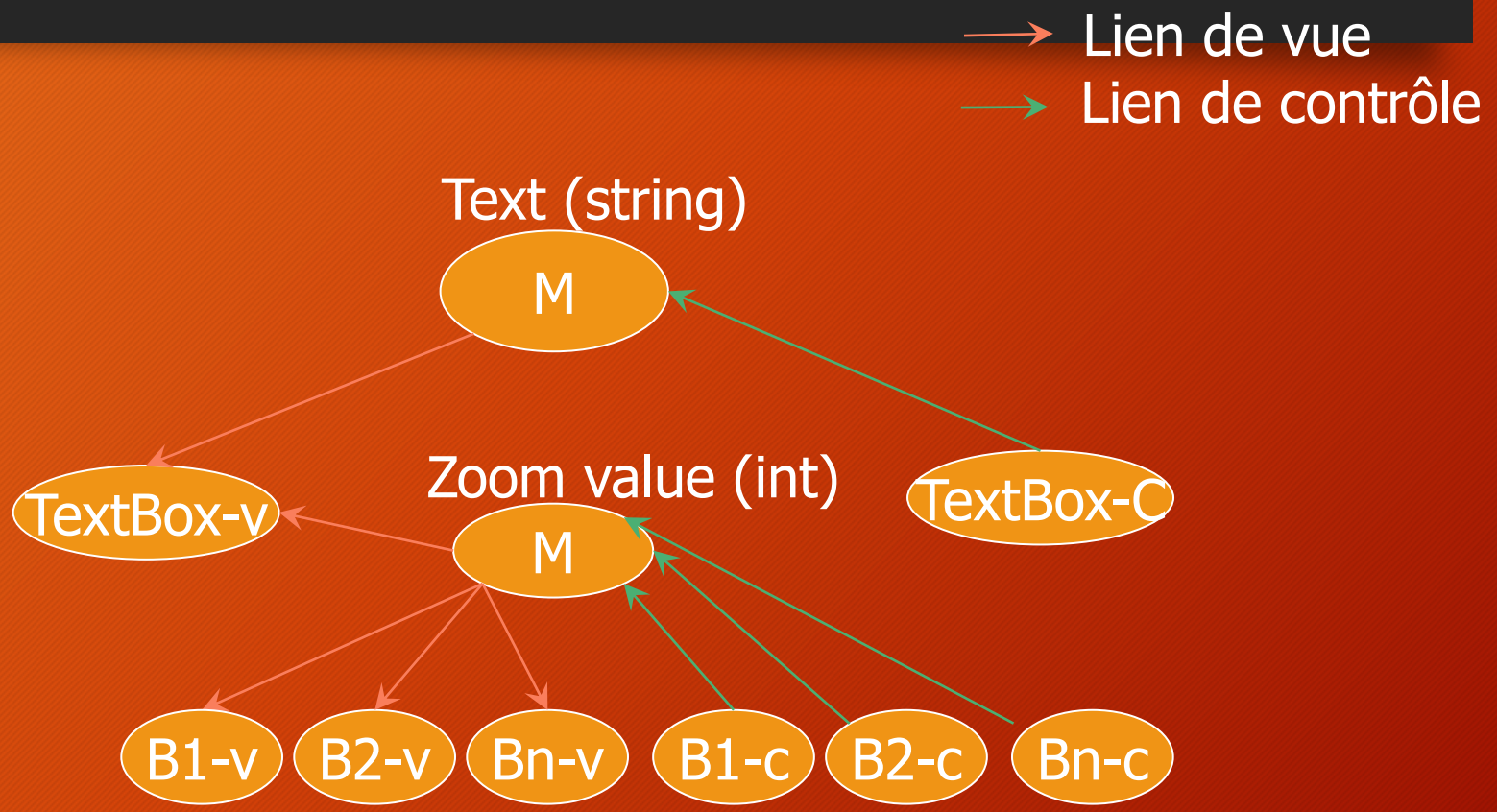
Exemple particulier le zoom



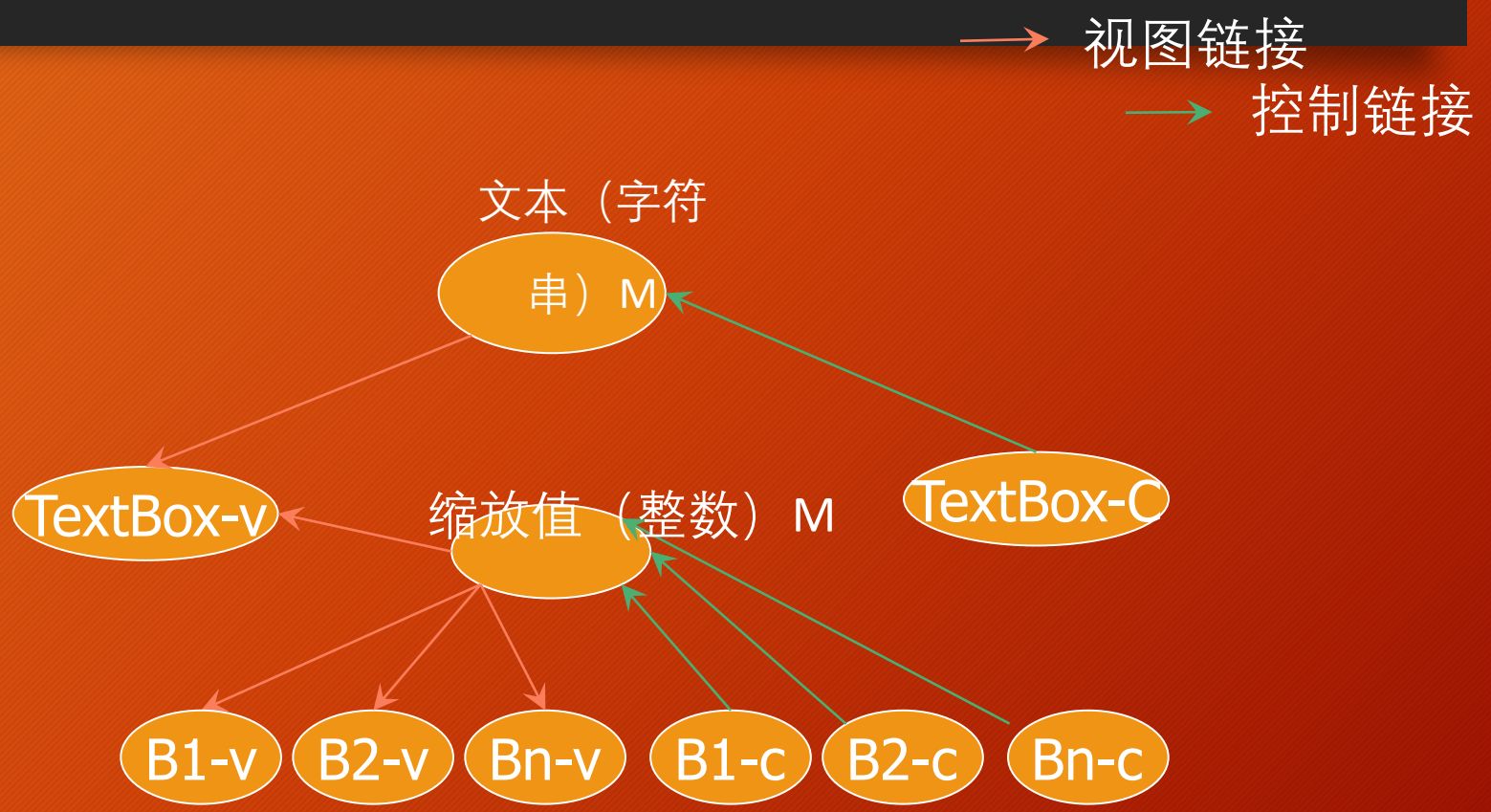
一个具体示例：缩放



Exemple particulier le zoom

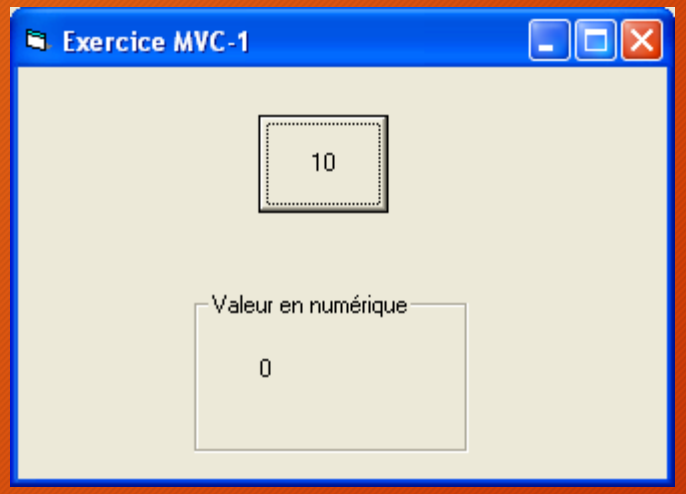


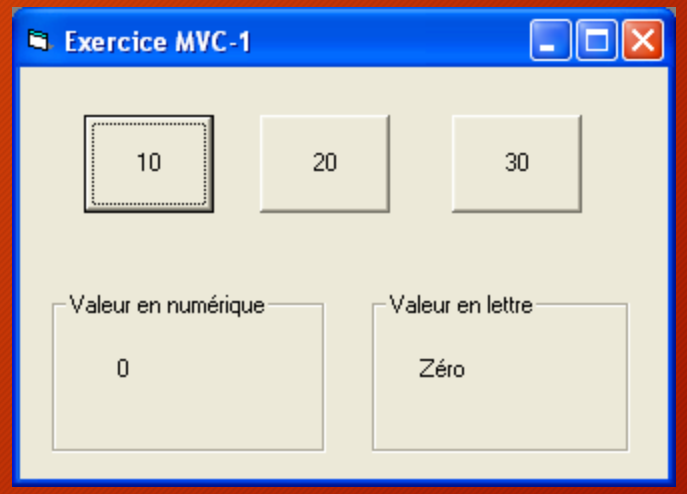
一个具体示例：缩放


视图链接
控制链接
文本（字符串）M
缩放值（整数）M
TextBox-v
TextBox-C
B1-v
B2-v
Bn-v
B1-c
B2-c
Bn-c

Exercice MVC

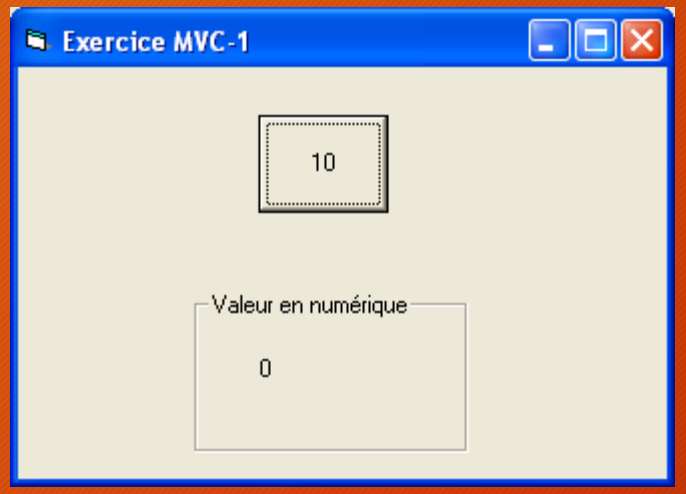
- 2 applications avec des modifications minimales grâce à une bonne structuration du code
- Objectif accroître la modifiabilité de l'application

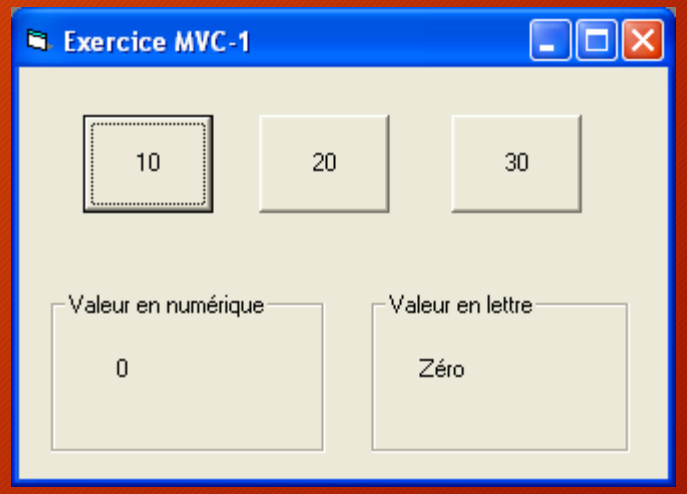




MVC 练习

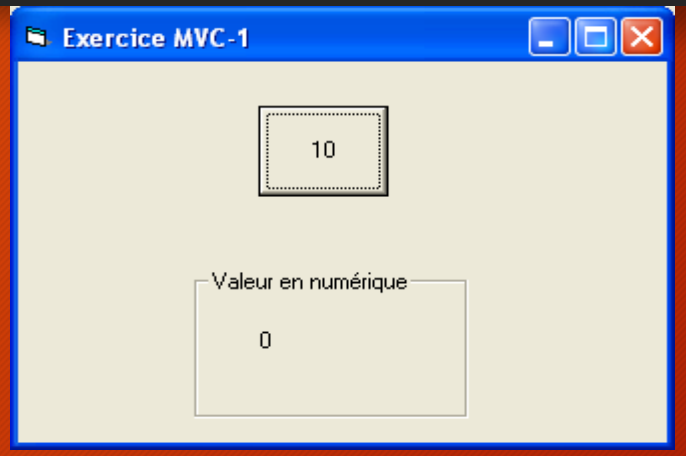
- 得益于良好的代码结构，仅需最小改动即可实现两个应用。
- 目标：提升应用程序的可定制性

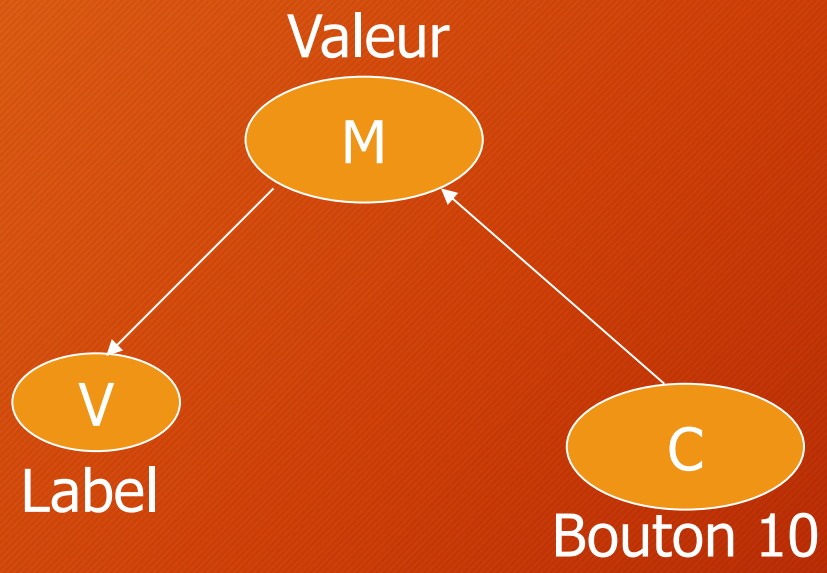




Exercice MVC

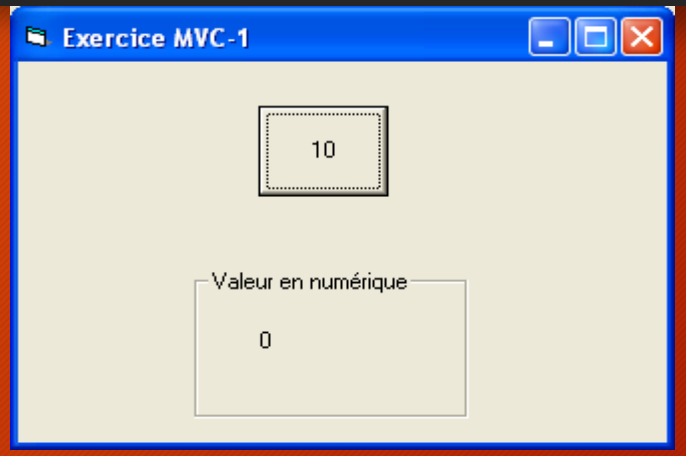
- Cas 1

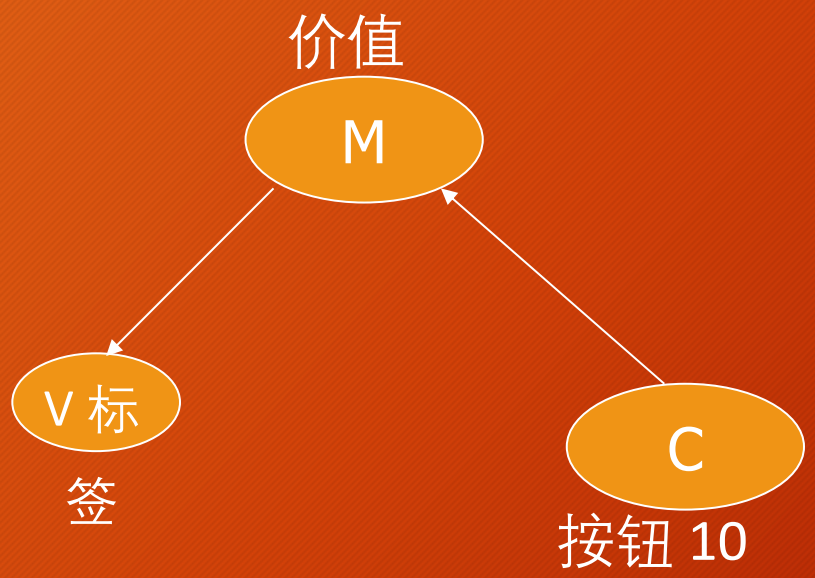




MVC 练习

- 案例 1





Exercice MVC : Interfaces

- 2 interfaces
  - IVue : offre une méthode notification()
  - IModel : offre une méthode addVue(IVue vue)
- Les vues sont "implements" de l'interface IVue
- Les modèles sont "implements" de l'interface IModel
- Objectif
  - Rapidité de création de nouvelles vues
  - Cohérence « forcée » des vues
  - Ne pas faire plusieurs fois la même chose

MVC 练习：接口

- 2 个接口
  - IVue：提供 notification() 方法
  - IModel：提供 addVue(IVue) 方法
- 视图是 IVue 接口的“实现”。
- 这些模型是 IModel 接口的“实现”。
- 目标
  - 新建视图的快速创建
  - 观点的“强制”一致性
  - 不要重复做同样的事情。

Exercice MVC : Classes

- VueLabel1
  - A une référence vers l'objet qui permet l'affichage
- ModelString
  - Contient un attribut (String valeur)
    - Offre l'accesseur setValeur (String chaine) : appelé par les contrôleurs
    - Offre l'accesseur getValeur (returns valeur) : appelé par les vues
  - Connait chacune des vues (géré dans une Collection)
  - Gère les vues en offrant un abonnement/désabonnement
  - Possède un itérateur pour la notification de toutes les vues
- jButton1: contrôleur direct (pas de classe supplémentaire)
  - Connait l'instance de ModelString
  - Appelle setValeur (dans l'event handler ou "actionPerformed")

MVC 示例：类

- VueLabel1
  - 一个指向对象的引用，用于实现显示功能
- 模型字符串
  - 包含一个属性（String 类型值）
    - 提供辅助函数 setValeur（字符串）：由控制器调用
    - 提供辅助函数 getValeur（返回值）：由视图调用
  - 了解每个视图（均在“集合”中管理）
  - 通过提供订阅/取消订阅服务来管理视图。
  - 具备用于通知所有视图的迭代器
- jButton1：直接控制器（无额外类）
  - 已知 ModelString 实例
  - 调用 setValeur（在事件处理程序或“actionPerformed”中）

Exercice MVC : code-contrôleur

- Le contrôleur (**code mauvais :** le contrôleur connait la vue et même ici l'interface graphique)
  - jButton1.addActionListener((e) -> { jLabel1.setText("10"); }

- Le contrôleur (**code bon :** le contrôleur connait le modèle)
  - jButton1.addActionListener ((e) -> {
  - model.setValeur("10"); }

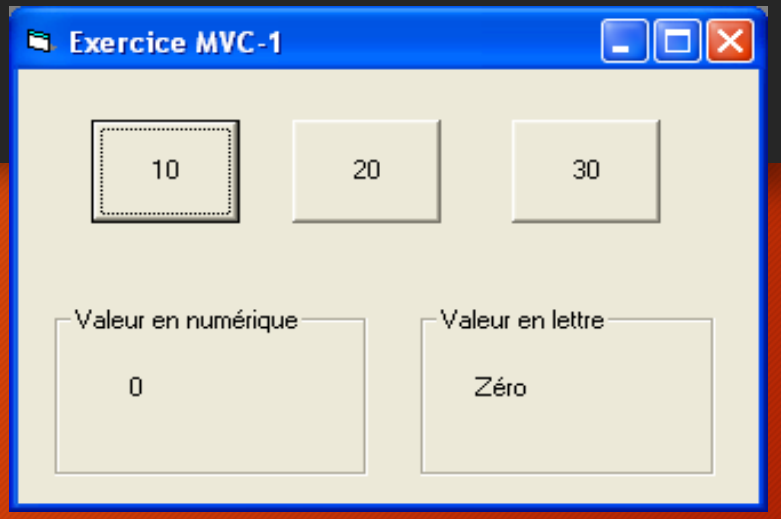
MVC 示例：控制器代码

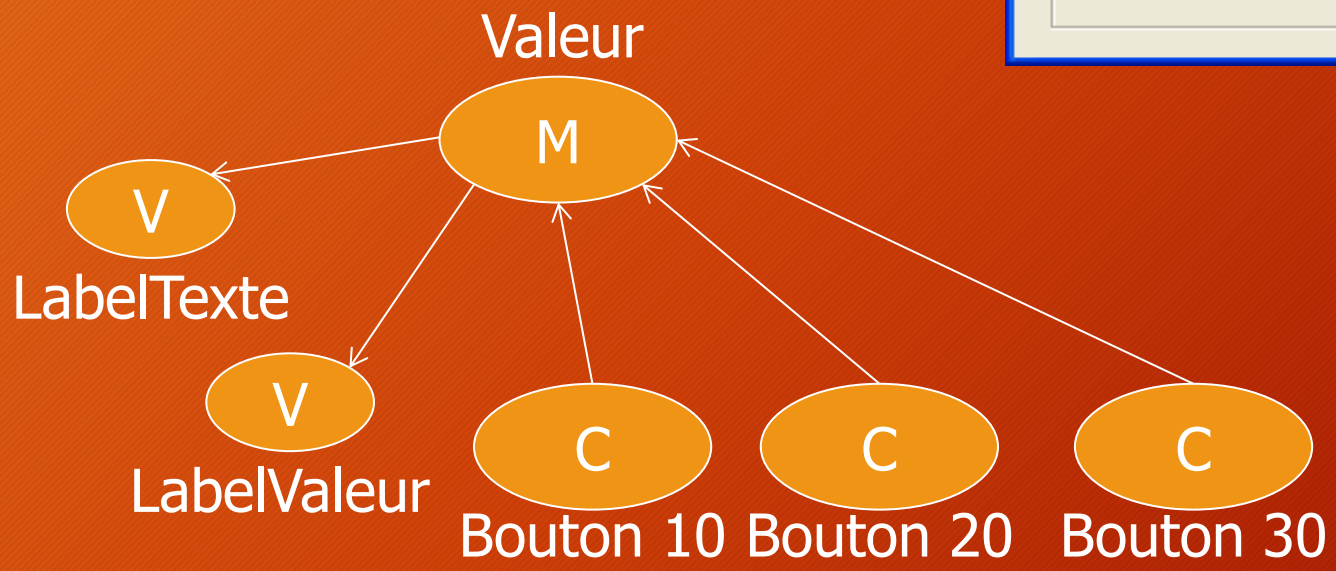
- 控制器（**错误代码**：控制器掌握视觉信息，甚至在此场景中也掌握图形界面）
  - jButton1.addActionListener((e) -> { jLabel1.setText（“10”）； }

- 控制器（**订单编号**：控制器已知晓该型号）
  - jButton1.addActionListener((e) -> {•model.
    setValeur（“10”）； }

Exercice MVC

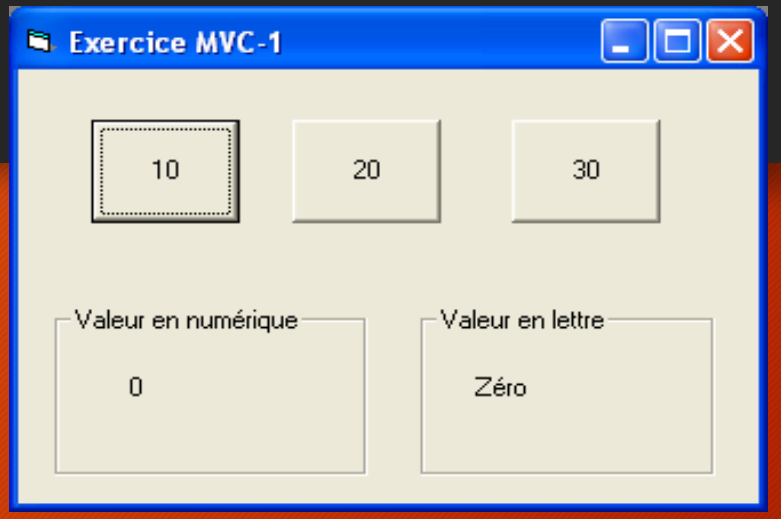
- Cas 2

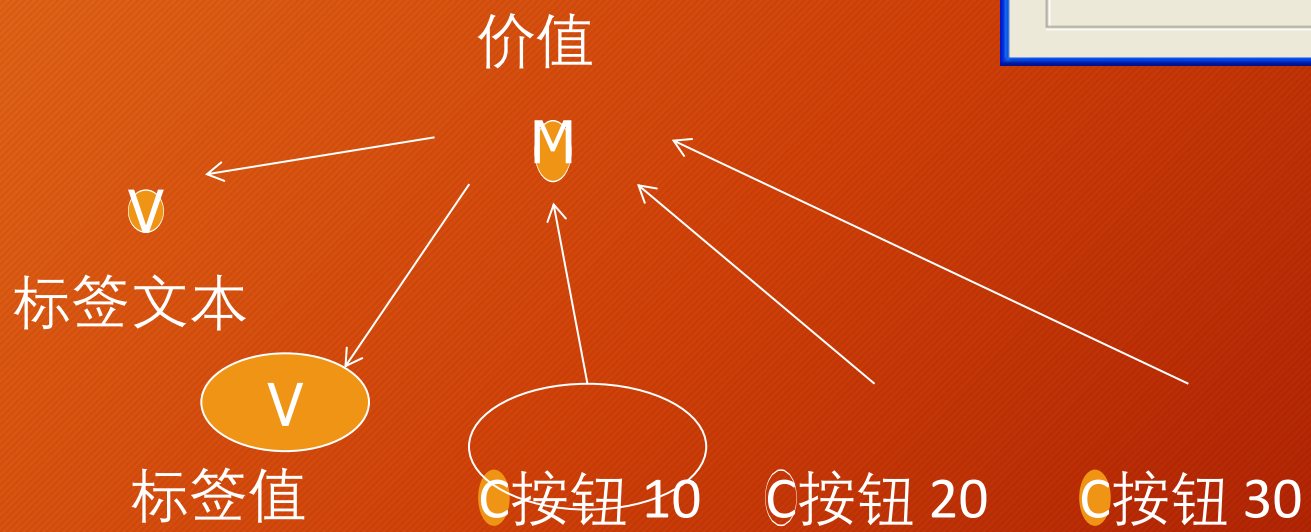




MVC 练习

- 案例 2


Exercice MVC-1
10
20
30
Valeur en numérique
0
Valeur en lettre
Zéro


价值
M
V
标签文本
V
标签值
C按钮 10
C按钮 20
C按钮 30

Exercice MVC: modification pour les 3 contrôleurs et les 2 vues

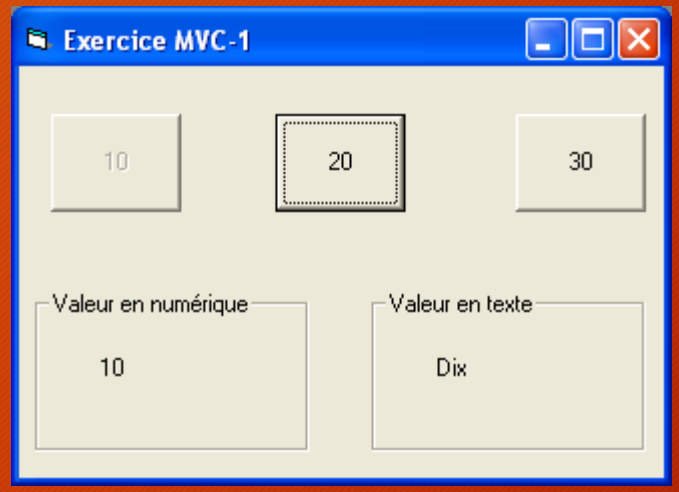
- …

MVC 示例：针对 3 个控制器与 2 个视图的修改

- …

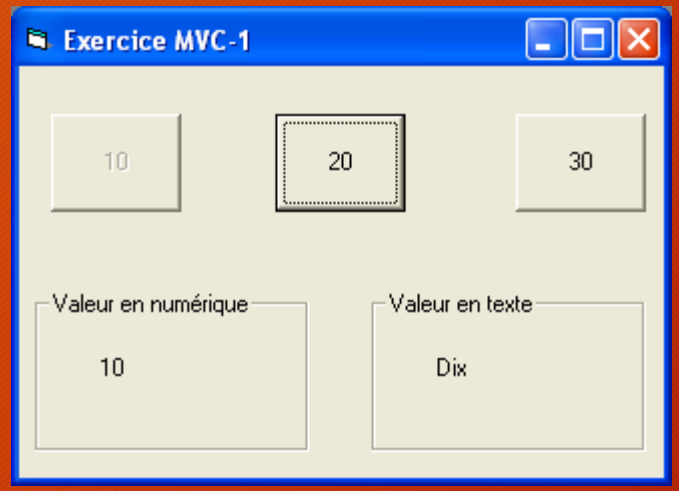
Exercice MVC-3

- Mélange états modèles et design patterns



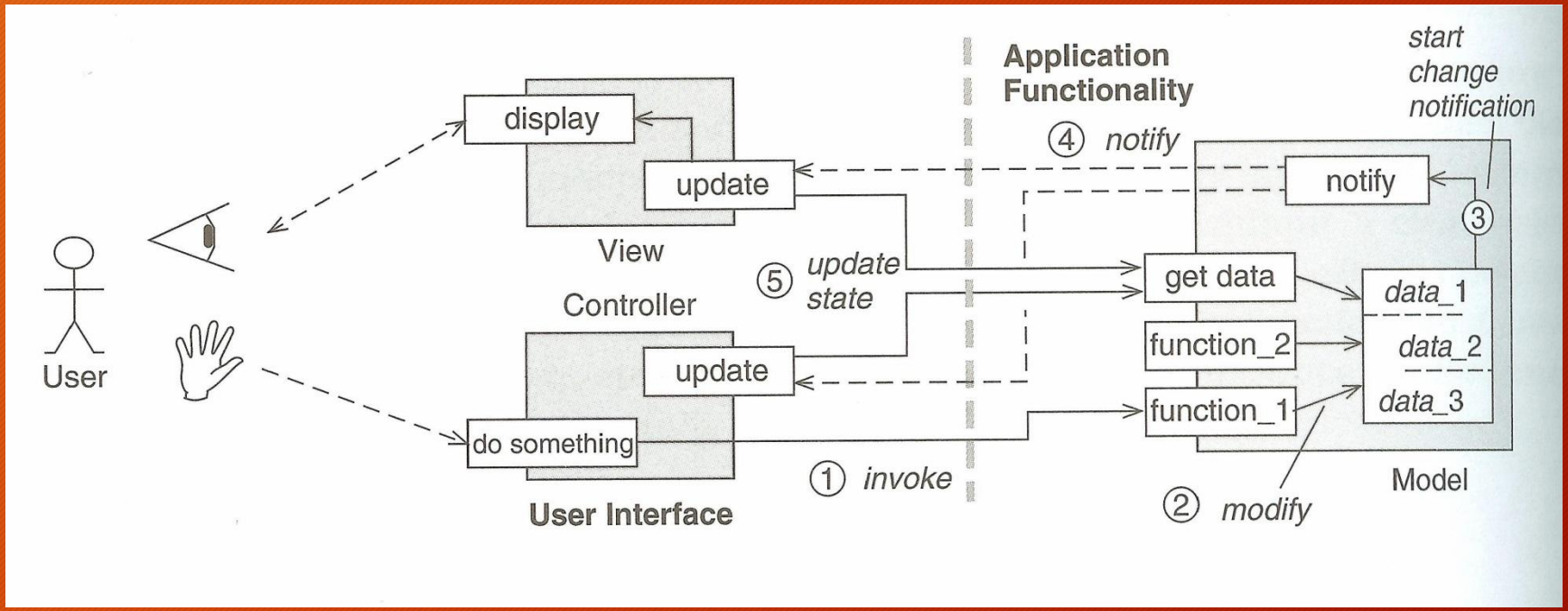
MVC-3 练习

- 状态模式与设计模式的结合



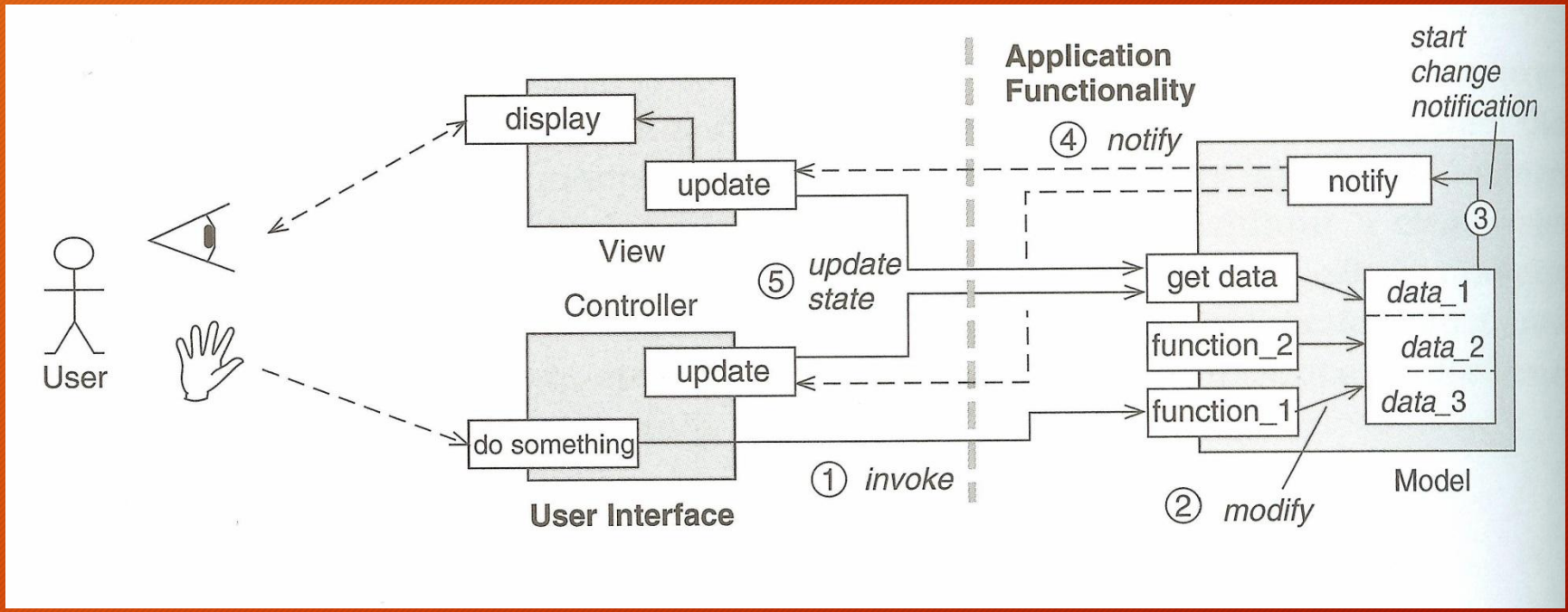
MVC : version « moderne »

- Buschmann F., Henney K., Schmidt D.C., Pattern-Oriented Software Architecture, A Pattern Language for Distributed Computing, Vol 4, Wiley Series, 2007



MVC：「现代」版本

- Buschmann F.、Henney K.、Schmidt D.C.，《面向模式的软件架构——分布式计算模式语言》，第 4 卷，Wiley系列丛书，2007



Design Patterns Comportementaux

Visitor

行为设计模式

访客

Design Pattern Visitor

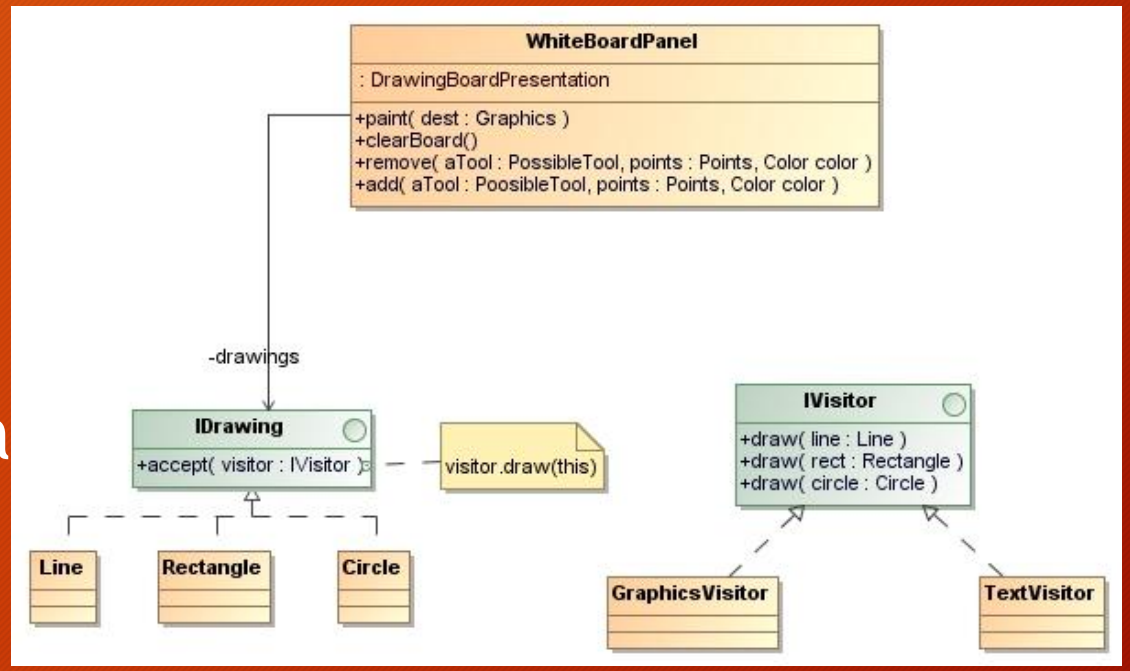
- Utilité : permettre de rajouter facilement des fonctionnalités à une hiérarchie de classes sans retoucher à la hiérarchie
- Quelle place pour visitor dans l'outil de dessin ?

设计模式：访问者

- 用途：可在不修改类层次结构的前提下，轻松为类层次结构添加新功能。
- 在绘图工具中，访客应使用哪个位置？

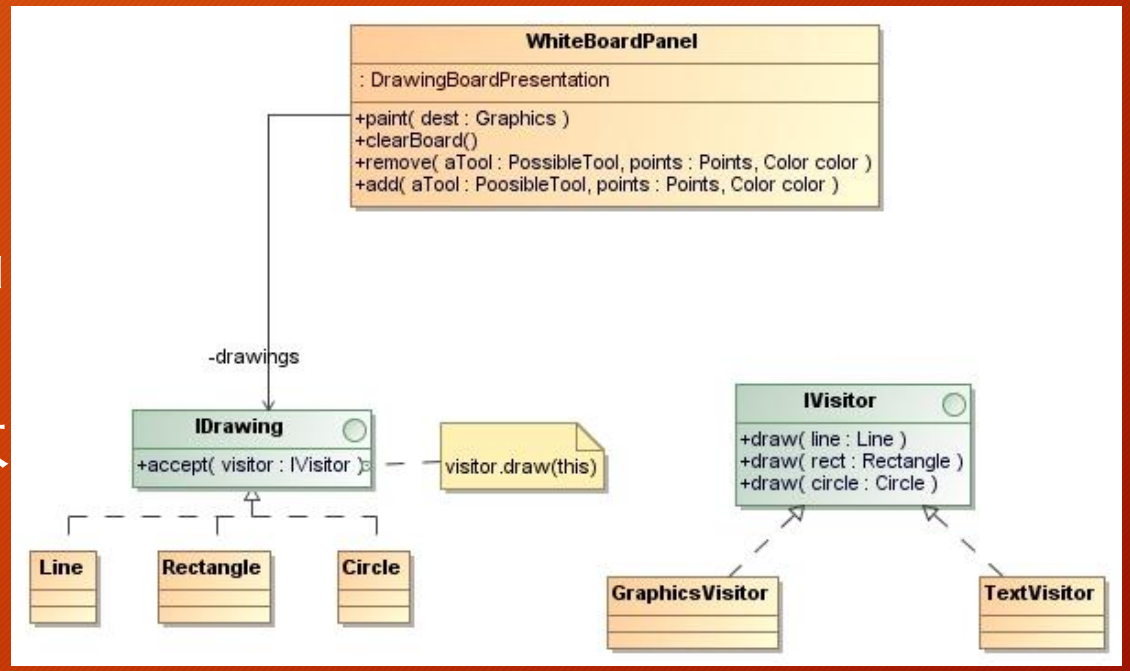
Visitor : Diagramme de classe

- Visiteur Graphique utilisé pour faire le dessin swing
- Visiteur Texte utilisé en passant par un menu de la fenêtre pour simuler la sauvegarde



访客：类图

- 用于进行摇摆绘图的图形访问器
- 访客：通过窗口中的菜单使用此文本来模拟备份操作



Design Patterns Comportementaux

Command

行为设计模式

命令

Undo / Redo : brutal

- Sauvegarder après chaque action l'état complet de l'application
  - Fonctionne, mais
  - Pb de capacité de stockage
- Idée, stocker le strict minimum à chaque action
  - Utilisation du pattern Command

撤销/重做：粗暴

- 每次操作后保存应用程序的完整状态
  - 可以运行，但
  - 存储容量铅

- 原则：每次操作时仅存储必要的最小量。
  - 使用 Command 模式

Pattern Command : principe

- Objectif : réifier une action et permettre de l'attacher à l'objet auquel elle se rapporte
- Permet la mise en œuvre de concepts transversaux aux IHM
  - Macro (liste de commandes regroupées)
  - Undo/Redo
  - Logging, Recover
  - Copy/Cut/Paste
  - Actions différées

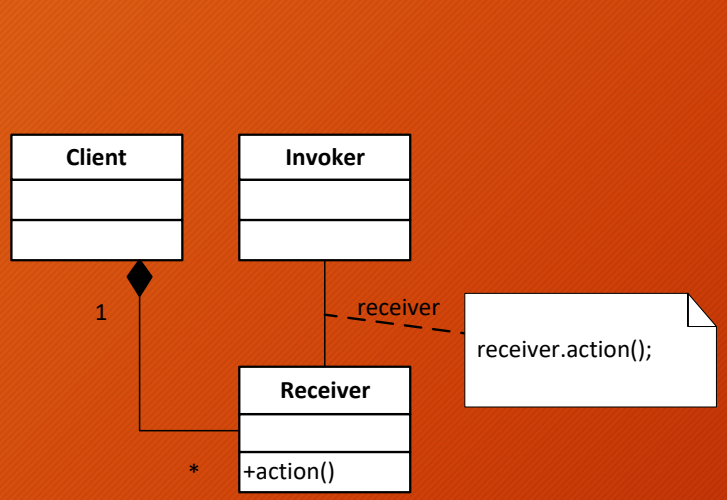
模式命令：原理

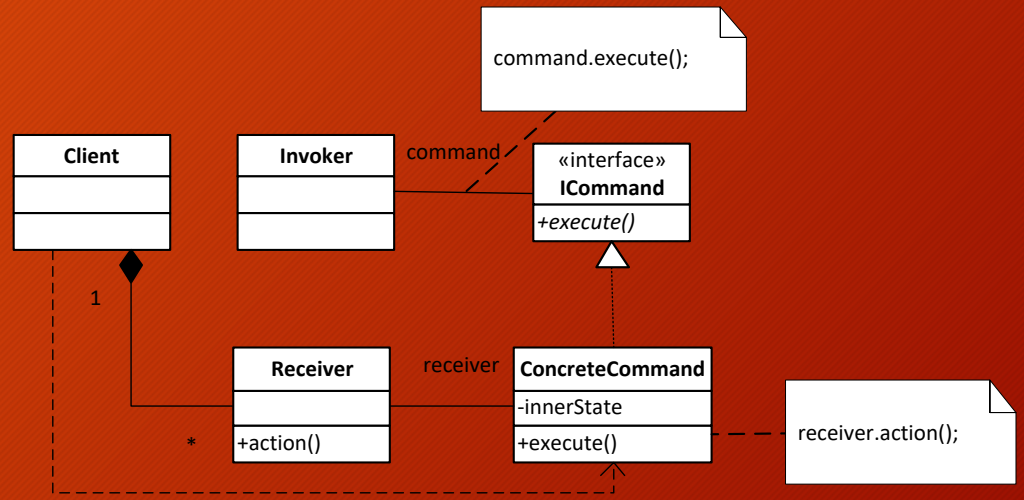
- 目标：实现一项操作，并将其与所关联的对象绑定。
- 助力在健康信息学（IHM）领域实施跨领域概念
  - 宏（批量订单列表）
  - 撤销/重做
  - 日志记录，恢复
  - 复制/剪切/粘贴
  - 延迟行动

Pattern Command : Structure

- Client = Application
- Invoker = Item de menu par exemple
- Receiver = Document ou application

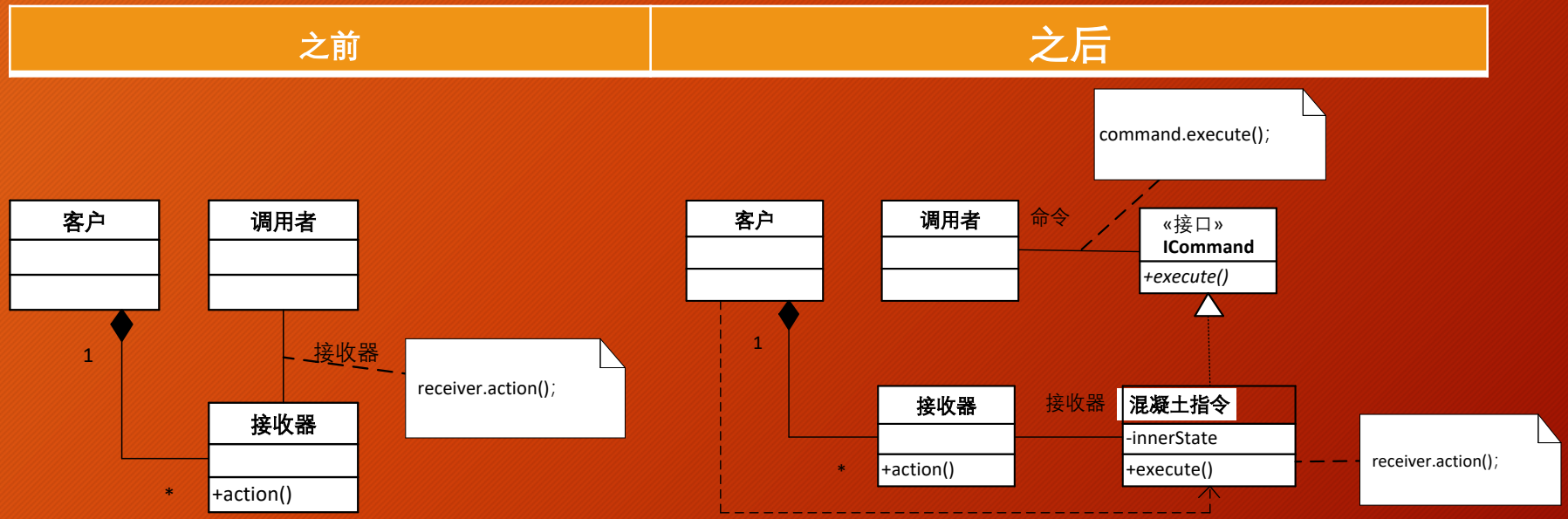
Avant	Après





模式命令：结构

- 客户端 = 应用程序
- Invoker = 例如菜单项
- 接收方 = 文档或应用程序



Undo / Redo : pattern Command

- Un objet command peut offrir un service de undo
- Pour assurer le undo, il faut que l'objet possède suffisamment d'information
  - Référence au receveur
  - Arguments de la méthode à invoquer
  - Le receveur doit fournir une méthode qui permettra de revenir en arrière

撤销/重做：模式命令

- 一个命令对象可提供撤销服务。
- 为确保撤销操作，对象必须具备足够的信息。
  - 关于接收方的说明
  - 待调用方法的参数
  - 接收方必须提供一种可回退的方法。

Undo / Redo : Comment ?

- La classe commande propose une méthode undo
- Le client doit gérer une historisation
  - Quelle structure de données ?
  - Quelle capacité

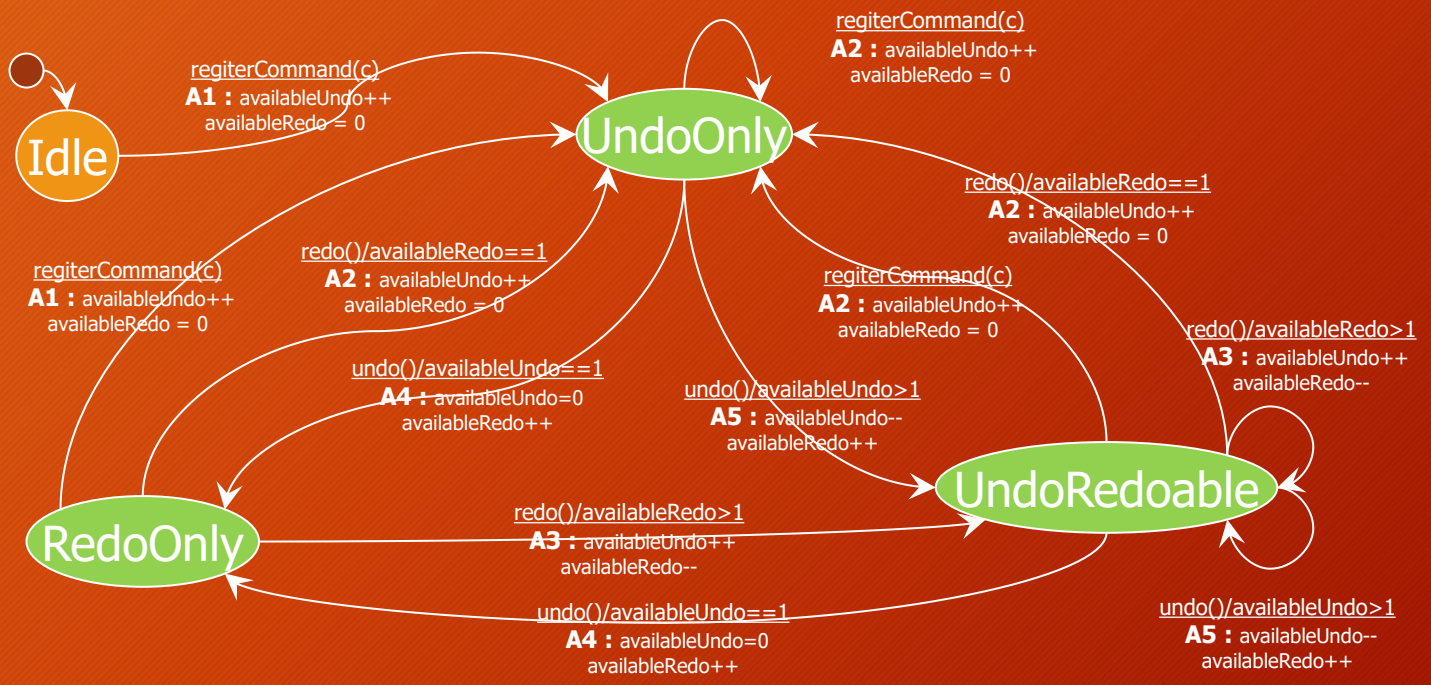
撤销/重做：有何意见？

- 命令类提供了一个撤销方法。
- 客户需管理历史记录功能。
  - 采用何种数据结构？
  - 何种容量

Undo/Redo : Comportement

Events:
- registerCommand(c)
- undo()
- redo()

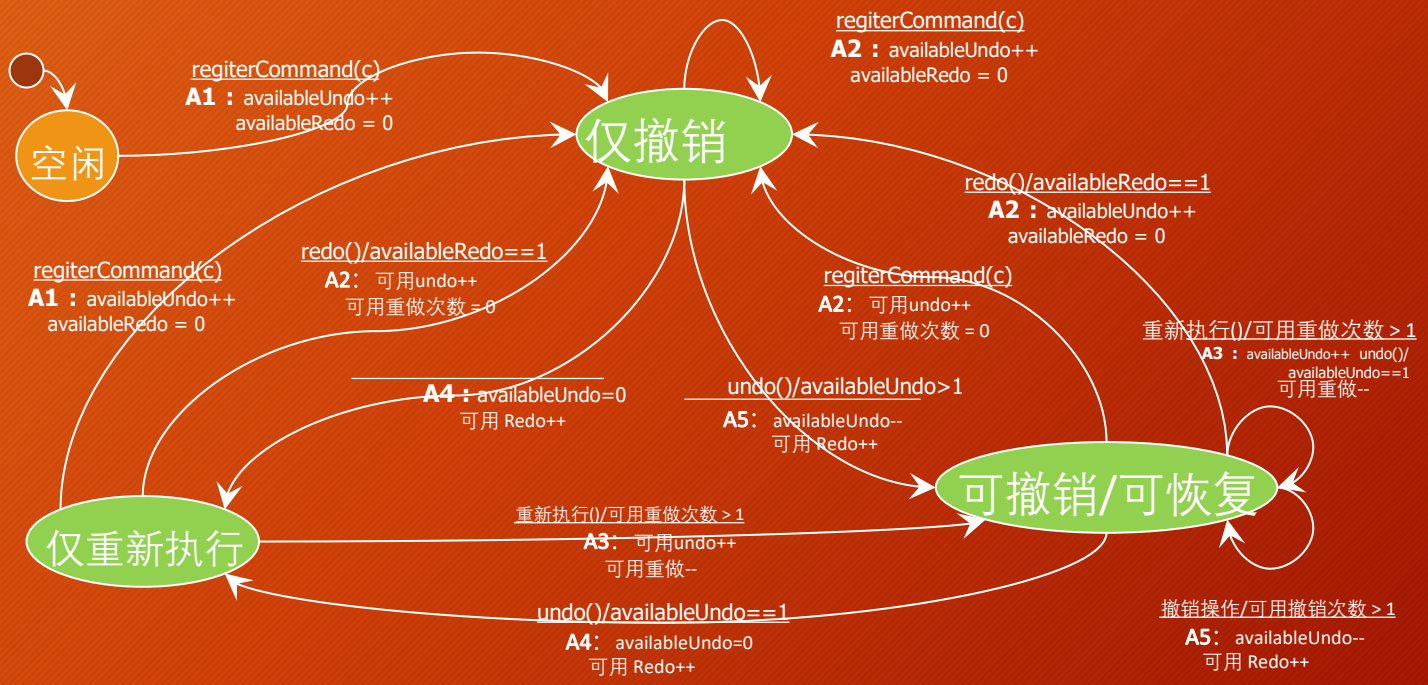
Actions:
-availableUndo, availableRedo : Collection<Command>
-availableXXX++ ⇔ add c to collection
-availableXXX-- ⇔ remove c from collection
-availableXXX = 0 ⇔ clear collection



撤销/重做：行为

活动:
- registerCommand(c)
- undo()
- 重新执行()

操作:
-可用撤销、可用重做：命令集合
-可用XXX++⇔ 将 c 添加到集合中
-可用XXX- -⇔ 从集合中移除元素 c
-可用XXX = 0⇔ 清收



Undo / Redo : Java Swing

- public class **UndoManager**
  extends CompoundEdit
  implements UndoableEditListener

```
JTextField tf = ...;
tf.getDocument().addUndoableEditListener(undoManager);
```

撤销/重做：Java Swing

- 公共类**UndoManager**
  继承自CompoundEdit
  实 现 了 UndoableEditListener JTextField

```
tf = . . . ;
tf.getDocument().addUndoableEditListener(undoManager);
```

Variantes de MVC

MVC 的变体

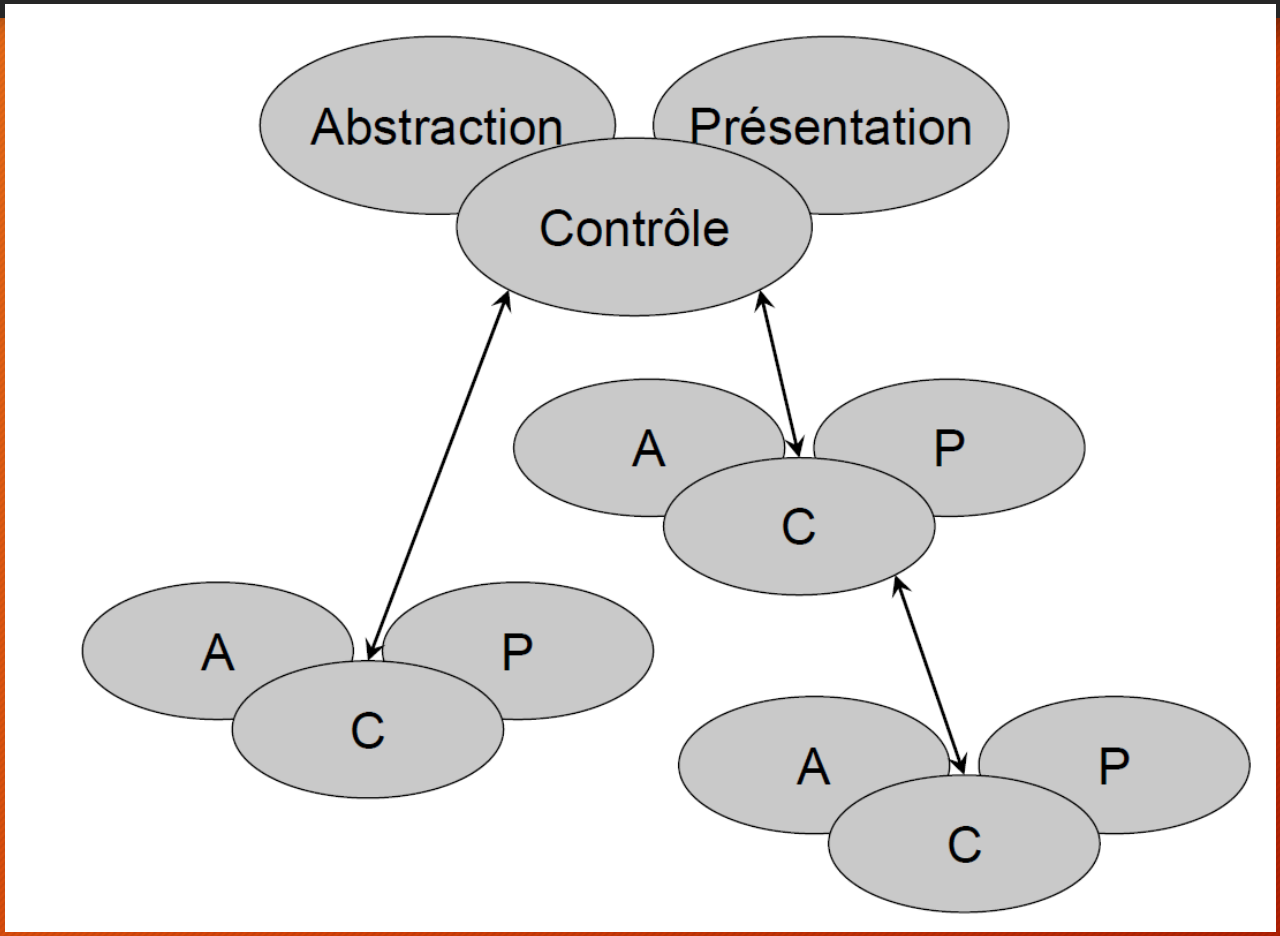
PAC : Définitions

- Joëlle Coutaz, 1987
- Présentation
  - définit le comportement, en entrée comme en sortie, de l'application par rapport à l'utilisateur
  - ne communique qu'avec son contrôleur
- Abstraction
  - les fonctionnalités de l'application
  - ne communique qu'avec son contrôleur
- Contrôle
  - maintien de la cohérence entre présentation et abstraction
  - peut communiquer avec d'autres contrôleurs

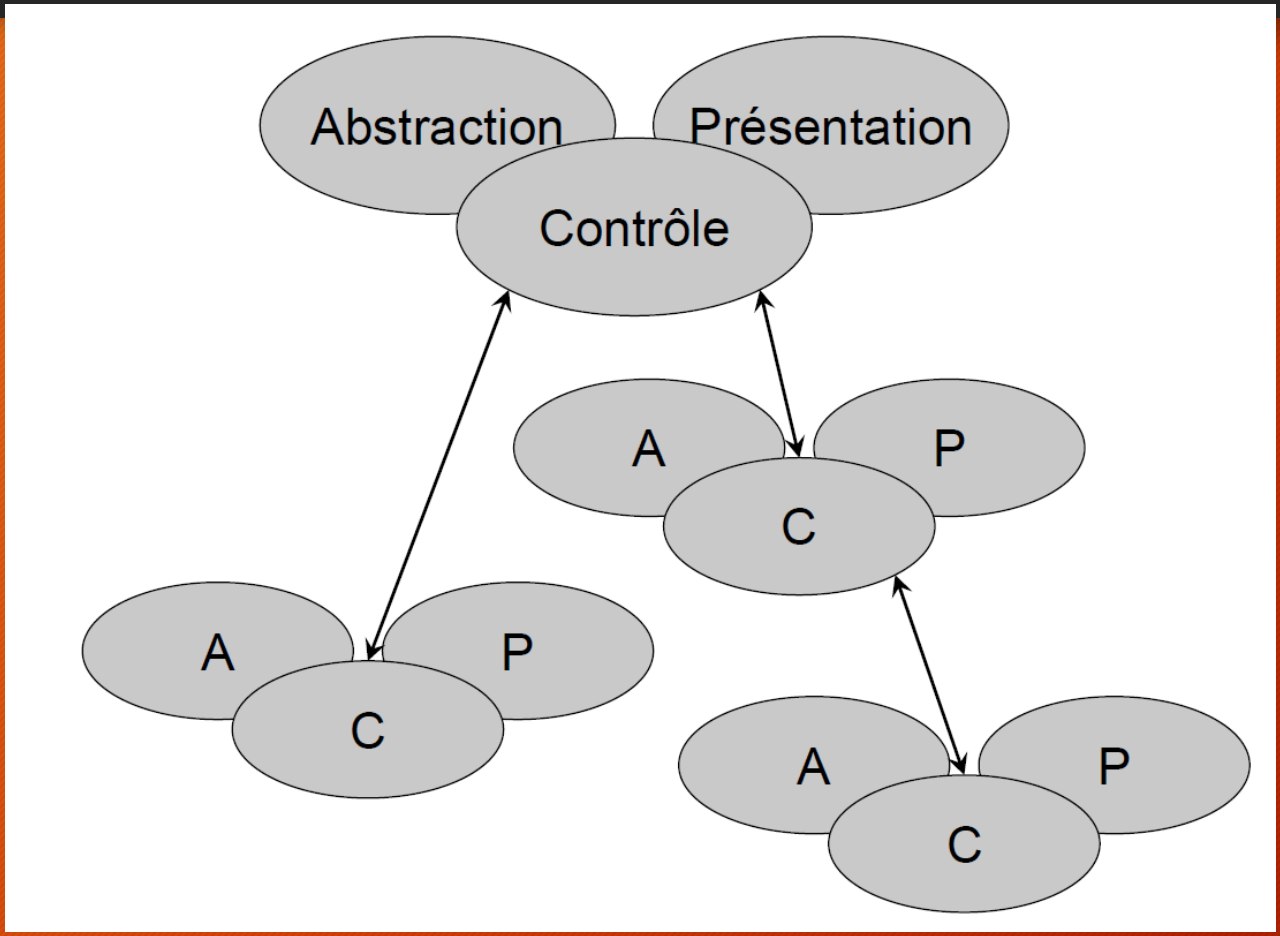
PAC：定义

- 若埃尔·库塔兹，1987年
- 介绍
  - 定义了应用程序相对于用户的输入与输出行为。
  - 仅与其主管沟通
- 抽象
  - 应用功能
  - 仅与其主管沟通
- 控制
  - 确保阐述与抽象之间的连贯性
  - 可与其他控制器进行通信。

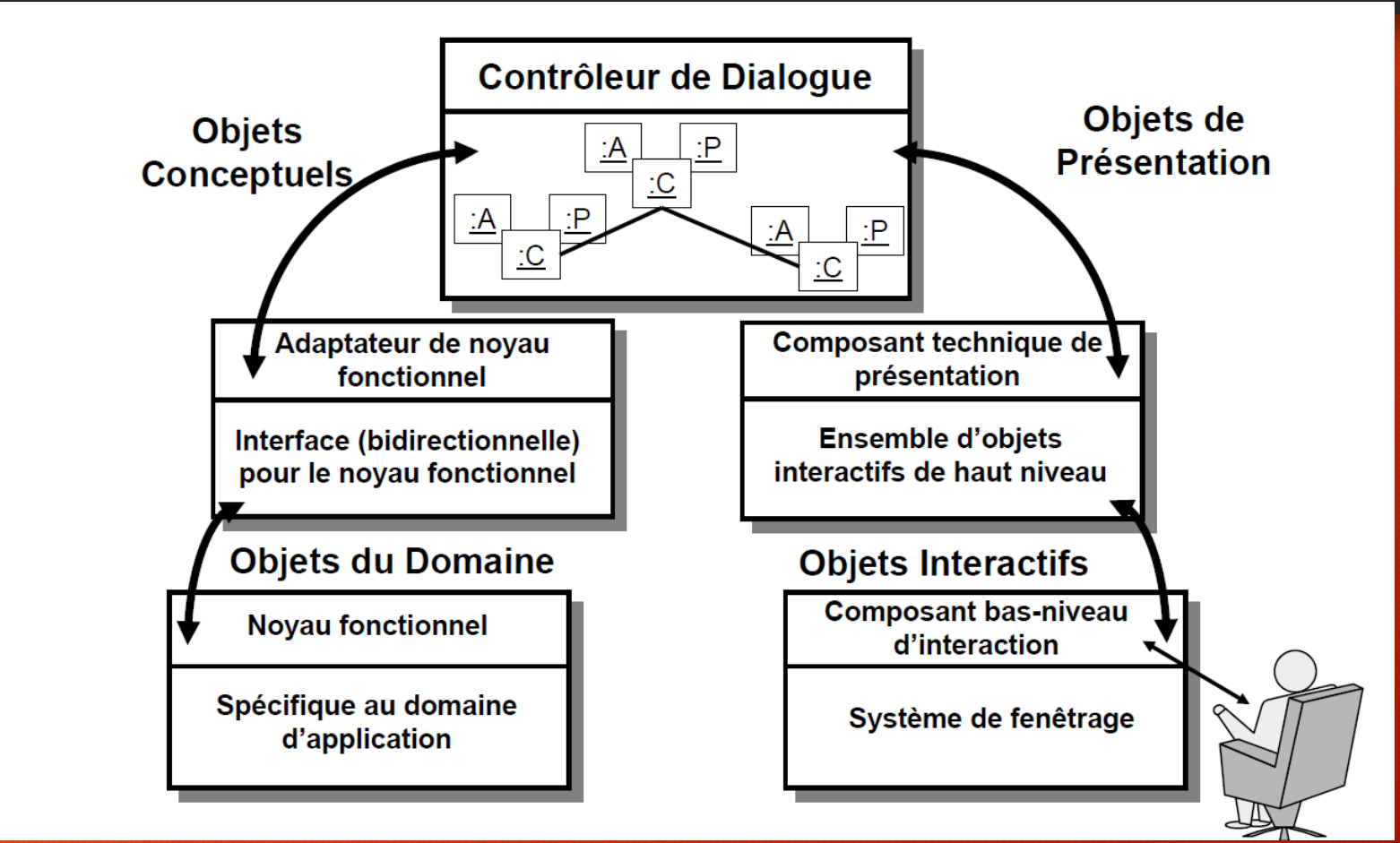
Presentation-Abstraction-Control (PAC)



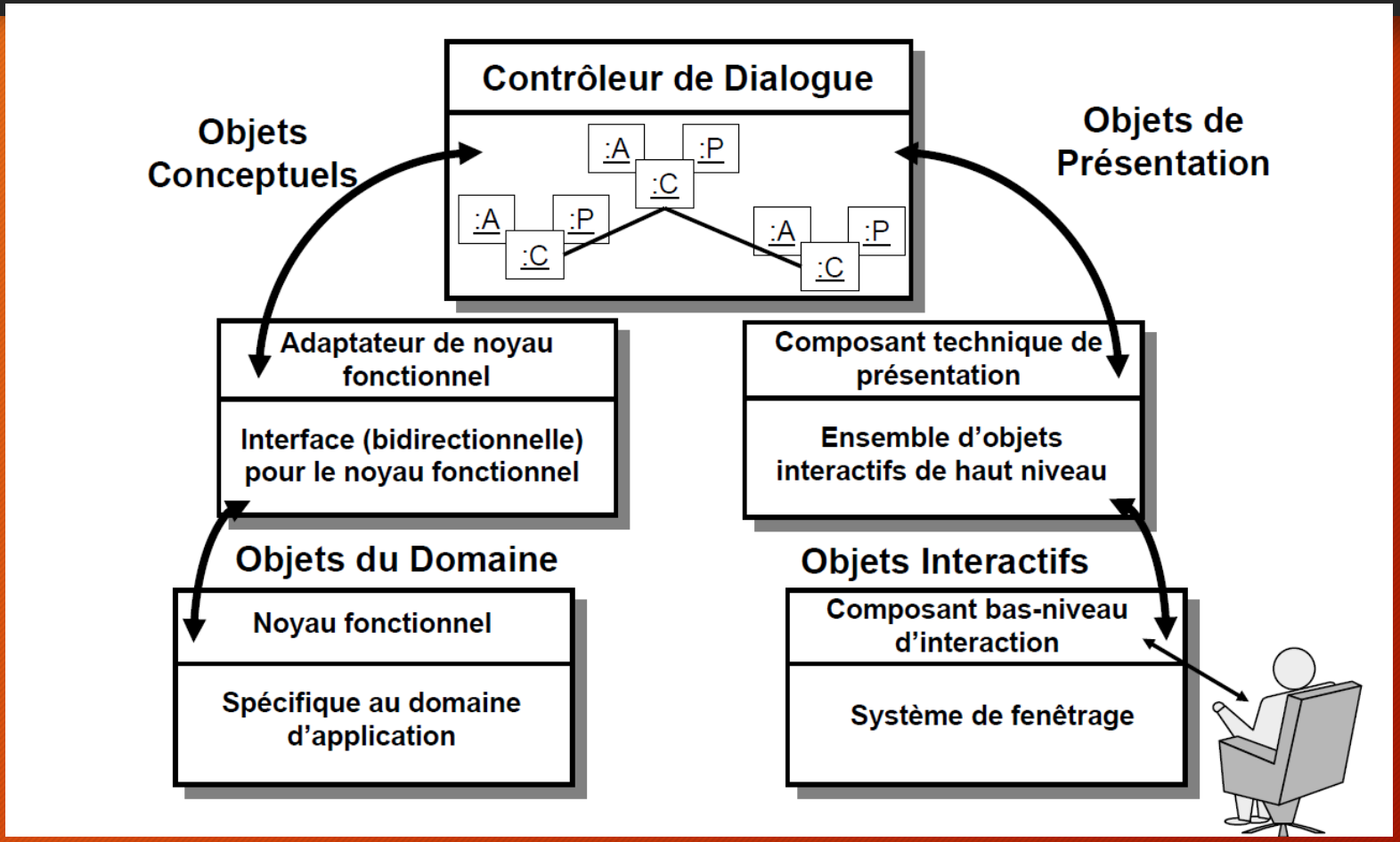
呈现-抽象-控制（PAC）



PAC-Amodeus

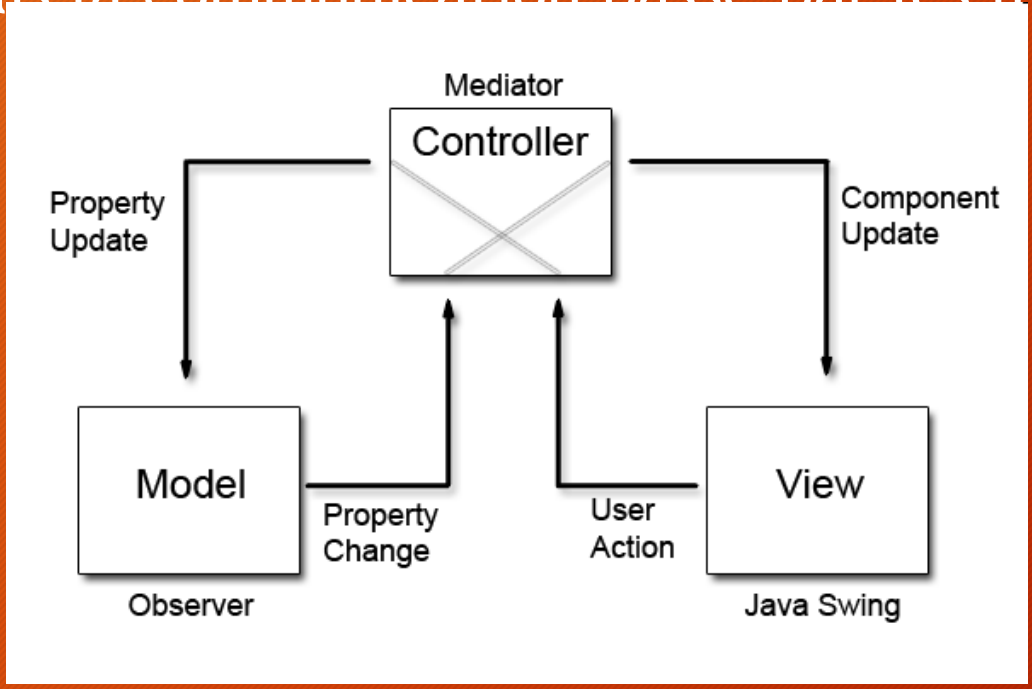

Contrôleur de Dialogue
:A
:P
:C
:A
:P
:C
:A
:P
:C
Objets Conceptuels
Objets de Présentation
Adaptateur de noyau fonctionnel
Interface (bidirectionnelle) pour le noyau fonctionnel
Composant technique de présentation
Ensemble d'objets interactifs de haut niveau
Objets du Domaine
Objets Interactifs
Noyau fonctionnel
Spécifique au domaine d'application
Composant bas-niveau d'interaction
Système de fenêtrage

PAC-Amodeus


Contrôleur de Dialogue
:A
:P
:C
:A
:P
:C
:A
:P
:C
Objets Conceptuels
Objets de Présentation
Adaptateur de noyau fonctionnel
Interface (bidirectionnelle) pour le noyau fonctionnel
Composant technique de présentation
Ensemble d'objets interactifs de haut niveau
Objets du Domaine
Noyau fonctionnel
Spécifique au domaine d'application
Objets Interactifs
Composant bas-niveau d'interaction
Système de fenêtrage

Modified MVC (Oracle/Sun)
OS X COCOA (Apple)

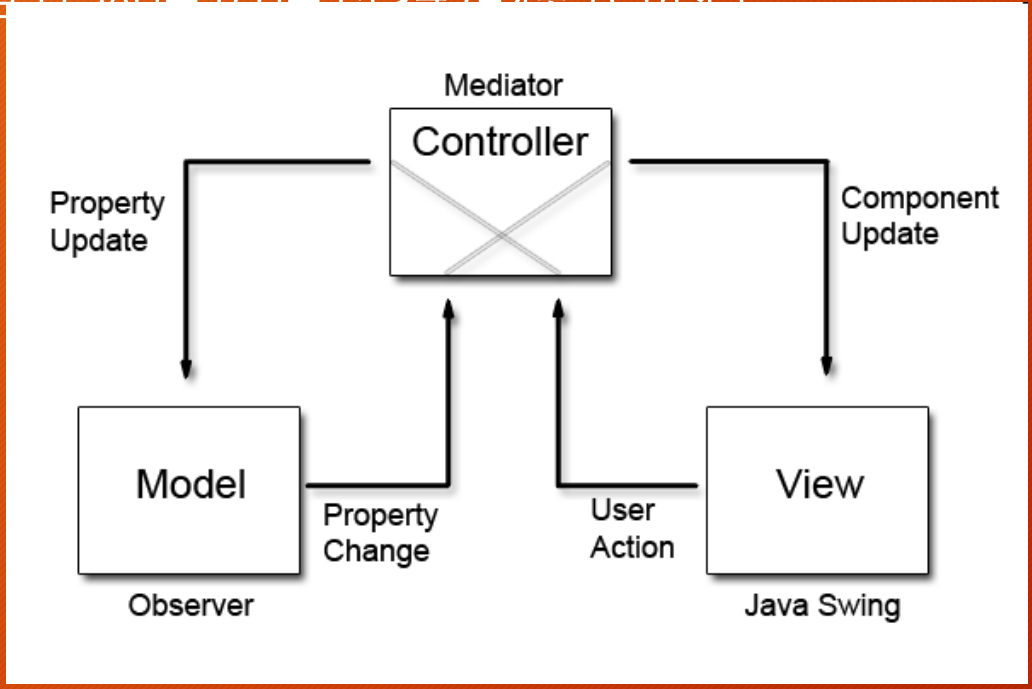
- Robert Eckstein, Mars 2007
- Toute communication passe par le contrôleur
- PAC mais la communication passe par les modèles





改良版 MVC（Oracle/Sun）OS X COCOA（Apple）

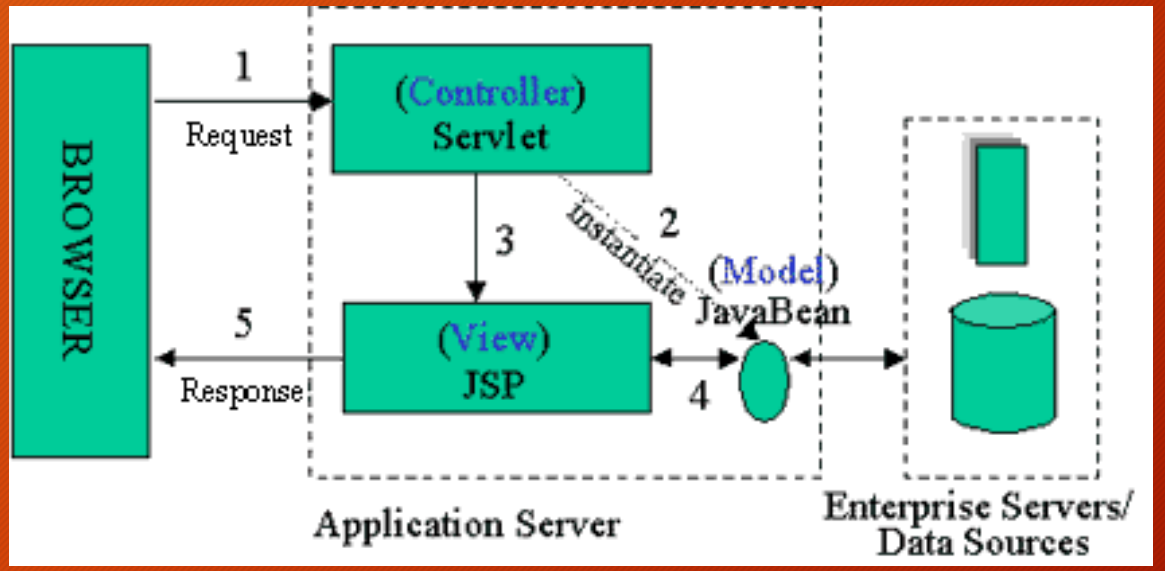
- 罗伯特·埃克斯坦，2007年3月
- 所有通信均经由控制器进行。
- PAC，但其沟通仍通过特定模式进行





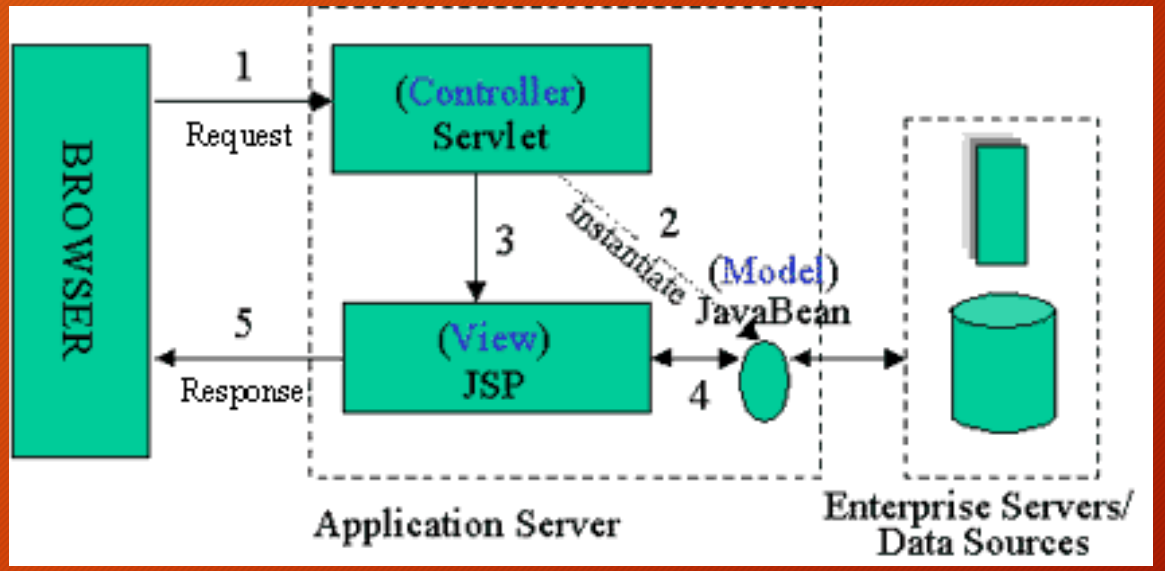
MVC 2

- JavaServer Pages
- Contrôleur unique
- Page affichée sélectionnée par le contrôleur



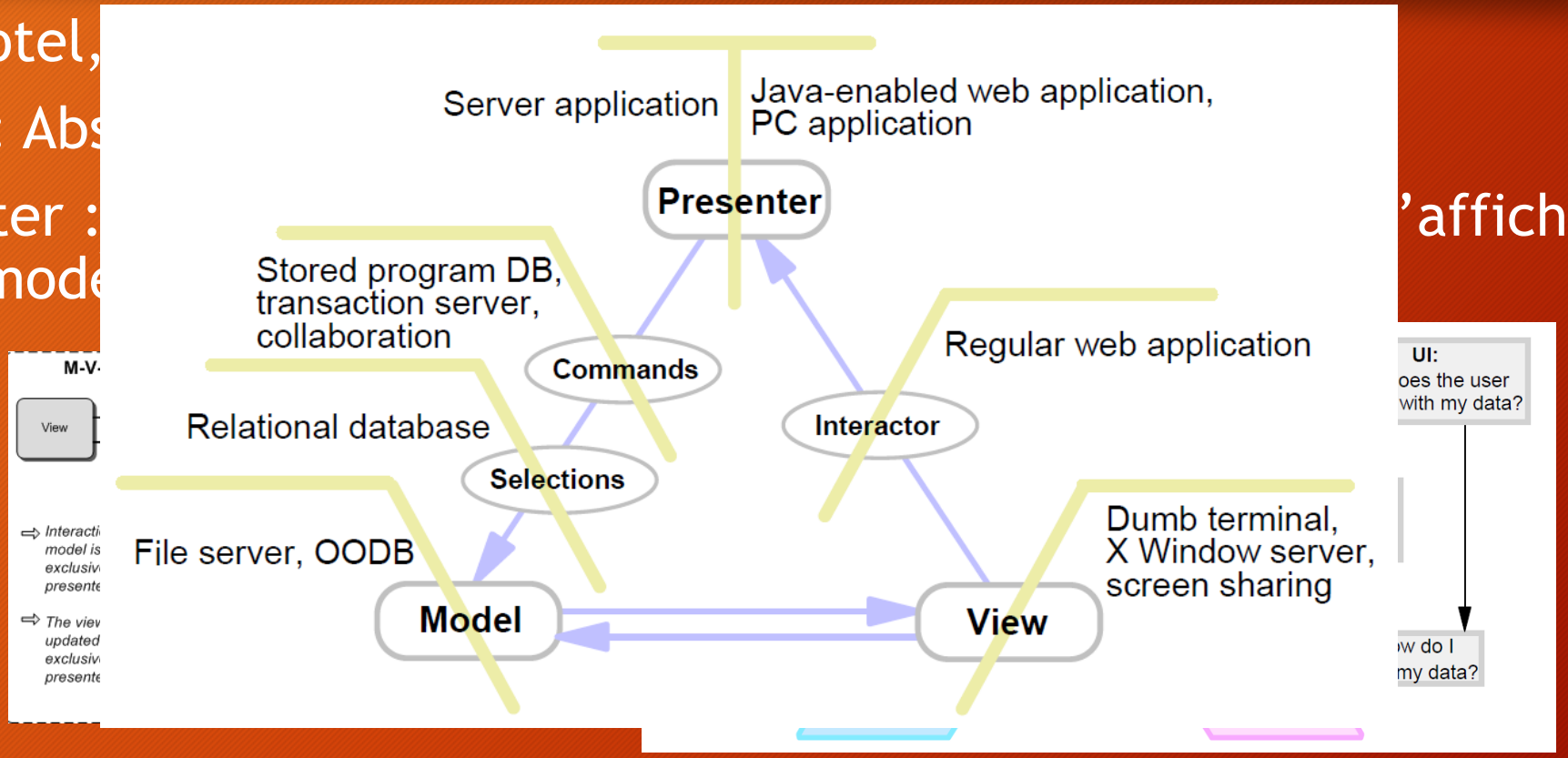
MVC 2

- JavaServer Pages
- 单一控制器
- 由控制器选中的页面



Model-View-Presenter (MVP)

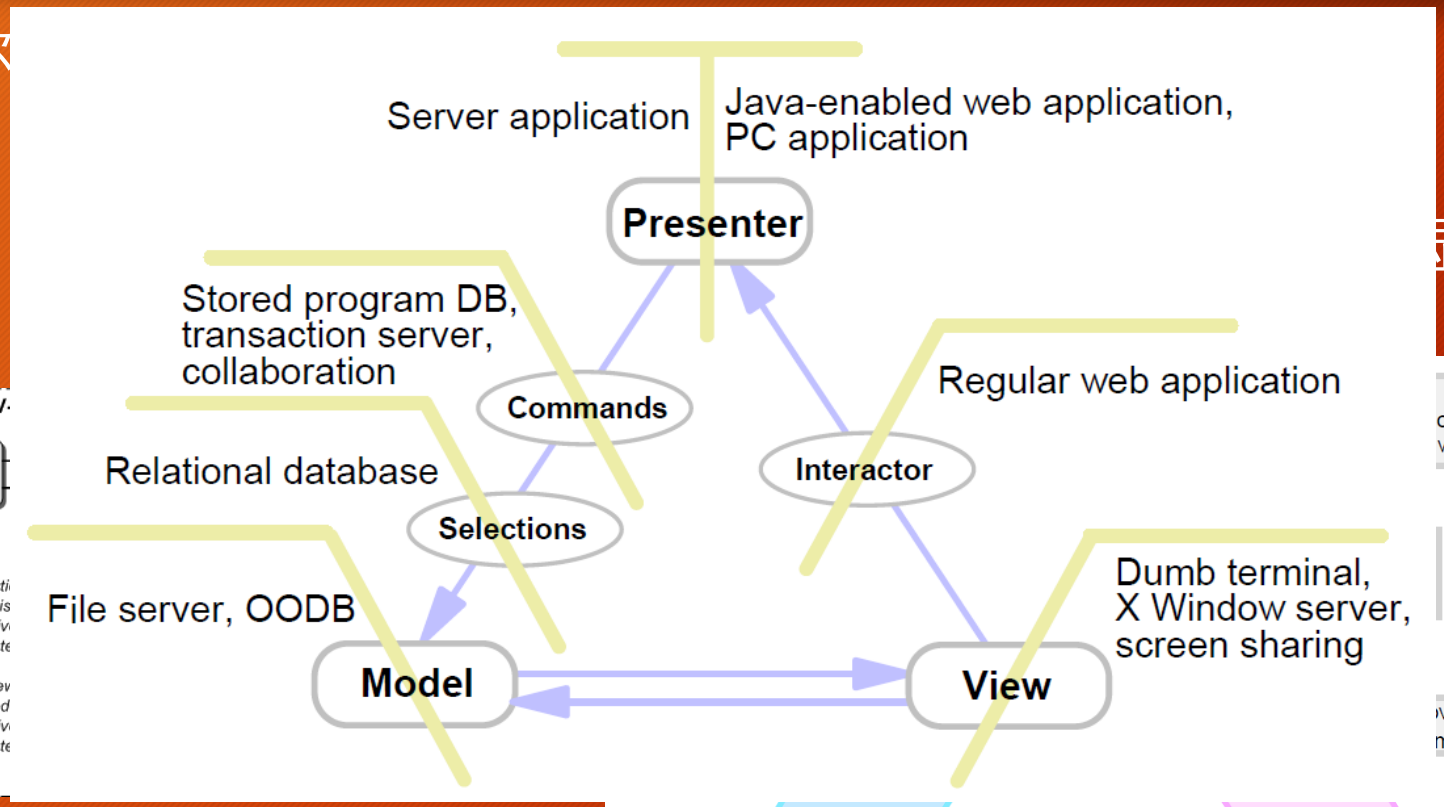
- Mike Potel,
- Model : Abs
- Presenter : ’affichage et agit sur le mode



模型-视图-控制器（MVP）

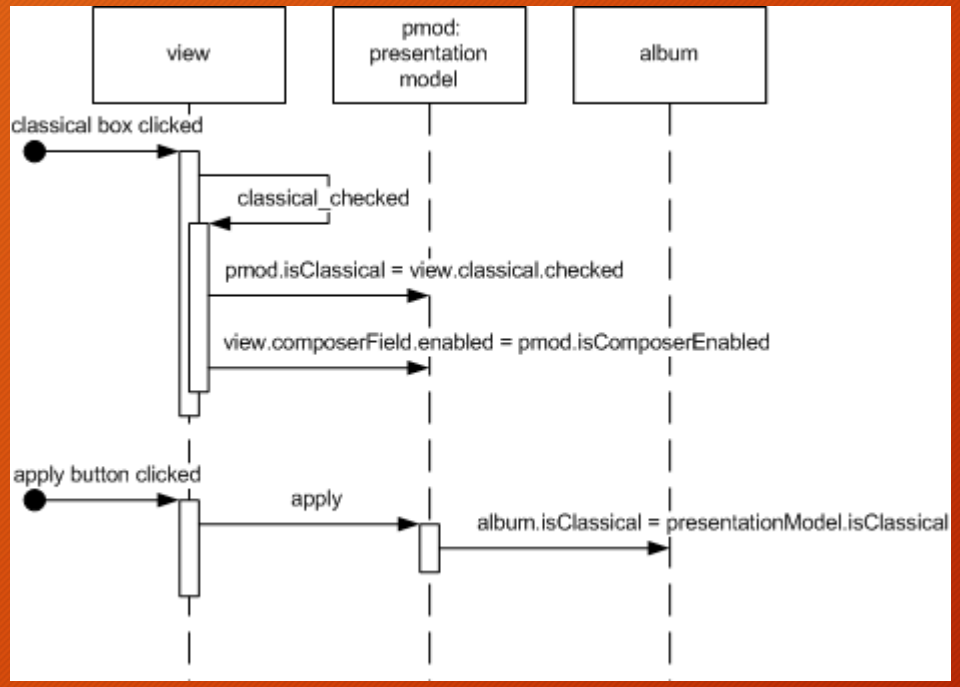
- 迈克·波特尔
- 模型：抽象
- 演讲者：
  基于该模型

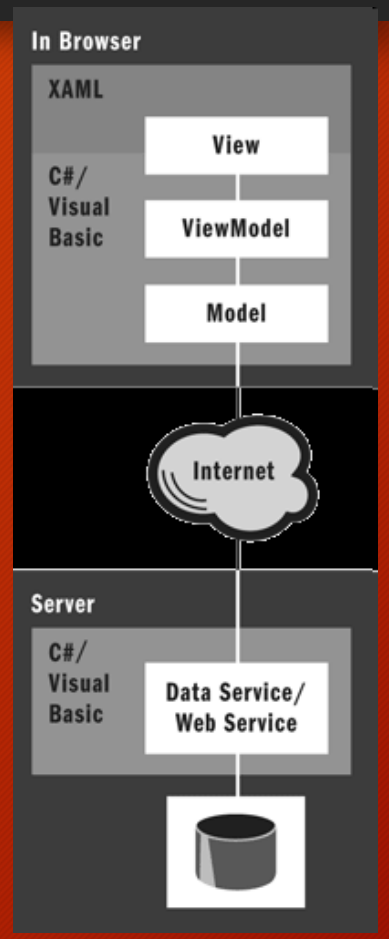
显示并执行



Model-View-ViewModel (MVVM) et Presentation Model (PM)

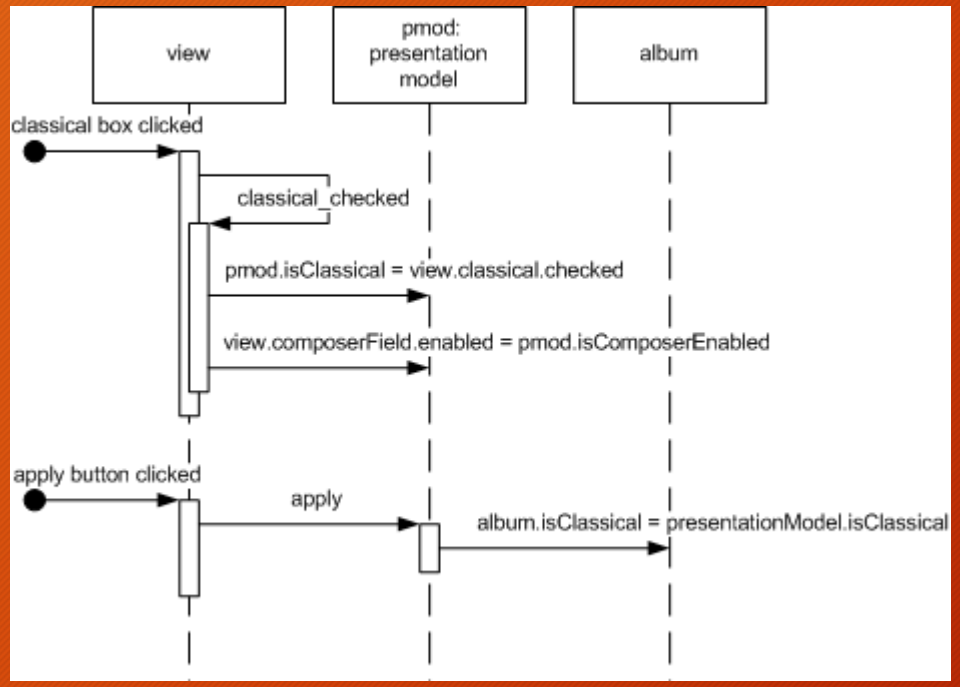
- PM Martin Fowley, 2004
- MVVM John Gossman(Microsoft), 2005, spécialisation de PM pour silverlight
- PM est une abstraction de la Vue, indépendante de la plateforme

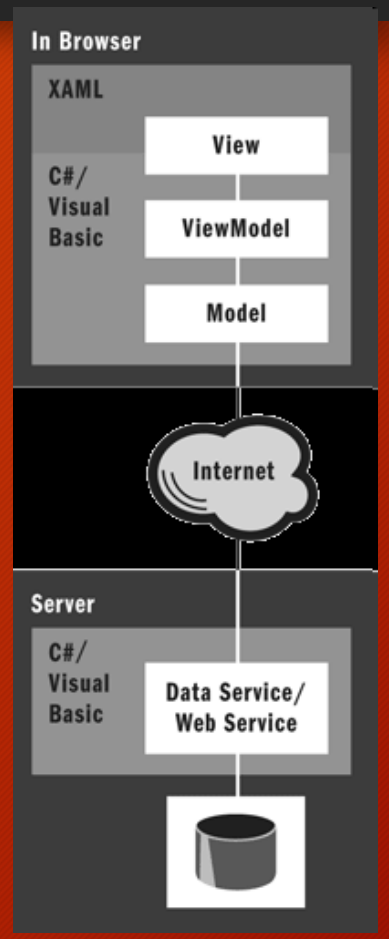




模型-视图-模型（MVVM）与呈现模型（PM）

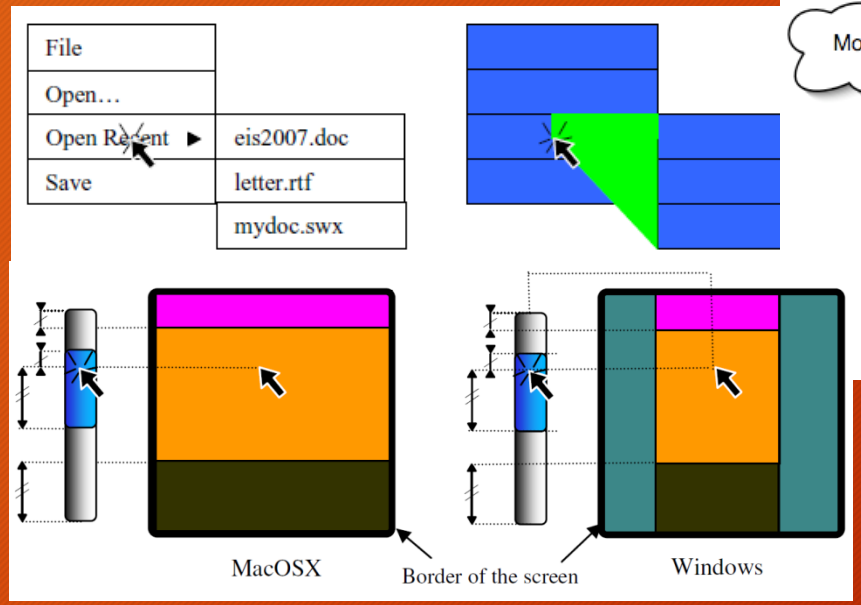
- 总理马丁·福利，2004年
- MVVM，John Gossman（微软），2005年，专为 Silverlight 开发的 PM 框架
- PM 是对视图的抽象，与具体平台无关。

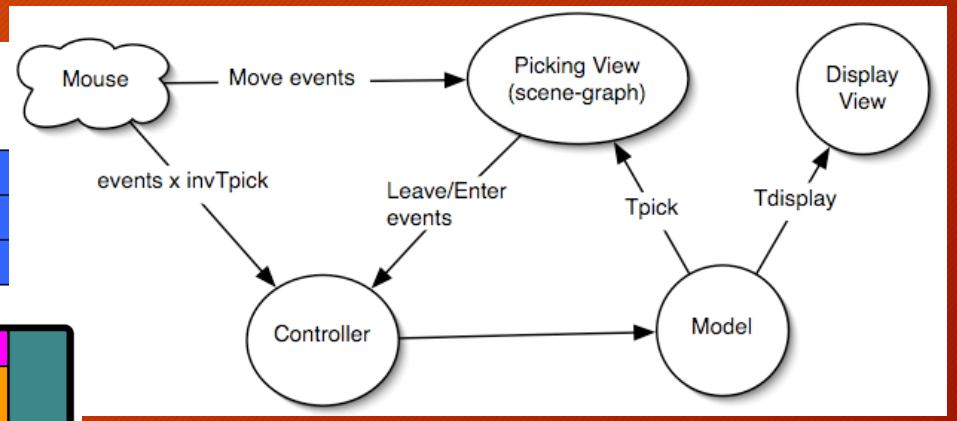




Model - Display View – Picking View – Controller (MDPC)

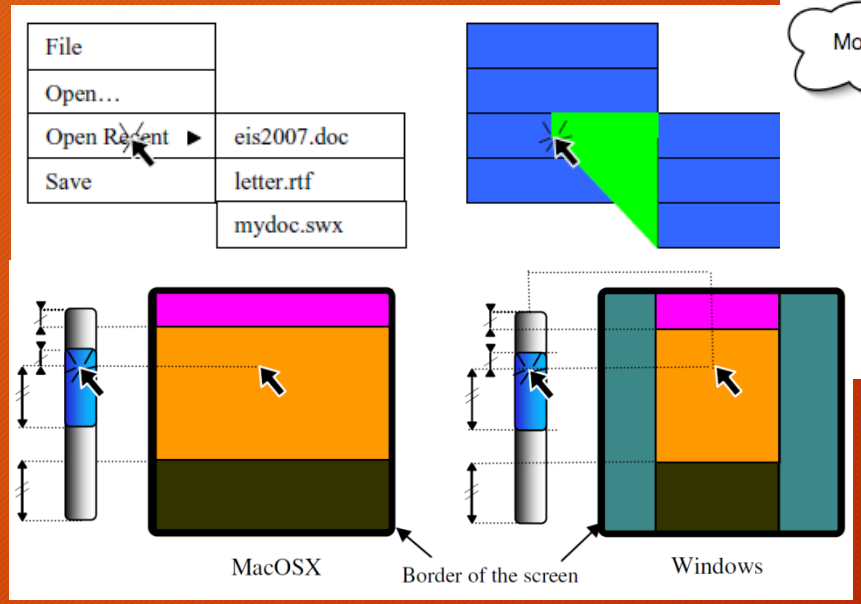
- Conversy, S., Barboni, E., Navarre, D., Palanque, P. "Improving modularity of interactive software with the MDPC architecture" EIS 2007.
- Vue invisible facilitant la description de l'interaction

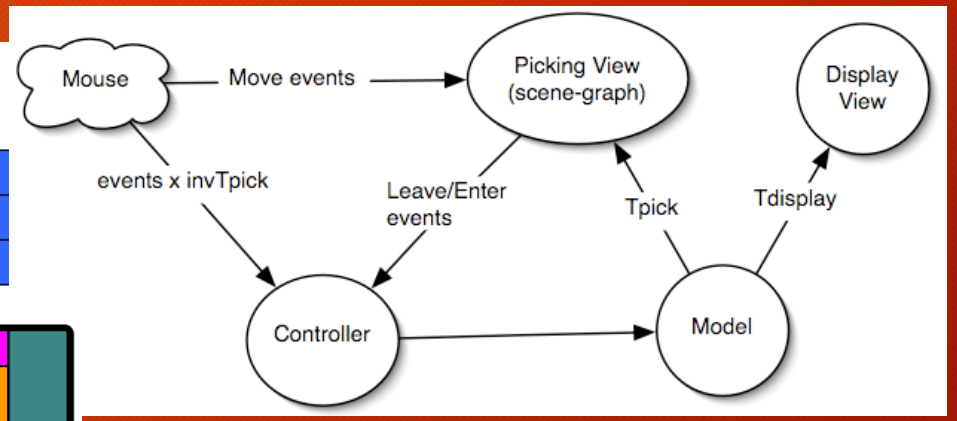




模型 - 显示视图 – 拾取视图 –控制器 (MDPC)

- Conversy，S.、Barboni，E.、Navarre，D.、Palanque，P. “利用 MDPC 架构提升交互式软件的模块化程度”——EIS 2007。
- Vue 的不可见性有助于简化交互描述





Quelle architecture choisir ?

- Un problème, une architecture ?

- Dépend de ce qui doit être « absorbé » par l'architecture
  - Au moment de la conception du système
  - Evolution du système

该选择哪种架构?

- 是一个问题，还是一种架构?

- 这取决于建筑体系需要“吸收”的内容。
  - 在系统设计阶段
  - 系统演进

Les design patterns comme support à l'utilisabilité

将设计模式作为提升可用性的支撑手段

De nombreux design patterns

- Archétypes de solution à des problèmes récurrents dans la conception de logiciel
- Bibliographie
  - Design Patterns, Elements of Reusable Object-Oriented Software, E.Gamma, R. Helm, R. Johnson, J. Vlissides, Addison-Wesley, le «Gang of Four», 1995
  - A System of Patterns : Pattern-oriented software architecture, Buschmann, F., R. Meunier, H. Rohnert, P. Sommerlad, and M. A Stal, Wiley 1996
  - …
- Généralement regroupés en pattern touchant la création, la structure ou le comportement

众多设计模式

- 软件设计中常见问题的解决方案范式
- 参考文献
  - 《设计模式——可复用面向对象软件的基础》，E. Gamma，R. Helm，R. Johnson，J. Vlissides，Addison-Wesley 出版社，“四人小组”，1995年
  - 模式体系：面向模式的软件架构，Buschmann，F.、R. Meunier、H. Rohnert、P. Sommerlad 及 M. A. Stal，Wiley，1996年
  - …
- 通常被归类为涉及创作、结构或行为的模式。

Design patterns orientés Systèmes Interactifs

- MVC
- Event Handling
- Object for State
- Visitor
- Command
- Cancellation
- …

- Comment choisir ?

面向交互系统的设计模式

- MVC
- 事件处理
- 国家目标
- 访客
- 命令
- 取消 • …

- 如何选择?

Achieving Usability Through Software Architecture

- Len Bass, Bonnie E. John, Jesse Kates. Technical Report, CMU/SEI-2001-TR-005, March 2001

- Principes
  - Identification de scénarios d'utilisabilité
  - Organisation de ces scénarios en terme de bénéfice pour l'utilisabilité
  - Organisation des scénarios en terme de mécanismes logiciels

- Résultat : une classification des patterns

通过软件架构实现可用性

- Len Bass、Bonnie E. John、Jesse Kates。技术报告，CMU/SEI-2001-TR-005,2001年3月

- 原则
  - 可用性场景识别
  - 从可用性角度对这些场景进行效益评估
  - 按软件机制对场景进行组织

- 结果：模式分类

Scénarios d'utilisabilité et pattern correspondant

- Identification de scénarios d'utilisabilité qui ont un impact sur l'architecture logicielle
- Exemple
  - L'utili
  - L'utili
  - Ex: La

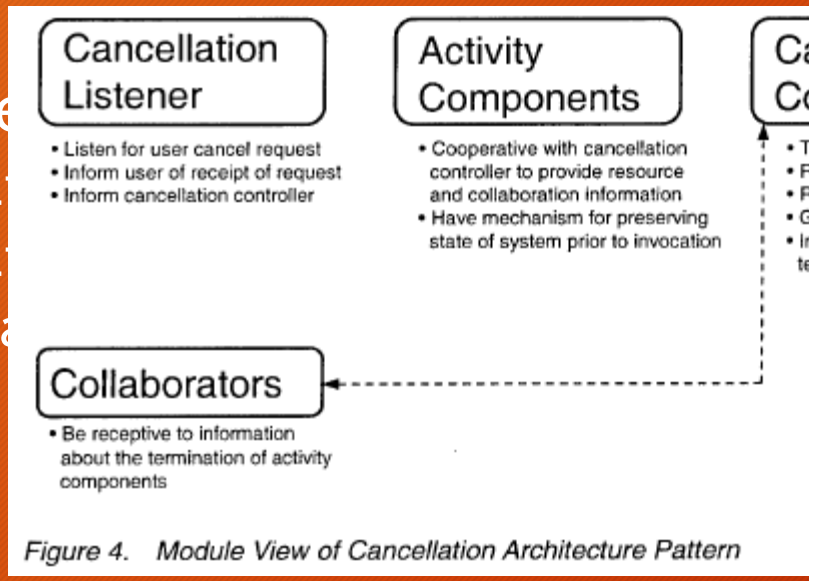


Figure 4. Module View of Cancellation Architecture Pattern

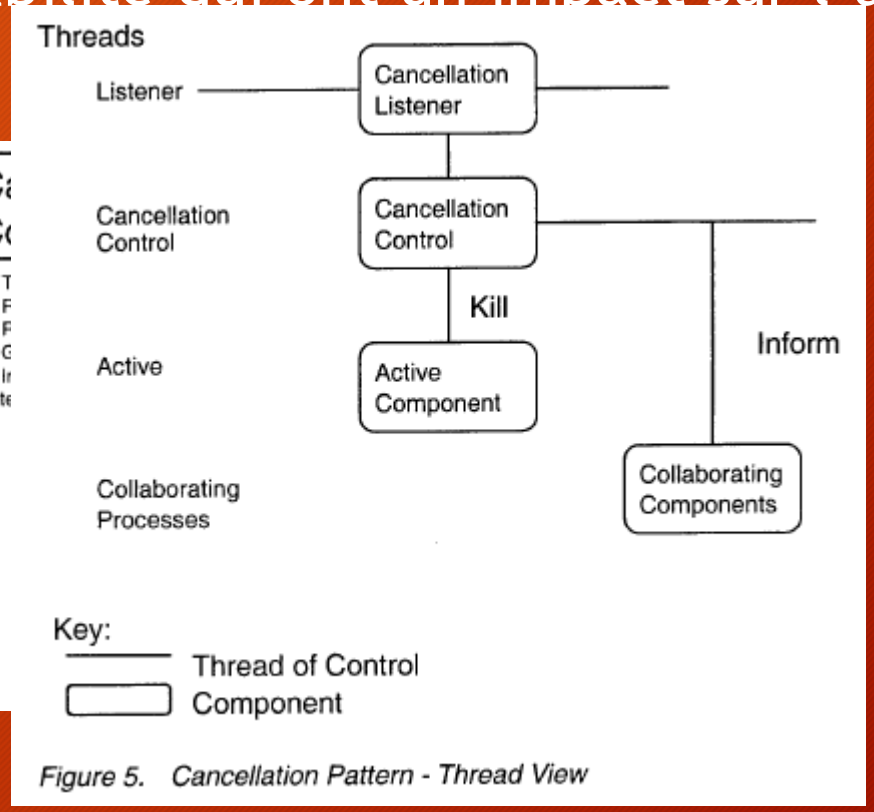


Figure 5. Cancellation Pattern - Thread View

可用性场景及对应模式

- 识别对软件架构具有影响的可用性场景
- 示例
  - 用户
  - 用户
  - 示例:

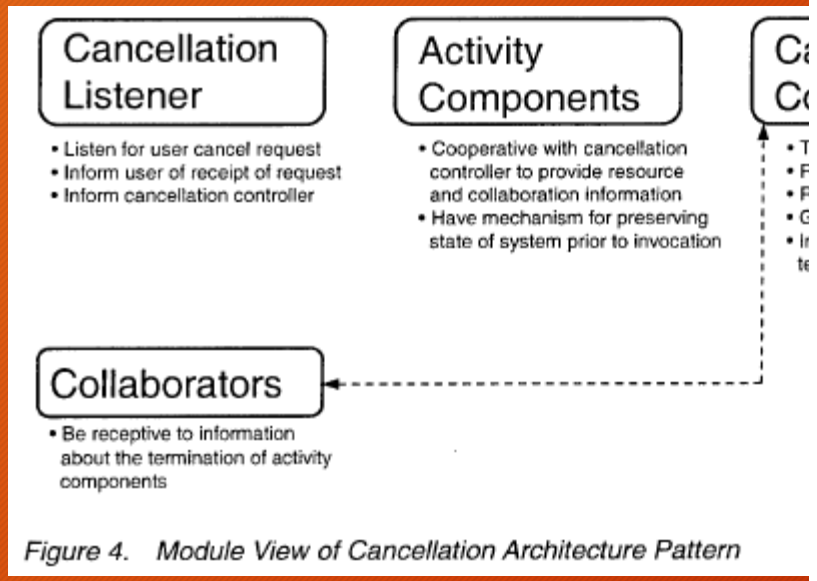


Figure 4. Module View of Cancellation Architecture Pattern

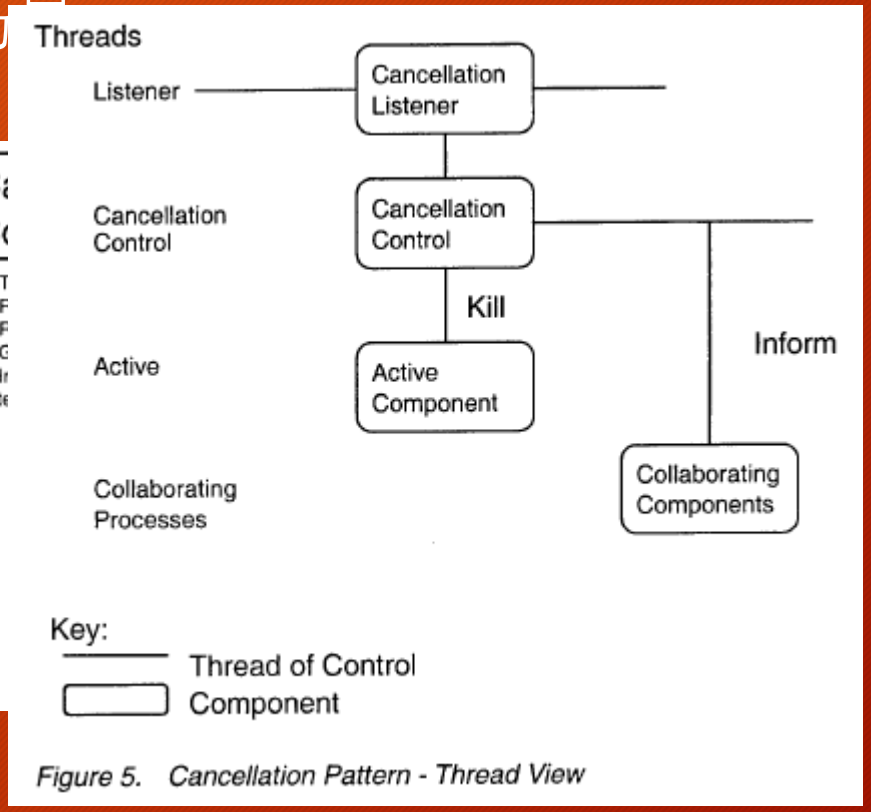


Figure 5. Cancellation Pattern - Thread View

Critère bénéfice pour l'utilisabilité

- **Les critères**

Increases individual user effectiveness

Expedites routine performance

Accelerates error-free portion of routine performance

Reduces the impact of routine user errors (slips)

Improves non-routine performance

Supports problem-solving

Facilitates learning

Reduces the impact of user errors caused by lack of knowledge (mistakes)

Prevents mistakes

Accommodates mistakes

Reduces the impact of system errors

Prevents system errors

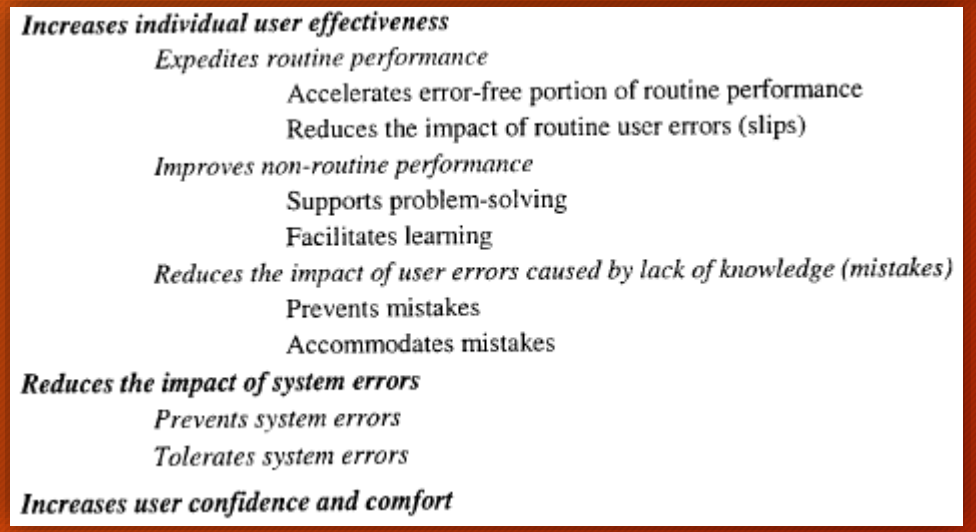
Tolerates system errors

Increases user confidence and comfort

- **Ex: Reduces the impact of routine user errors (slips)**
  - The negative impact of routine user errors can be reduced in two ways. First, since users will always slip, reducing the number of opportunities for error (roughly corresponding to the number and difficulty of steps in a given procedure) will usually reduce its occurrence. Second, systems can be designed to better accommodate user slips by providing adequate recovery methods

可用性效益标准

- **筛选标准**

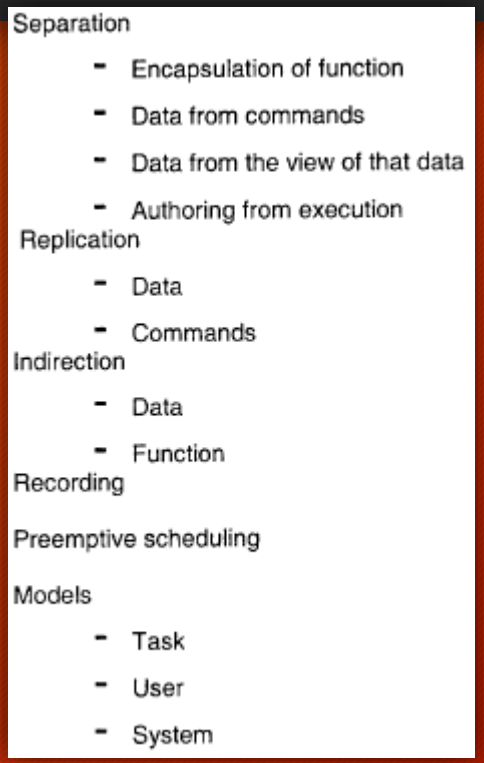


- **例如：可降低常规用户操作失误（滑移）带来的影响**
  - 常规用户操作失误带来的负面影响可通过以下两种方式予以缓解：首先，由于用户难免会出现操作失误，减少出错机会（大致对应于特定操作流程中步骤的数量与难度）通常可降低此类失误的发生率；其次，可通过提供充足的恢复机制，将系统设计得更能应对用户操作失误。

Critères mécanismes logiciels

- **Les critères**

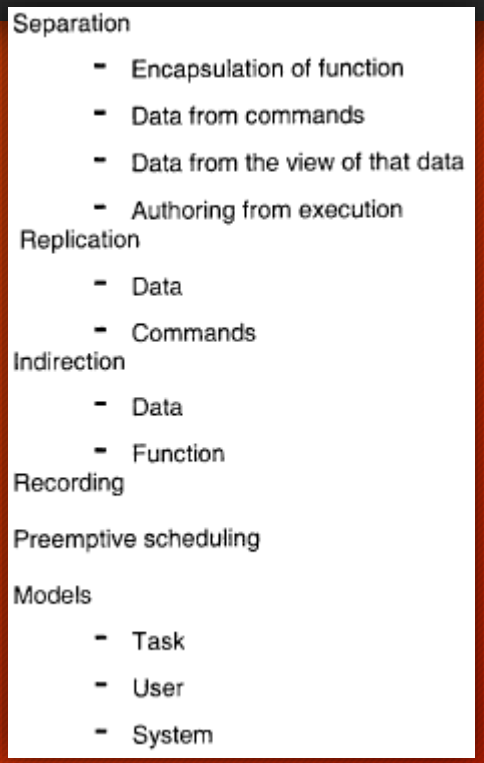
- **Ex: Preemptive Scheduling**
  - Preemptive scheduling allows the software system to have multiple simultaneous activities. In fact, the activities are not simultaneous when examined at a tiny time scale (measured in terms of microseconds) but they appear simultaneous when examined at a larger time scale (measured in terms of 10s of milliseconds). The term thread refers to a logical sequence of activities within the computer system. At any point in time, a thread is either active (consuming the processor resource) or blocked (waiting for a resource or for some input). Having multiple simultaneous activities is expressed as having multiple threads. Having multiple threads is most often accomplished (although not exclusively) by using a preemptive processor scheduling strategy



软件机制评估标准

- **筛选标准**

- **示例：抢占式调度**
  - 抢占式调度使软件系统能够同时执行多个活动。事实上，若以极短的时间尺度（以微秒为单位）观察，这些活动并非同时发生；但若以较宽的时间尺度（以数十毫秒为单位）观察，它们则看似同时发生。术语“线程”是指计算机系统内一系列逻辑上的活动序列。在任何给定时刻，一个线程要么处于活动状态（占用处理器资源），要么处于阻塞状态（等待某项资源或特定输入）。同时存在多个活动可表述为存在多个线程。实现多线程机制最常用的方式（尽管并非唯一方式）是采用抢占式处理器调度策略。



Résultat : Classification

Usability Benefits → / Architectural Mechanisms ↓		Increases individual effectiveness: Expedites routine performance — Accelerates error-free portion	Increases individual effectiveness: Expedites routine performance — Reduces impact of slips	Increases individual effectiveness: Improves non-routine performance — Supports problem-solving	Increases individual effectiveness: Improves non-routine performance — Facilitates learning	Increases individual effectiveness: Reduces impact of mistakes — Prevents mistakes	Increases individual effectiveness: Reduces impact of mistakes — Accommodates mistakes	Reduces impact of system errors — Tolerates system errors	Reduces impact of system errors — Prevents system errors	Increases confidence and comfort
Separation	Encapsulation of function	4, 13, 14, 15 20, 23		4, 13, 20	4, 13, 20	4, 13, 20	9, 14		23	
	Data from the view of that	12, 13, 24, 25	12	12, 13, 22, 24, 25, 26	12, 13, 24	12, 13, 22, 24	12			12
	Data from commands	1, 24, 25	5, 17	5, 17, 24, 25, 26	5, 17, 24	1, 5, 17, 24	1, 5, 17			17
	Authoring from execution	1, 2	2			1, 2	1, 2			
Replication	Data	16								
	Commands	2	2	22		2, 22	2			
Indirection	Data	7, 11, 14	11	7, 11			14			
	Function	6, 14, 20		6, 20	20	20	14		6	
Recording		2, 7	2, 3, 21	3, 7, 21		2	2, 3, 21	3, 8		
Preemptive Scheduling		15, 18, 19	3, 5, 17, 18	3, 5, 10, 17	5, 10, 17	5, 17, 19	3, 5, 17	3		17, 18
Models	Task	18, 19	5, 17, 18	5, 10, 17	5, 10, 17	5, 17, 19	5, 17			17, 18
	User	12, 18	5, 12, 17, 18	5, 10, 12, 17, 22	5, 10, 12, 17	5, 12, 17, 22	5, 12, 17			12, 17, 18
	System	4, 6, 19, 23	3, 5, 17	3, 4, 5, 6, 17	4, 5, 17	4, 5, 17, 19	3, 5, 17	3	6, 23	17

1. Aggregating data
2. Aggregating commands
3. Canceling commands
4. Using applications concurrently
5. Checking for correctness
6. Maintaining device independence
7. Evaluating the system
8. Recovering from failure
9. Retrieving forgotten passwords
10. Providing good help
11. Reusing information
12. Supporting international use
13. Levering Human Knowledge
14. Modifying interfaces
15. Supporting multiple activity
16. Navigating within a single view
17. Observing system state
18. Working at the user's pace
19. Predicting task duration
20. Supporting comprehensive searching
21. Supporting Undo
22. Working in an unfamiliar context
23. Verifying resources
24. Operating consistently across views
25. Making views accessible
26. Supporting visualization

结果：分类

Usability Benefits → / Architectural Mechanisms ↓		Increases individual effectiveness: Expedites routine performance — Accelerates error-free portion	Expedites routine performance — Reduces impact of slips	Improves non-routine performance — Supports problem-solving	Improves non-routine performance — Facilitates learning	Reduces impact of mistakes — Prevents mistakes	Reduces impact of mistakes — Accommodates mistakes	Reduces impact of system errors — Tolerates system errors	Reduces impact of system errors — Prevents system errors	Increases confidence and comfort
Separation	Encapsulation of function	4, 13, 14, 15 20, 23		4, 13, 20	4, 13, 20	4, 13, 20	9, 14		23	
	Data from the view of that	12, 13, 24, 25	12	12, 13, 22, 24, 25, 26	12, 13, 24	12, 13, 22, 24	12			12
	Data from commands	1, 24, 25	5, 17	5, 17, 24, 25, 26	5, 17, 24	1, 5, 17, 24	1, 5, 17			17
	Authoring from execution	1, 2	2			1, 2	1, 2			
Replication	Data	16								
	Commands	2	2	22		2, 22	2			
Indirection	Data	7, 11, 14	11	7, 11			14			
	Function	6, 14, 20		6, 20	20	20	14		6	
Recording		2, 7	2, 3, 21	3, 7, 21		2	2, 3, 21	3, 8		
Preemptive Scheduling		15, 18, 19	3, 5, 17, 18	3, 5, 10, 17	5, 10, 17	5, 17, 19	3, 5, 17	3		17, 18
Models	Task	18, 19	5, 17, 18	5, 10, 17	5, 10, 17	5, 17, 19	5, 17			17, 18
	User	12, 18	5, 12, 17, 18	5, 10, 12, 17, 22	5, 10, 12, 17	5, 12, 17, 22	5, 12, 17			12, 17, 18
	System	4, 6, 19, 23	3, 5, 17	3, 4, 5, 6, 17	4, 5, 17	4, 5, 17, 19	3, 5, 17	3	6, 23	17

1. Aggregating data
2. Aggregating commands
3. Canceling commands
4. Using applications concurrently
5. Checking for correctness
6. Maintaining device independence
7. Evaluating the system
8. Recovering from failure
9. Retrieving forgotten passwords
10. Providing good help
11. Reusing information
12. Supporting international use
13. Levering Human Knowledge
14. Modifying interfaces
15. Supporting multiple activity
16. Navigating within a single view
17. Observing system state
18. Working at the user's pace
19. Predicting task duration
20. Supporting comprehensive searching
21. Supporting Undo
22. Working in an unfamiliar context
23. Verifying resources
24. Operating consistently across views
25. Making views accessible
26. Supporting visualization

Avantages

- Avantages
  - Design rationale
  - Evaluation d’architecture existante
  - Evaluation du coût d’une évolution
- Limites
  - Evaluation du coût en termes de performance, sécurité, sûreté non faite

优势

- 优势
  - 设计原理
  - 现有架构评估
  - 演进成本评估
- 限制
  - 尚未就性能、安全性及可靠性方面进行成本评估。